

Computational Statistics, 1st Edition: Notes on Selected Topics

**For Instructional Purposes Only
No Commercial Use Allowed
Do Not Delete This Page**

Draft provided by: Geof H. Givens and Jennifer A. Hoeting

Alterations: The authors take no responsibility for changes made by subsequent users.

Omissions: These slides cover only a fraction of the 1st edition (2005).
Slides do not reflect new and changed material in the 2nd edition (2012).

**COMPUTATIONAL STATISTICS, SECOND EDITION (2012)
by G. H. Givens and J. A. Hoeting
now available everywhere**

Complete R code and datasets available online.

Book webpage: www.stat.colostate.edu/computationalstatistics/

Focus

Statisticians face many numerical challenges including optimization, integration, simulation, and smoothing.

Today's focus:

Integration using Monte Carlo methods and optimization of a real-valued function $g(\mathbf{x})$

Key uses in statistics:

Maximum likelihood, model fitting, solving nonlinear equations, Bayesian estimation, etc.

Computer languages:

Everyone has their favorite. We recommend R or Matlab. This course is language-independent. We focus on how and why methods work, not how to code them.

Part I: Optimization Methods

1. Optimizing smooth univariate functions (Chapter 2)

Bisection, Newton's method, Fisher scoring, secant method, scaled fixed point iteration

2. Optimizing smooth multivariate functions (Chapter 2)

Newton's method, Fisher scoring, Gauss-Newton method, ascent algorithms, discrete Newton method, scaled fixed point iteration, quasi-Newton methods, nonlinear Gauss-Seidel iteration

3. Combinatorial optimization (Chapter 3)

local search, ascent algorithms, random starts local search, tabu algorithms, simulated annealing, genetic algorithms

4. EM Optimization strategies (Chapter 4)

EM, SEM, MCEM, ECM, EM gradient, accelerated EM strategies

5. Concluding remarks

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration (Chapter 6)

Monte Carlo integration; variance reduction techniques: importance sampling, antithetic sampling, control variates; sampling importance resampling (SIR)

8. Markov Chain Monte Carlo Methods (Chapter 7)

Metropolis-Hastings algorithm, Gibbs sampling, implementation issues

9. Advanced MCMC Methods (Chapters 7 and 8)

Slice sampling and other auxiliary variable methods, Reversible jump MCMC, Perfect sampling, Langevin Metropolis-Hastings Algorithm, Hit-and-run algorithm, Multiple-try Metropolis-Hastings Algorithm

10. Integration: Concluding remarks

Introduction to optimization

The real-valued function $g(\mathbf{x})$ to be maximized may be a likelihood, a profile likelihood, a Bayesian posterior, or some other function.

For maximum likelihood estimation, g is the log likelihood function, l , and $\mathbf{x}$ is the corresponding parameter vector, $\boldsymbol{\theta}$. If $\hat{\boldsymbol{\theta}}$ is a MLE, it maximizes the log likelihood. Therefore $\hat{\boldsymbol{\theta}}$ is a solution to the **score equation**

$$\mathbf{l}'(\boldsymbol{\theta}) = \mathbf{0}, \tag{1}$$

where $\mathbf{l}'(\boldsymbol{\theta}) = \left(\frac{dl(\boldsymbol{\theta})}{d\theta_1}, \dots, \frac{dl(\boldsymbol{\theta})}{d\theta_n} \right)^T$ and $\mathbf{0}$ is a column vector of zeros.

Immediately, we see that optimization is intimately linked with solving nonlinear equations. **Finding a MLE amounts to finding a root of the score equation.** The maximum of g is a solution to $g'(\mathbf{x}) = \mathbf{0}$.

(Conversely, one may also turn a univariate root-finding exercise into an optimization problem by minimizing $|g'(x)|$ with respect to x , where g' is the function whose root is sought.)

Univariate Optimization for Smooth g

Example 1:

Maximize

$$g(x) = \frac{\log\{x\}}{1+x} \tag{2}$$

with respect to x .

We cannot find the root of $g'(x) = \frac{1+1/x-\log x}{(1+x)^2}$ analytically.

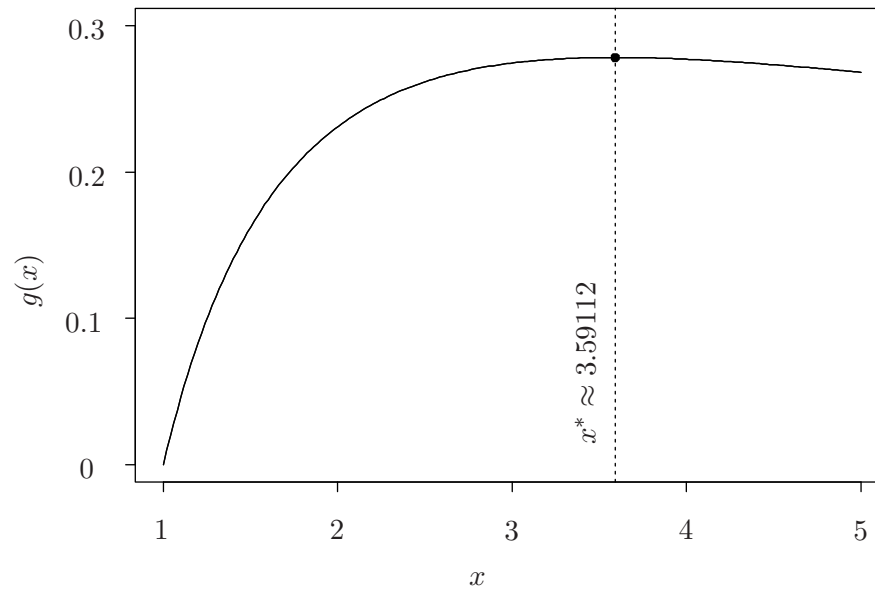


Figure 1: The maximum of $g(x) = \frac{\log\{x\}}{1+x}$ occurs at $x^* \approx 3.59112$, indicated by the vertical line.

Example 2:

The following data are an i.i.d. sample from a $\text{Cauchy}(\theta, 1)$ distribution:

1.77, -0.23, 2.76, 3.80, 3.47, 56.75, -1.34, 4.24, -2.44, 3.29, 3.71, -2.40, 4.53, -0.07, -1.05, -13.87, -2.53, -1.75, 0.27, 43.21.

The likelihood function is

$$\prod_{i=1}^{20} \frac{1}{\pi \left(1 + (x_i - \theta)^2\right)} \quad (3)$$

Find the MLE for θ .

The score function has multiple roots requiring numerical solution.

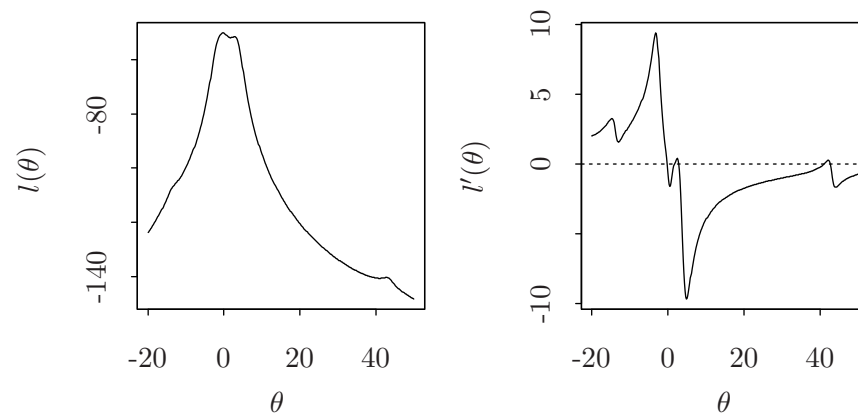


Figure 2: Log likelihood and score function for the Cauchy data.

Example 3:

Survival analysis can arise in many contexts:

time-to-death

time-to-recovery

time-until-failure

Data are **right-censored** because some experimental units have not yet “died”. Censoring often leads to difficult likelihoods.

Let the survival time, $t \sim f(t) = \alpha t^{\alpha-1} \exp \{ \mathbf{x}^T \boldsymbol{\beta} - t^\alpha \exp \{ \mathbf{x}^T \boldsymbol{\beta} \} \}$.

The *survivor function* is then $S(t) = 1 - F(t)$. The *hazard function* is $h(t) = f(t) / (1 - F(t))$, which measures the instantaneous risk of dying at time t given survival to time t .

A proportional hazards model posits that the hazard function depends on both time and a vector of covariates, $\mathbf{x}$, through the model

$$h(t|x) = \lambda(t) \exp \{ \mathbf{x}^T \boldsymbol{\beta} \}$$

where $\boldsymbol{\beta}$ is a parameter vector.

If $\Lambda(t) = \int_{-\infty}^t \lambda(u) du$, it is easy to show that $S(t) = \exp \{ -\Lambda(t) \exp \{ \mathbf{x}^T \boldsymbol{\beta} \} \}$.

Let w_i to be 1 if t_i is an uncensored time (“dead”) and 0 if t_i is a censored time (“still alive”). Then the log likelihood takes the form

$$\sum_{i=1}^n (w_i \log \{ \mu_i \} - \mu_i) + \sum_{i=1}^n w_i \log \left\{ \frac{\lambda(t_i)}{\Lambda(t_i)} \right\}$$

where $\mu_i = \Lambda(t_i) \exp \{ \mathbf{x}_i^T \boldsymbol{\beta} \}$, $\mathbf{x}_i^T \boldsymbol{\beta} = \beta_0 + \delta_i \beta_1$ where δ_i is 1 if the i th patient was in the treatment group and 0 otherwise, $\Lambda(t) = t^\alpha$ for $\alpha > 0$, and $h(t) \propto \alpha t^{\alpha-1}$.

The parameters are α , β_0 and β_1 . MLEs for the parameters must be obtained via numerical optimization in three dimensions.

Recall the simple example where we seek to maximize

$$g(x) = \frac{\log\{x\}}{1+x} \tag{4}$$

with respect to x .

We will rely on successive approximation of the solution. Figure 1 showed that the maximum is around 3. Therefore it might be reasonable to use $x^{(0)} = 3.0$ as an initial guess, or *starting value*. An *updating equation* will be used to produce an improved guess, $x^{(t+1)}$, from the most recent value $x^{(t)}$, for $t = 0, 1, 2, \dots$ until iterations are stopped.

Bisection Method

If g' is continuous on $[a_0, b_0]$ and $g'(a_0)g'(b_0) \leq 0$ then the intermediate value theorem implies that there exists at least one $x^* \in [a_0, b_0]$ for which $g'(x^*) = 0$ and hence x^* is a local optimum of g .

To find such a root, the bisection method systematically shrinks the interval from $[a_0, b_0]$ to $[a_1, b_1]$ to $[a_2, b_2]$ and so on, where $[a_0, b_0] \supset [a_1, b_1] \supset [a_2, b_2] \supset \dots$ and so forth. If these intervals are chosen to retain $g'(a_i)g'(b_i) \leq 0$, then the i th interval contains a root.

Let $x^{(0)} = (a_0 + b_0)/2$ be the starting value. The updating equations are

$$[a_{t+1}, b_{t+1}] = \begin{cases} [a_t, x^{(t)}] & \text{if } g'(a_t)g'(x^{(t)}) \leq 0 \\ [x^{(t)}, b_t] & \text{if } g'(a_t)g'(x^{(t)}) > 0 \end{cases} \quad (5)$$

and

$$x^{(t+1)} = (a_{t+1} + b_{t+1})/2. \quad (6)$$

If g has more than one root in the starting interval, it is easy to see that bisection will find one of them, but will not find the rest.

Example:

Optimizing

$$g(x) = \frac{\log\{x\}}{1+x} \tag{7}$$

To find the value of x maximizing (7), we might take $a_0 = 1$, $b_0 = 5$, and $x^{(0)} = 3$. Figure 3 illustrates the first few steps of the bisection algorithm.

For continuous smooth functions, bisection is guaranteed to converge to a root because a root is always in the interval and the length of the interval halves at each iteration. However, the method is slow.

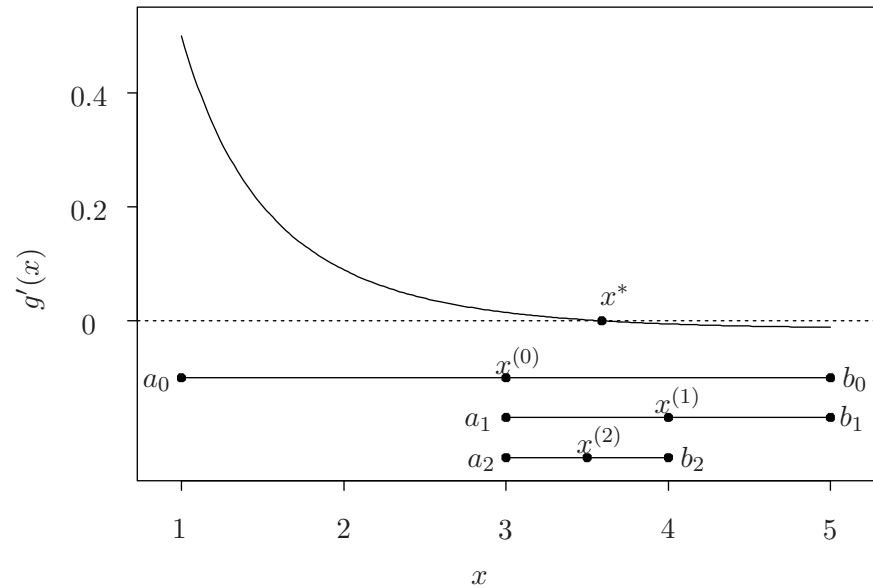


Figure 3: Illustration of the bisection method. The top portion of this graph shows $g'(x)$ and its root at x^* . The bottom portion shows the first three intervals obtained using the bisection method with $(a_0, b_0) = (1, 5)$. The t th estimate of the root is at the center of the t th interval.

Stopping Criteria

The *absolute convergence criterion* mandates stopping when

$$\left| x^{(t+1)} - x^{(t)} \right| < \epsilon, \quad (8)$$

where ϵ is a constant chosen to indicate tolerable imprecision.

The *relative convergence criterion* mandates stopping when iterations have reached a point for which

$$\frac{\left| x^{(t+1)} - x^{(t)} \right|}{\left| x^{(t)} \right|} < \epsilon. \quad (9)$$

This criterion enables the specification of a target precision (e.g., ‘within 1%’) without worrying about the units of x .

If the scale of x is huge (or tiny) relative to ϵ , an absolute convergence criterion may stop iterations too reluctantly (or too soon). The relative convergence criterion corrects for the scale of x , but can become unstable if $x^{(t)}$ values (or the true solution) lie too close to zero. In this latter case, another option is to monitor relative convergence by stopping when

$$\frac{\left| x^{(t+1)} - x^{(t)} \right|}{\left| x^{(t)} \right| + \epsilon} < \epsilon.$$

Also important to include stopping rules that flag a failure to converge:

- Stop after N iterations, regardless of convergence.

Do not devote all affordable iterations to one attempt! Budget time for many smaller attempts, anticipating convergence failures, data corrections, multiple starting values, etc.

- Could stop if any convergence measure fails to decrease or cycle over several iterations, or if the solution itself cycle unsatisfactorily.
- It is also sensible to stop if the procedure appears to be converging to a point at which $g(x)$ is inferior to another value you have already found (i.e., a known false peak or local maximum).

Regardless of which such stopping rules you employ, any indication of poor convergence behavior means that $x^{(t+1)}$ must be discarded and the procedure somehow restarted in a manner more likely to yield successful convergence.

Newton's Method

Suppose that g' is continuously differentiable and that $g''(x^*) \neq 0$. At iteration t , the approach approximates $g'(x^*)$ by the linear Taylor series expansion:

$$0 = g'(x^*) \approx g'(x^{(t)}) + (x^* - x^{(t)})g''(x^{(t)}). \quad (10)$$

Since g' is approximated by its tangent line at $x^{(t)}$, it seems sensible to approximate the root of g' by the root of the tangent line. Thus, solving for the root,

$$x^* \equiv x^{(t+1)} = x^{(t)} - \frac{g'(x^{(t)})}{g''(x^{(t)})} = x^{(t)} + h^{(t)} \quad (11)$$

When the optimization of g corresponds to a MLE problem where $\hat{\theta}$ is a solution to $l'(\theta) = 0$, the updating equation for Newton's method is

$$\theta^{(t+1)} = \theta^{(t)} - \frac{l'(\theta^{(t)})}{l''(\theta^{(t)})}. \quad (12)$$

Example:

For the simple function in (2),

$$h^{(t)} = \frac{(x^{(t)} + 1)(1 + 1/x^{(t)} - \log\{x^{(t)}\})}{3 + 4/x^{(t)} + 1/(x^{(t)})^2 - 2\log\{x^{(t)}\}}. \quad (13)$$

Figure 4 illustrates the first several iterations. Starting from $x^{(0)} = 3.0$, Newton's method quickly finds $x^{(4)} \approx 3.59112$. For comparison, the first five decimal places of x^* are not correctly determined by the bisection method until iteration 19.

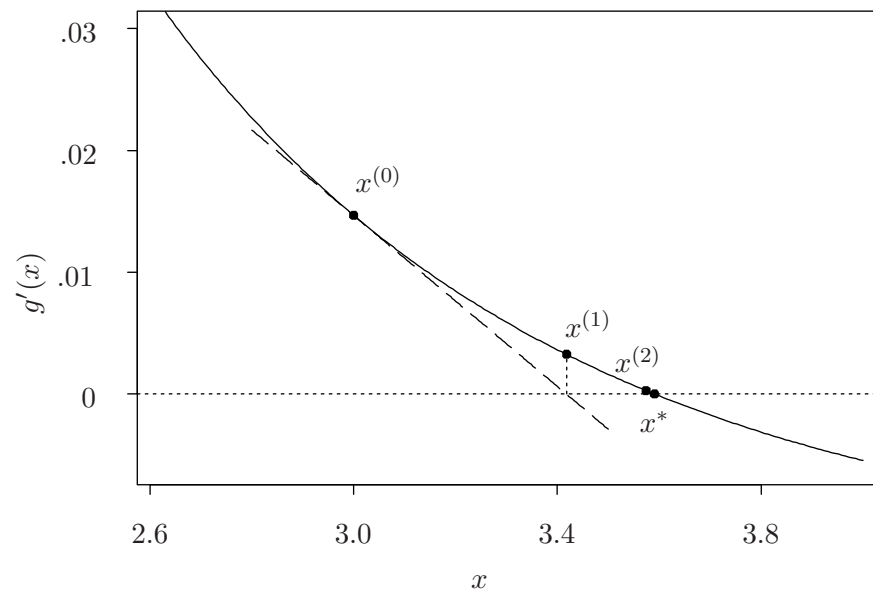


Figure 4: Illustration of Newton's method applied to maximize (2). At the first step, Newton's method approximates g' by its tangent line at $x^{(0)}$ whose root, $x^{(1)}$, serves as the next approximation of the true root, x^* . The next step similarly yields $x^{(2)}$, which is already quite close to the root at x^* .

Convergence

Define $\epsilon^{(t)} = x^{(t)} - x^*$

A method has *convergence of order* β if $\lim_{t \rightarrow \infty} \epsilon^{(t)} = 0$ and

$$\lim_{t \rightarrow \infty} \frac{|\epsilon^{(t+1)}|}{|\epsilon^{(t)}|^\beta} = c \quad (14)$$

for some constants $c \neq 0$ and $\beta > 0$.

Higher orders of convergence are better in the sense that precise approximation of the true solution is more quickly achieved. Unfortunately, high orders are sometimes achieved at the expense of robustness: some slow algorithms are more foolproof than their faster counterparts.

Newton's method has quadratic convergence order, $\beta = 2$

Speed is not the only factor to consider.

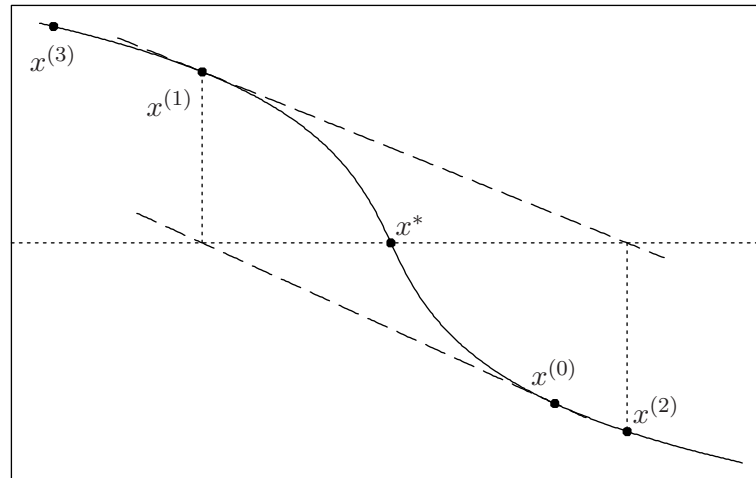


Figure 5: Starting from $x^{(0)}$, Newton's method diverges by taking steps that are increasingly distant from the true root, x^* .

Bisection would have found this root easily.

Starting values are critical. Cauchy example:

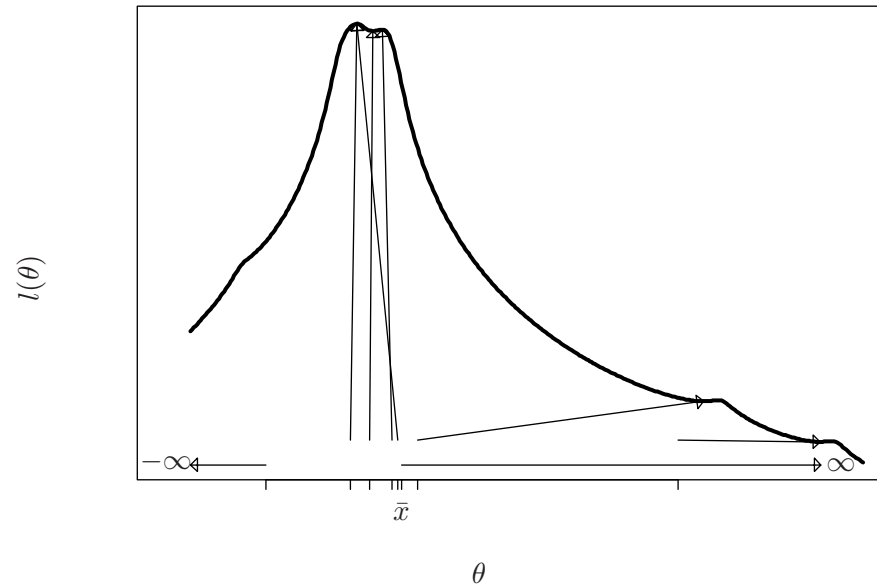


Figure 6: Log-likelihood for the Cauchy data. Arrows show convergence of Newton's method from several starting values.

Fisher Scoring

Fisher information is

$$\mathbf{I}(\boldsymbol{\theta}) = \mathbf{E}\{\mathbf{l}'(\boldsymbol{\theta})^2\} =^* -\mathbf{E}\{\mathbf{l}''(\boldsymbol{\theta})\} \quad (15)$$

*under regularity conditions.

Usually $I(\hat{\theta}) \approx -l''(\hat{\theta})$. This suggests using the increment $h^{(t)} = l'(\theta^{(t)})/I(\theta^{(t)})$ where $I(\theta^{(t)})$ is the Fisher information evaluated at $\theta^{(t)}$. This yields

$$\theta^{(t+1)} = \theta^{(t)} + l'(\theta^{(t)})I(\theta^{(t)})^{-1} \quad (16)$$

Fisher scoring and Newton's method share the same asymptotic properties; either may be easier for a particular problem. Generally, FS makes rapid improvements initially, while NM gives better refinements near the end.

Secant Method

When differentiating g' is difficult, we can replace the derivative by the discrete differenced approximation, $\frac{g'(x^{(t)}) - g'(x^{(t-1)})}{x^{(t)} - x^{(t-1)}}$.

This yields the update

$$x^{(t+1)} = x^{(t)} - g'(x^{(t)}) \frac{x^{(t)} - x^{(t-1)}}{g'(x^{(t)}) - g'(x^{(t-1)})} \quad (17)$$

for $t \geq 1$. Requires two starting points, $x^{(0)}$ and $x^{(1)}$.

Figure 7 illustrates the first steps of the method for maximizing the simple function in (2).

Order of convergence is superlinear: $\beta \approx 1.62$

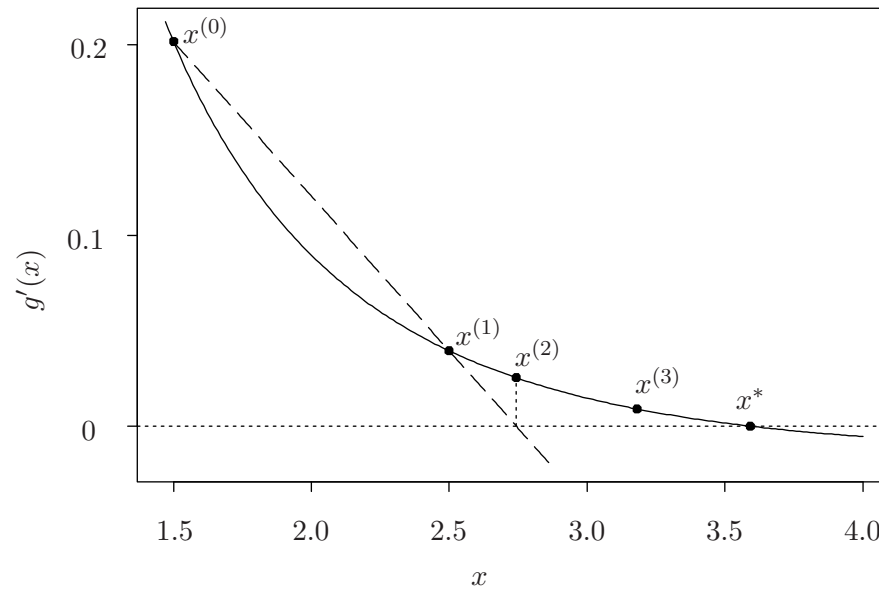


Figure 7: The secant method locally approximates g' using the secant line between $x^{(0)}$ and $x^{(1)}$. The corresponding estimated root, $x^{(2)}$, is used with $x^{(1)}$ to generate the next approximation.

(Scaled) Fixed Point Iteration

A fixed point of a function is a point whose evaluation by that function equals itself.

The fixed point strategy for finding roots is to determine a function G for which $g'(x) = 0$ if and only if $G(x) = x$. This transforms the problem of finding a root of g' into a problem of finding a fixed point of G . Then the simplest way to hunt for a fixed point is to use the updating equation

$$x^{(t+1)} = G(x^{(t)}) \quad (18)$$

Any suitable function G may be tried, but the most obvious choice is $G(x) = g'(x) + x$. This yields the updating equation

$$x^{(t+1)} = x^{(t)} + g'(x^{(t)}) \quad (19)$$

The convergence of this algorithm depends on whether G is *contractive*. (Roughly, G maps any interval into itself with bounded derivative; see refs)

We can often fix non-contractive functions by rescaling g or G !

For example, set $G(x) = \alpha g'(x) + x$ for a suitable α . Find α by trial and error or analysis of g'' .

Effectiveness of fixed point iteration also depends strongly on the chosen functional form of G .

Suppose the log likelihood is (nearly) quadratic. Then the score function is (nearly) locally linear, and l'' is roughly a constant, say γ . For a quadratic log likelihood, Newton's method would use the updating equation $\theta^{(t+1)} = \theta^{(t)} - l'(\theta)/\gamma$. If we use scaled fixed point iteration with $\alpha = -1/\gamma$, we get the same updating equation!

Many log likelihoods are approximately locally quadratic. Thus, scaled fixed point iteration can be very effective. It is also generally quite stable and easy to code.

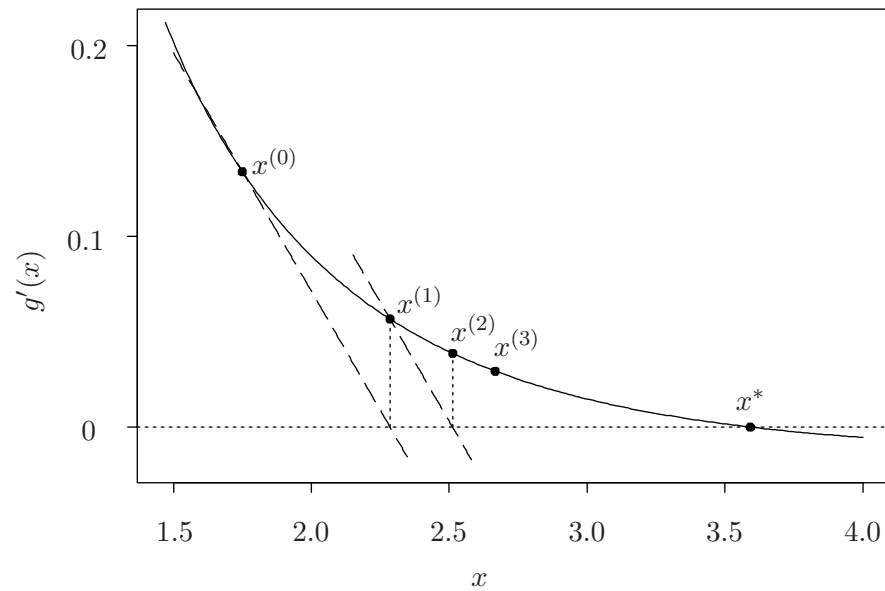


Figure 8: The first three steps of scaled fixed point iteration to maximize $g(x) = \frac{\log\{x\}}{1+x}$ using $G(x) = g'(x)+x$ and scaling with $\alpha = 4$.

Part I: Optimization Methods

1. Optimizing smooth univariate functions

2. Optimizing smooth multivariate functions

Newton's method, Fisher scoring, ascent algorithms, discrete Newton method, scaled fixed point iteration, quasi-Newton methods, Gauss-Newton method, nonlinear Gauss-Seidel iteration

3. Combinatorial optimization

4. EM Optimization strategies

5. Concluding remarks

Multivariate optimization for smooth g

Let $g : \mathbf{x} \in \mathbb{R}^p \rightarrow \mathbb{R}^1$

Can use analogous stopping criteria:

$$D(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}) < \epsilon, \quad \frac{D(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)})}{D(\mathbf{x}^{(t)}, \mathbf{0})} < \epsilon, \quad \text{or} \quad \frac{D(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)})}{D(\mathbf{x}^{(t)}, \mathbf{0}) + \epsilon} < \epsilon.$$

for $D(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^p |u_i - v_i|$ or $D(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^p (u_i - v_i)^2}$.

Same strategy of iterative approximation. We will extend previous methods and introduce new options.

Multivariate Newton's method and Fisher scoring

Multivariate Taylor series argument leads to the following updates:

Newton's method:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \mathbf{g}''(\mathbf{x}^{(t)})^{-1} \mathbf{g}'(\mathbf{x}^{(t)}). \quad (20)$$

Fisher scoring:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \mathbf{I}(\boldsymbol{\theta}^{(t)})^{-1} \mathbf{l}'(\boldsymbol{\theta}^{(t)}) \quad (21)$$

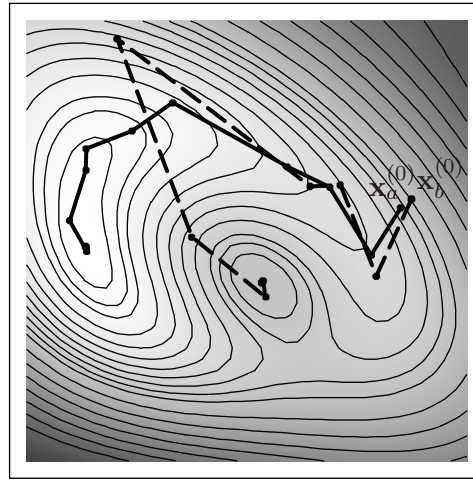


Figure 9: An application of Newton's method for maximizing a complex bivariate function. The surface of the function is indicated by shading and contours, with light shading corresponding to high values. Two runs starting from $\mathbf{x}_a^{(0)}$ and $\mathbf{x}_b^{(0)}$ are shown. These converge to the true maximum and to a local minimum, respectively.

Newton's method is not guaranteed to walk uphill. It is not guaranteed to find a local maximum. Step length matters even when step direction is good.

Newton-like methods

Some very effective methods rely on updating equations of the form

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - (\mathbf{M}^{(t)})^{-1} \mathbf{g}'(\mathbf{x}^{(t)}) \quad (22)$$

where $\mathbf{M}^{(t)}$ is a $p \times p$ matrix approximating the Hessian, $\mathbf{g}''(\mathbf{x}^{(t)})$.

- Want to avoid calculating Hessian if it is computationally expensive or analytically difficult
- Want to guarantee uphill steps

Ascent algorithms

Ascent algorithms: If we use the updating increment

$$\mathbf{h}^{(t)} = -\alpha^{(t)} [\mathbf{M}^{(t)}]^{-1} \mathbf{g}'(\mathbf{x}^{(t)}). \quad (23)$$

then any positive definite matrix $-\mathbf{M}^{(t)}$ will ensure ascent for a sufficiently small $\alpha^{(t)}$

Steepest ascent: $\mathbf{M}^{(t)} = -\mathbf{I}$. Thus $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \mathbf{g}'(\mathbf{x}^{(t)})$

Backtracking: Attempt a step for, say, $\alpha = 1$. If it is downhill, reduce (e.g., halve) α . If the step is still downhill, continue halving $\alpha^{(t)}$ until a sufficiently small step is found to be uphill.

$-\mathbf{M}^{(t)} = \mathbf{I}$ is positive definite, therefore backtracking with steepest ascent guarantees uphill steps.

$-\mathbf{M}^{(t)} = \mathbf{I}(\boldsymbol{\theta}^{(t)})$ is positive semi-definite. Therefore backtracking with Fisher scoring avoids stepping downhill.

Proof that backtracking works

For any fixed $\mathbf{x}^{(t)}$ and negative definite $\mathbf{M}^{(t)}$, note that as $\alpha^{(t)} \rightarrow 0$ we have

$$\begin{aligned} g(\mathbf{x}^{(t+1)}) - g(\mathbf{x}^{(t)}) &= g(\mathbf{x}^{(t)} + \mathbf{h}^{(t)}) - g(\mathbf{x}^{(t)}) \\ &= -\alpha^{(t)} \mathbf{g}'(\mathbf{x}^{(t)})^T (\mathbf{M}^{(t)})^{-1} \mathbf{g}'(\mathbf{x}^{(t)}) + o(\alpha^{(t)}), \end{aligned} \quad (24)$$

where the second equality follows from the linear Taylor expansion

$$g(\mathbf{x}^{(t)} + \mathbf{h}^{(t)}) = g(\mathbf{x}^{(t)}) + \mathbf{g}'(\mathbf{x}^{(t)})^T \mathbf{h}^{(t)} + o(\alpha^{(t)}).$$

Therefore if $-\mathbf{M}^{(t)}$ is positive definite, ascent can be ensured by choosing $\alpha^{(t)}$ sufficiently small, yielding

$$g(\mathbf{x}^{(t+1)}) - g(\mathbf{x}^{(t)}) > 0$$

from (24) since $o(\alpha^{(t)})/\alpha^{(t)} \rightarrow 0$ as $\alpha^{(t)} \rightarrow 0$.

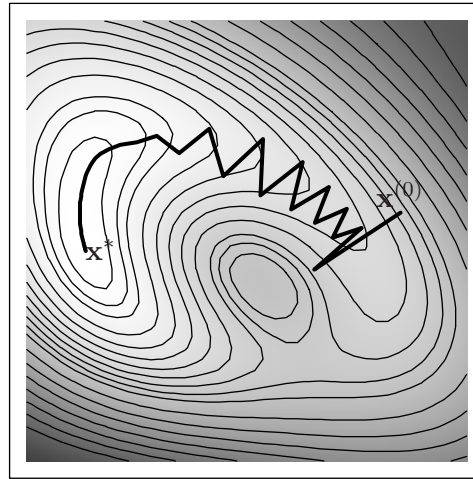


Figure 10: Steepest ascent with backtracking, using $\alpha = 0.25$ initially at each step.

The steepest ascent direction is not necessarily the wisest, and backtracking doesn't prevent oversteps.

Discrete Newton method

To avoid calculating the Hessian, one could resort to an analogue of the 1-dimensional secant method. For example,

$$\mathbf{M}_{ij}^{(t)} = \frac{g'_i(\mathbf{x}^{(t)} + h_{ij}^{(t)} \mathbf{e}_j) - g'_i(\mathbf{x}^{(t)})}{h_{ij}^{(t)}} \quad (25)$$

where $g'_i(\mathbf{x}) = dg(\mathbf{x})/dx_i$ is the i th element of $\mathbf{g}'(\mathbf{x})$, $\mathbf{e}_j$ is the p -vector with a 1 in the j th position and zeros elsewhere, and $h_{ij}^{(t)}$ are some constants.

$h_{ij}^{(t)} = h$ for all (i, j) and t leads to linear convergence order: $\beta = 1$.
Alternatively, $h_{ij}^{(t)} = x_j^{(t)} - x_j^{(t-1)}$ for all i gives superlinear convergence.

Multivariate fixed point iteration

Use an initial approximation of $\mathbf{g}''$ throughout the iterative updating. If this approximation is a matrix of constants, so $\mathbf{M}^{(t)} = \mathbf{M}$ for all t , then the updating equation is

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \mathbf{M}^{-1} \mathbf{g}'(\mathbf{x}^{(t)}). \quad (26)$$

A reasonable choice for $\mathbf{M}$ is $\mathbf{g}''(\mathbf{x}^{(0)})$.

If $\mathbf{M}$ is diagonal, then this amounts to applying the univariate scaled fixed point algorithm separately to each component of $\mathbf{g}$.

Quasi-Newton methods

The discrete Newton method strategy is computationally burdensome because $\mathbf{M}^{(t)}$ is wholly updated at every step.

A more efficient approach can be designed based on the direction of the most recent step. When $\mathbf{x}^{(t)}$ is updated to $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \mathbf{h}^{(t)}$, the opportunity is presented to learn about the curvature of $\mathbf{g}'$ in the direction of $\mathbf{h}^{(t)}$ near $\mathbf{x}^{(t)}$.

$\mathbf{M}^{(t)}$ can then be efficiently updated to incorporate this information.

$\mathbf{M}^{(t+1)}$ satisfies the secant condition if

$$\mathbf{g}'(\mathbf{x}^{(t+1)}) - \mathbf{g}'(\mathbf{x}^{(t)}) = \mathbf{M}^{(t+1)}(\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}). \quad (27)$$

Goal: generate $\mathbf{M}^{(t+1)}$ from $\mathbf{M}^{(t)}$ in a manner that requires few calculations and satisfies (27), while learning about the curvature of $\mathbf{g}'$ in the direction of the most recent step.

“Davidon’s update” is the simplest method that meets these requirements.

Let $\mathbf{z}^{(t)} = \mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}$ and $\mathbf{y}^{(t)} = \mathbf{g}'(\mathbf{x}^{(t+1)}) - \mathbf{g}'(\mathbf{x}^{(t)})$. Then we can write the update to $\mathbf{M}^{(t)}$ as

$$\mathbf{M}^{(t+1)} = \mathbf{M}^{(t)} + c^{(t)} \mathbf{v}^{(t)} (\mathbf{v}^{(t)})^T \quad (28)$$

where $\mathbf{v}^{(t)} = \mathbf{y}^{(t)} - \mathbf{M}^{(t)} \mathbf{z}^{(t)}$ and $c^{(t)} = \frac{1}{(\mathbf{v}^{(t)})^T \mathbf{z}^{(t)}}$.

Another well-known method is the “BFGS update”

$$\mathbf{M}^{(t+1)} = \mathbf{M}^{(t)} - \frac{\mathbf{M}^{(t)} \mathbf{z}^{(t)} (\mathbf{M}^{(t)} \mathbf{z}^{(t)})^T}{(\mathbf{z}^{(t)})^T \mathbf{M}^{(t)} \mathbf{z}^{(t)}} + \frac{\mathbf{y}^{(t)} (\mathbf{y}^{(t)})^T}{(\mathbf{z}^{(t)})^T \mathbf{y}^{(t)}} \quad (29)$$

Davidon’s update does not confer **hereditary positive definiteness**; BFGS does.

Backtracking is normally used. Monitoring of $c^{(t)}$ is required to maintain numerical stability.

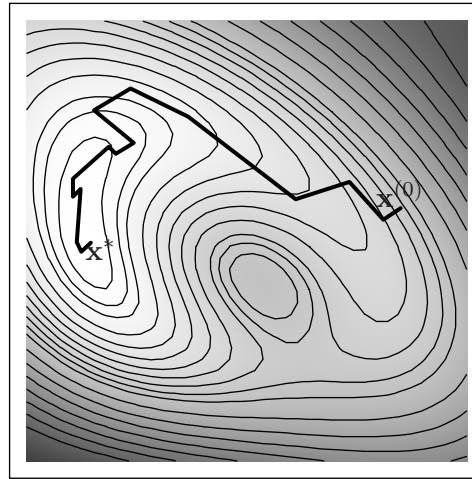


Figure 11: Quasi-Newton optimization with the BFGS update and backtracking to ensure ascent.

Convergence of quasi-Newton methods is generally superlinear, but not quadratic. These are powerful and popular methods, available, for example, in the R function `optim()`.

Gauss-Newton method

For *nonlinear least squares* problems with observed data $(y_i, \mathbf{z}_i)$ for $i = 1, \dots, n$ and model

$$Y_i = f(\mathbf{z}_i, \boldsymbol{\theta}) + \epsilon_i \quad (30)$$

for some non-linear function, f , and random error, ϵ_i .

We seek to estimate $\boldsymbol{\theta}$ by maximizing an objective function

$$g(\boldsymbol{\theta}) = - \sum_{i=1}^n (y_i - f(\mathbf{z}_i, \boldsymbol{\theta}))^2. \quad (31)$$

Newton's method approximates g via Taylor series. The Gauss-Newton approach approximates f itself by its linear Taylor series expansion about $\boldsymbol{\theta}^{(t)}$, so

$$Y_i \approx f(\mathbf{z}_i, \boldsymbol{\theta}^{(t)}) + (\boldsymbol{\theta} - \boldsymbol{\theta}^{(t)})^T \mathbf{f}'(\mathbf{z}_i, \boldsymbol{\theta}^{(t)}) + \epsilon_i = \tilde{f}(\mathbf{z}_i, \boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}) + \epsilon_i \quad (32)$$

where for each i , $\mathbf{f}'(\mathbf{z}_i, \boldsymbol{\theta}^{(t)})$ is the column vector of partial derivatives of f with respect to $\theta_j^{(t)}$, for $j = 1, \dots, p$, evaluated at $(\mathbf{z}_i, \boldsymbol{\theta}^{(t)})$.

Now maximize

$$\tilde{g}(\boldsymbol{\theta}) = - \sum_{i=1}^n \left(y_i - \tilde{f}(\mathbf{z}_i, \boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}) \right)^2 \quad (33)$$

with respect to $\boldsymbol{\theta}$.

Let $X_i^{(t)}$ denote a working response whose observed value is $x_i^{(t)} = y_i - f(\mathbf{z}_i, \boldsymbol{\theta}^{(t)})$, and define $\mathbf{a}_i^{(t)} = \mathbf{f}'(\mathbf{z}_i, \boldsymbol{\theta}^{(t)})$. Then the approximated problem can be re-expressed as minimizing the squared residuals of the linear regression model

$$\mathbf{X}^{(t)} = \mathbf{A}^{(t)}(\boldsymbol{\theta} - \boldsymbol{\theta}^{(t)}) + \boldsymbol{\epsilon} \quad (34)$$

where $\mathbf{X}^{(t)}$ and $\boldsymbol{\epsilon}$ are column vectors whose i th elements consist of $X_i^{(t)}$ and ϵ_i , respectively. Similarly, $\mathbf{A}^{(t)}$ is a matrix whose i th row is $(\mathbf{a}_i^{(t)})^T$.

This is a regression problem! Thus,

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \left((\mathbf{A}^{(t)})^T \mathbf{A}^{(t)} \right)^{-1} (\mathbf{A}^{(t)})^T \mathbf{x}^{(t)}. \quad (35)$$

Requires no computation of Hessian. Works best when the model fits fairly well and f is not severely nonlinear.

Example: Fitting a population model

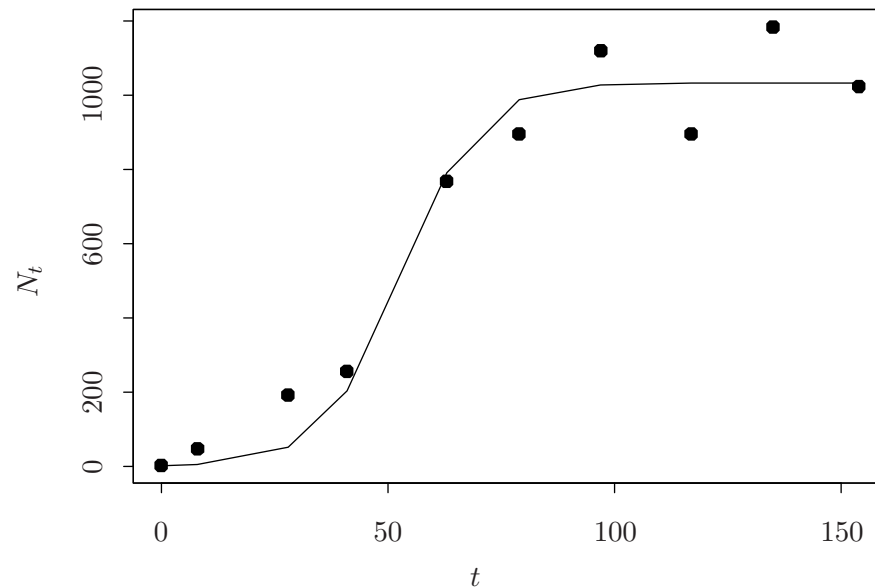


Figure 12: Population growth of flour beetles, and a logistic model fit.

An elementary model for population growth is the logistic model given by

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right) \quad (36)$$

where N is population size, t is time, r is a growth rate parameter, and K is a parameter that represents the population carrying capacity of the environment.

Thus

$$N_t = f(t) = \frac{K N_0}{N_0 + (K - N_0) \exp\{-rt\}} \quad (37)$$

where N_t denotes the population size at time t .

1. Compute $f'(t)$ at each t using the current guesses for K and r , thereby forming A .
2. Compute working responses as the residuals between N_t and the model predictions given the current guesses for K and r .
3. Regress the working responses on A , providing updated estimates for K and r .
4. Iterate.

Gauss-Seidel iteration

Also called *backfitting* or *cyclic coordinate ascent*. One key application is for fitting additive models, GAMs, etc.

The equation $\mathbf{g}'(\mathbf{x}) = 0$ is a system of p nonlinear equations in p unknowns. For $j = 1, \dots, p$, Gauss-Seidel iteration proceeds by viewing the j th component of $\mathbf{g}'$ as a univariate real function of only x_j . Any convenient univariate optimization method can be used to solve for the one-dimensional root of $g'_j(x_j^{(t+1)}) = 0$. All p components are cycled through in succession, and at each stage of the cycle the most recent values obtained for each coordinate are used. At the end of the cycle, the complete set of most recent values constitutes $\mathbf{x}^{(t+1)}$.

Advantages:

1. Simplifies a potentially difficult problem
2. Solution of each univariate problem is easier and more stable
3. Often less clock time required due to simple computations

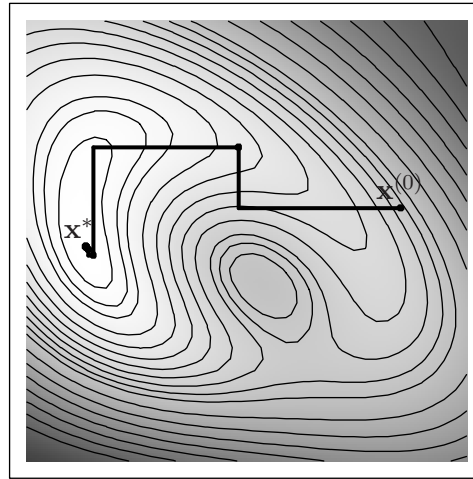


Figure 13: Gauss-Seidel iteration with univariate quasi-Newton sub-steps. Each line segment represents a change of a single coordinate in the current solution, so complete steps from $\mathbf{x}^{(t)}$ to $\mathbf{x}^{(t+1)}$ correspond to pairs of adjacent segments.

Note that the very first univariate optimization (one horizontal step left from $\mathbf{x}^{(0)}$) actually failed, finding a local univariate minimum instead of the global univariate maximum. Not advised, but subsequent Gauss-Seidel iterations overcame this misstep!

Example: Survival analysis

Recall the log likelihood:

$$l(\alpha, \beta_0, \beta_1) = \sum_{i=1}^n (w_i \log\{\mu_i\} - \mu_i) + \sum_{i=1}^n w_i \log\{\alpha/t_i\}$$

where:

$w_i = 1$ if t_i is uncensored (dead) and 0 if t_i is censored (still alive),
 $\mu_i = t_i^\alpha \exp\{\mathbf{x}_i^T \boldsymbol{\beta}\},$
 $\mathbf{x}_i^T \boldsymbol{\beta} = \beta_0 + \delta_i \beta_1$, and
 δ_i is 1 if the i th patient was in the treatment group and 0 otherwise

Table 1: Survival times for “patients” in the treatment and control groups, with parentheses indicating censored values. For censored cases, survival is known to exceed the indicated value.

Treatment	(6)	6	6	6	7	(9)	(10)
	10	(11)	13	16	(17)	(19)	(20)
	22	23	(25)	(32)	(32)	(34)	(35)
Control	1	1	2	2	3	4	4
	5	5	8	8	8	8	11
	11	12	12	15	17	22	23

Take partial derivatives of $l(\alpha, \beta_0, \beta_1)$:

$$\frac{dl}{d\alpha} = \sum_{i=1}^n w_i \log\{t_i\} - \sum_{i=1}^n t_i^\alpha \log\{t_i\} \exp\{\beta_0 + \beta_1 \delta_i\} + \sum_{i=1}^n w_i / \alpha \quad (38)$$

$$\frac{dl}{d\beta_0} = \sum_{i=1}^n w_i - \sum_{i=1}^n t_i^\alpha \exp\{\beta_0 + \beta_1 \delta_i\} \quad (39)$$

$$\frac{dl}{d\beta_1} = \sum_{i=1}^n w_i \delta_i - \sum_{i=1}^n \delta_i t_i^\alpha \exp\{\beta_0 + \beta_1 \delta_i\} \quad (40)$$

Cycle between (38), (39), and (40), finding the root of each using any simple univariate method (e.g. Newton) to solve for α , β_0 , or β_1 , respectively, using the most recent values of the other parameters.

Iteration, j	$\alpha^{(j)}$	$\beta_0^{(j)}$	$\beta_1^{(j)}$
0	1.00000000	0.00000000	1.00000000
1	0.13796362	-1.29854133	0.08054851
2	0.53687453	-1.68868929	-0.64029361
5	0.99751712	-2.20974308	-1.46890498
10	1.18513849	-2.62143475	-1.62495608
30	1.34838991	-3.02691913	-1.72092673
50	1.36370975	-3.06553576	-1.72970124
99	1.36575020	-3.07068572	-1.73086755
100	1.36575099	-3.07068767	-1.73086800

Part I: Optimization Methods

1. Optimizing smooth univariate functions

2. Optimizing smooth multivariate functions

3. Combinatorial optimization

Local search, ascent algorithms, random starts local search, tabu algorithms, simulated annealing, genetic algorithms

4. EM Optimization strategies

5. Concluding remarks

Combinatorial optimization

Assume we seek the maximum of $f(\boldsymbol{\theta})$ with respect to $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)$ where $\boldsymbol{\theta} \in \Theta$ and Θ consists of N elements for a finite positive integer N .

Θ is generally generated by combining or sequencing p items, which may be done in a very large number of ways.

In statistical applications, it is not uncommon for a likelihood function to depend on configuration parameters which describe the form of a statistical model and for which there are many discrete choices, as well as a small number of other parameters that are easily optimized if the best configuration were known. In such cases, we may view $f(\boldsymbol{\theta})$ as the log profile likelihood of a configuration, $\boldsymbol{\theta}$, i.e., the highest likelihood attainable using that configuration.

Hard optimization problems are generally combinatorial. We need search methods that don't require f to be smooth, continuous, or differentiable, and that can effectively search over vast discrete spaces.

Example: traveling salesman problem

The salesman must visit each of p cities exactly once and return to his point of origin, using the shortest total travel distance. We seek to minimize the total travel distance over all possible routes. There are $(p - 1)!/2$ possible routes (since the point of origin and direction of travel are arbitrary).

Any tour corresponds to a permutation of the integers $1, \dots, p$, which specifies the sequence in which the cities are visited.

Statisticians frequently face combinatorial problems where Θ has $p!$ or 2^p elements.

The traveling salesman problem is *NP-complete*. There is no known solution in $\mathcal{O}(p^k)$ steps for any k .

Consider two problems. The first can be solved in $\mathcal{O}(p^2)$ operations, and the second $\mathcal{O}(p!)$ operations. They both require 1 minute of computing time when $p = 20$.

Time to solve problem of order...		
p	$\mathcal{O}(p^2)$	$\mathcal{O}(p!)$
20	1 minute	1 minute
21	1.10 minutes	21 minutes
25	1.57 minutes	12.1 years
30	2.25 minutes	207 million years
50	6.25 minutes	2.4×10^{40} years

The big number exceeds a billion billion billion times longer than the age of the universe. Plenty of time to go down the hall for coffee.

This leads us to the realization that there are optimization problems that are inherently too difficult to solve exactly by traditional means. Many problems in bioinformatics, experimental design, and nonparametric statistical modeling, for example, require combinatorial optimization.

Example: Genetic mapping

A genetic map of p markers consists of an ordering of their loci and a list of distances or probabilities of recombination between adjacent loci.

The degree of linkage, or *map distance*, between two loci relates to the expected number of crossovers between the two loci.

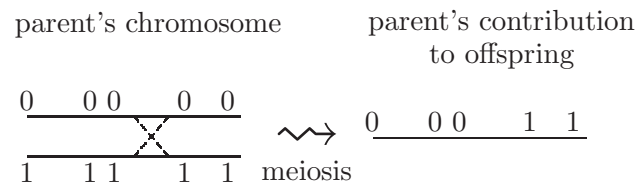


Figure 14: During meiosis, a crossover occurs between the third and fourth loci. The zeros and ones indicate the origin of each allele in the contributed chromosome. Only one parental contribution is shown for simplicity.

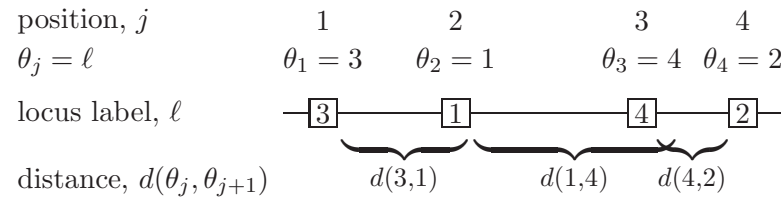


Figure 15: Notation for gene mapping example with $p = 4$ loci. The loci are labeled in boxes at their positions along the chromosome. The correct sequential ordering of loci is defined by the θ_j values. Distances between loci are given by $d(\theta_j, \theta_{j+1})$ for $j = 1, \dots, 3$.

Assign to each locus a label, ℓ , for $\ell = 1, \dots, p$. The ordering component of the map, denoted $\theta = (\theta_1, \dots, \theta_p)$, describes the arrangement of the p locus labels in order of their positions along the chromosome, with $\theta_j = \ell$ if the locus labeled ℓ lies at the j th position along the chromosome. Thus, θ is a permutation of the integers $1, \dots, p$.

The other component of a genetic map is a list of distances between adjacent loci. Denote the probability of recombination between adjacent loci θ_j and θ_{j+1} as $d(\theta_j, \theta_{j+1})$. This amounts to the map distance between these loci.

Goal: estimate the map.

Data: n chromosomes generated during the meiosis from a parent that is heterozygous at all p loci.

Let X_{i,θ_j} denote the origin of the allele in locus labeled θ_j for the i th chromosome. The dataset consists of observations, x_{i,θ_j} .

A recombination for two adjacent markers has been observed in the i th case if $|x_{i,\theta_j} - x_{i,\theta_{j+1}}| = 1$, and no recombination has been observed if $|x_{i,\theta_j} - x_{i,\theta_{j+1}}| = 0$.

If recombination events are assumed to occur independently in each interval, the probability of a given map is

$$\prod_{j=1}^{p-1} \prod_{i=1}^n \left\{ (1 - d(\theta_j, \theta_{j+1})) (1 - |x_{i,\theta_j} - x_{i,\theta_{j+1}}|) + d(\theta_j, \theta_{j+1}) |x_{i,\theta_j} - x_{i,\theta_{j+1}}| \right\}. \quad (41)$$

Given an ordering θ , the MLEs for the recombination probabilities are

$$\hat{d}(\theta_j, \theta_{j+1}) = \frac{1}{n} \sum_{i=1}^n |x_{i,\theta_j} - x_{i,\theta_{j+1}}|. \quad (42)$$

Given $d(\theta_j, \theta_{j+1})$, the number of recombinations between the loci in positions j and $j + 1$ is $\sum_{i=1}^n |X_{i,\theta_j} - X_{i,\theta_{j+1}}|$, which has a $\text{Bin}(n, d(\theta_j, \theta_{j+1}))$ distribution.

Profile likelihood for θ : add the log likelihoods of the $p - 1$ sets of adjacent loci and replace each $d(\theta_j, \theta_{j+1})$ by its conditional maximum likelihood estimate $\hat{d}(\theta_j, \theta_{j+1})$. Yields

$$\begin{aligned}
 l(\theta | \hat{\mathbf{d}}(\theta)) &= \sum_{j=1}^{p-1} n \left\{ \hat{d}(\theta_j, \theta_{j+1}) \log\{\hat{d}(\theta_j, \theta_{j+1})\} \right. \\
 &\quad \left. + (1 - \hat{d}(\theta_j, \theta_{j+1})) \log\{1 - \hat{d}(\theta_j, \theta_{j+1})\} \right\} \\
 &\equiv \sum_{j=1}^{p-1} T(\theta_j, \theta_{j+1})
 \end{aligned} \tag{43}$$

where $T(\theta_j, \theta_{j+1})$ is defined to be zero if $\hat{d}(\theta_j, \theta_{j+1})$ is zero or one.

The maximum likelihood genetic map is obtained by maximizing (43) over all permutations θ .

This is a variant of the traveling salesman problem, where each genetic marker corresponds to a city and the distance between cities i and j is $T(i, j)$.

Example: Variable selection in regression

Given a dependent variable Y and a set of candidate predictors $x_1, x_2, \dots, x_p$, we must find the best model of the form $Y = \beta_0 + \sum_{j=1}^s \beta_{i_j} x_{i_j} + \epsilon$ where $\{i_1, \dots, i_s\}$ is a subset of $\{1, \dots, p\}$.

The goal is to use the Akaike information criterion (AIC) to select the best model (i.e., minimize AIC)

Variable selection problem requires an optimization over a space of 2^{p+1} possible models, since each variable and the intercept may be included or omitted.

Heuristic search

Abandon algorithms that are guaranteed to find the global maximum (under suitable conditions) but will never succeed within a practical time limit. Turn instead to algorithms that can find a good local maximum within tolerable time.

Explicit trade of global optimality for speed.

Local search

Relies on

- iterative improvement of a current candidate solution, and
- limitation of the search to a local neighborhood at any particular iteration.

$\theta^{(t)} \longrightarrow \theta^{(t+1)}$ is a *move or step*.

Definition of neighborhood, $\mathcal{N}(\boldsymbol{\theta}^{(t)})$

Keep it simple. E.g.: define $\mathcal{N}(\boldsymbol{\theta}^{(t)})$ by allowing k changes to the current candidate solution. This is a k -neighborhood, and any move is called a k -change.

For the gene mapping problem, suppose $\boldsymbol{\theta}^{(t)}$ is a current ordering of genetic markers. A simple 2-neighborhood is the set of all orderings that can be obtained by swapping the locations of only two markers on the chromosome whose order is $\boldsymbol{\theta}^{(t)}$.

In the regression model selection problem, suppose $\boldsymbol{\theta}^{(t)}$ is a binary vector coding the inclusion/exclusion of each potential predictor. A simple 1-neighborhood is the set of models that either add or omit one predictor from $\boldsymbol{\theta}^{(t)}$.

Simple Ascent: $\boldsymbol{\theta}^{(t+1)} = \text{any better } \boldsymbol{\theta} \text{ in } \mathcal{N}(\boldsymbol{\theta}^{(t)})$.

Steepest Ascent: $\boldsymbol{\theta}^{(t+1)} = \text{best } \boldsymbol{\theta} \text{ in } \mathcal{N}(\boldsymbol{\theta}^{(t)})$.

Random starts local search

- Select many starting points by stratified or completely at random sampling
- Run simple/steepest ascent from each point
- Choose best solution as final answer.

Works surprisingly well compared to more sophisticated local search techniques.

The random starts strategy can be overlaid on any optimization method to improve the odds of finding a globally competitive final answer, or even the global optimum.

Example: Regression with 27 predictors and 2^{27} possible models

Table 2: Random starts local search model selection results using 1-change steepest ascent. The bullets indicate inclusion of the corresponding predictor in each model selected.

	Predictors selected																										
Method	1	2	3	6	7	8	9	10	12	13	14	15	16	18	19	20	21	22	24	25	26	AIC					
LS (2,4)		•	•	•		•		•		•	•	•	•						•	•	•	−416.95					
S-Plus	•		•	•		•		•		•	•	•	•						•	•	•	−416.94					
LS (1)			•	•		•		•	•	•	•						•	•	•			−414.60					
LS (3)			•	•		•		•		•	•						•	•	•	•	•	−414.16					
LS (5)			•			•	•	•		•	•					•	•	•	•			−413.52					
Efroy.			•		•	•		•		•	•				•				•		•	−400.16					

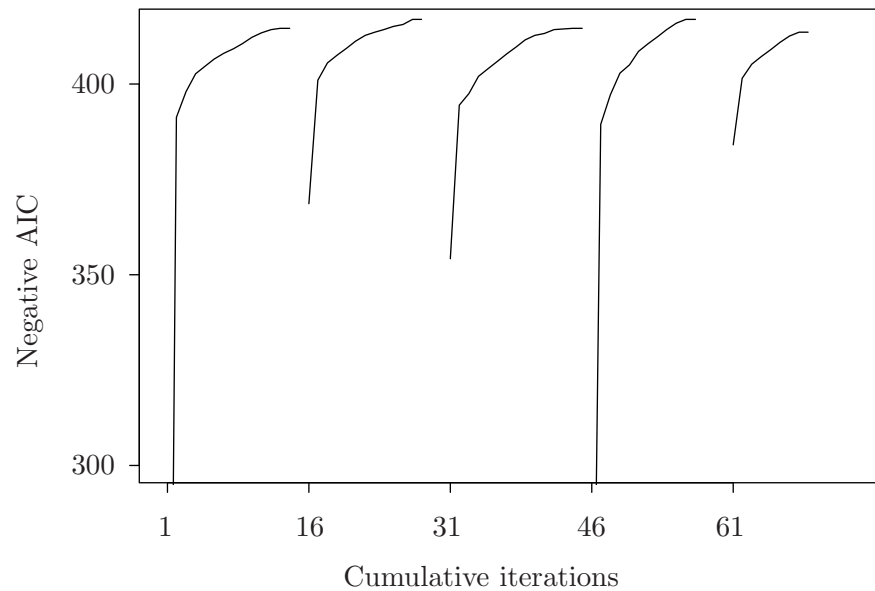


Figure 16: Results of random starts local search by 1-change steepest ascent for the regression example, for up to 15 iterations from each of five random starts. Only AIC values between -300 and -420 are shown.

The second and fourth starts found the best solution.

Tabu algorithms

Local search with variable neighborhoods and additional rules to promote the discovery of the global maximum.

For standard ascent algorithms, entrapment in a globally uncompetitive local maximum is likely because no downhill moves are allowed.

Tabu search allows downhill moves when no uphill move can be found in the current neighborhood (and possibly other situations too), thereby potentially escaping entrapment.

If a downhill step is chosen, care must be taken to ensure that the next step (or a future one) does not simply reverse the downhill move. Such cycling would eliminate the potential long-term benefit of the downhill move. To prevent such cycling, certain moves are temporarily forbidden, or made *tabu*, based on the recent history of the algorithm.

An early form of tabu search called steepest ascent/mildest descent moved to the least unfavorable neighbor when there was no uphill move.

Move attributes

A single step from $\theta^{(t)}$ to $\theta^{(t+1)}$ can be characterized by many *attributes*. Attributes will be used to describe moves or types of moves that will be forbidden, encouraged, or discouraged in future iterations of the algorithm.

Unlike simple local search, a tabu algorithm generates a neighborhood of the current candidate solution that depends on the search history; denote this $\mathcal{N}(\theta^{(t)}, H^{(t)})$. Furthermore, the identification of the preferred $\theta^{(t+1)}$ in $\mathcal{N}(\theta^{(t)}, H^{(t)})$ may depend not only on f but also on the search history. Thus, we may assess neighbors using an augmented objective function, $f_{H^{(t)}}$.

The *recency* of an attribute is the number of steps that have passed since a move most recently had that attribute. Let $R(A_a, H^{(t)}) = 0$ if the a th attribute is expressed in the move yielding $\theta^{(t)}$, $R(A_a, H^{(t)}) = 1$ if it is most recently expressed in the move yielding $\theta^{(t-1)}$, and so forth.

Examples of attributes

Table 3: Examples of attributes. The left column gives examples in a generic context. The right column gives corresponding attributes in the specific context of 2-change neighborhoods in a regression model selection problem.

Attribute	Model Selection Example
A change in the value of $\theta_i^{(t)}$. The attribute may be the value from which the move began, or the value to which it arrived.	A_1 : Whether the i th predictor is added (or deleted) from the model.
A swap in the values of $\theta_i^{(t)}$ and $\theta_j^{(t)}$ when $\theta_i^{(t)} \neq \theta_j^{(t)}$.	A_2 : Whether the absent variable is exchanged for the variable present in the model.
A change in the value of f resulting from the step, $f(\boldsymbol{\theta}^{(t+1)}) - f(\boldsymbol{\theta}^{(t)})$.	A_3 : The reduction in AIC achieved by the move.
The value of some other strategically chosen function, g , i.e., $g(\boldsymbol{\theta}^{(t+1)})$.	A_4 : The number of predictors in the new model.
A change in the value of g resulting from the step, $g(\boldsymbol{\theta}^{(t+1)}) - g(\boldsymbol{\theta}^{(t)})$.	A_5 : The change in a different variable selection criterion such as Mallows' C_p or adjusted R^2 .

The tabu list temporarily forbids some types of moves.

Each time a move with attribute A_a is taken, $\overline{A}_a$ is put on a tabu list for τ iterations.

$$\mathcal{N}(\boldsymbol{\theta}^{(t)}, H^{(t)}) = \left\{ \boldsymbol{\theta} : \boldsymbol{\theta} \in \mathcal{N}(\boldsymbol{\theta}^{(t)}) \text{ and no attribute of } \boldsymbol{\theta} \text{ is currently tabu} \right\}. \quad (44)$$

When $R(A_a, H^{(t)})$ first equals τ , the tabu expires and $\overline{A}_a$ is removed from the tabu list. This prevents undoing a downstep for τ iterations, thereby discouraging cycling. By the time that the tabu has expired, enough other aspects of the candidate solution should have changed that reversing the move may no longer be counterproductive.

Recommended tabu tenures: $7 \leq \tau \leq 20$, or $.5\sqrt{p} \leq \tau \leq 2\sqrt{p}$.

The tabu list is a list of attributes, not moves, so a single tabu attribute may forbid entire classes of moves.

More examples of tabus: genetic mapping example

Swap attribute: Suppose that A_a is the swap attribute corresponding to exchanging two particular loci along the chromosome. When a move A_a is taken, it is counterproductive to immediately undo the swap, so $\overline{A}_a$ is placed on the tabu list.

Strongest linkage attribute: Let A_a identify the two loci in the new chromosome that are nearest each other. If $\overline{A}_a$ is put on the tabu list, any move to a chromosome for which other loci are closer will be forbidden moves for τ iterations.

Step length attribute: Let A_a indicate whether the mean distance between adjacent loci has changed by at least c . Put A_a (not its complement!) on the tabu list to forbid any more drastic changes until the current region of solution space has been explored a bit.

Three other rule types:

Aspiration: Want to override the tabu when it is wise. Most common: permit a tabu move if it provides a higher value of the objective function than has been found in any iteration so far. “Aspiration by influence”: overrides the tabu of reversing a low-influence move if a high-influence move is made prior to the reversal. The recent high-influence step may have moved search to a new region where further local exploration is useful.

Diversification: Construct $f_{H^{(t)}}$ to penalize (bad) moves/attributes with high residence or transition frequency. Bad candidates with high residence frequency may correspond to entrapment. High transition frequency may correspond to “crack fillers”: these can fine tune good solutions but rarely offer major improvement. Example:

$$f_{H^{(t)}}(\boldsymbol{\theta}^{(t+1)}) = \begin{cases} f(\boldsymbol{\theta}^{(t+1)}) & \text{if } f(\boldsymbol{\theta}^{(t+1)}) \geq f(\boldsymbol{\theta}^{(t)}) \\ f(\boldsymbol{\theta}^{(t+1)}) - cF(A_a, H^{(t)}) & \text{if } f(\boldsymbol{\theta}^{(t+1)}) < f(\boldsymbol{\theta}^{(t)}) \end{cases} \quad (45)$$

Intensification: Construct $f_{H^{(t)}}$ to extend search near good solutions with high frequencies. Reward retention of attributes that correlate with high-quality high-frequency solutions; penalize removal of such attributes.

After initialization and identification of a list of problem-specific attributes, the algorithm proceeds as follows:

1. Construct $f_{H(t)}(\theta)$ based on $f(\theta)$, and perhaps diversification/intensification penalties or incentives.
2. Identify neighbors of $\theta^{(t)}$, namely the members of $\mathcal{N}(\theta^{(t)})$.
3. Rank the neighbors in decreasing order of improvement, as evaluated by $f_{H(t)}$.
4. Select the highest ranking neighbor.
5. Is this neighbor currently on the tabu list? If not, go to step 8.
6. Does this neighbor pass an aspiration criterion? If so, go to step 8.
7. If all neighbors of $\theta^{(t)}$ have been considered and none have been adopted as $\theta^{(t+1)}$, then stop. Otherwise, select the next best neighbor and go to step 5.
8. Adopt this solution as $\theta^{(t+1)}$ and update the tabu list.
9. Has a stopping criterion been met? If so, stop. Otherwise, increment t and go to step 1.

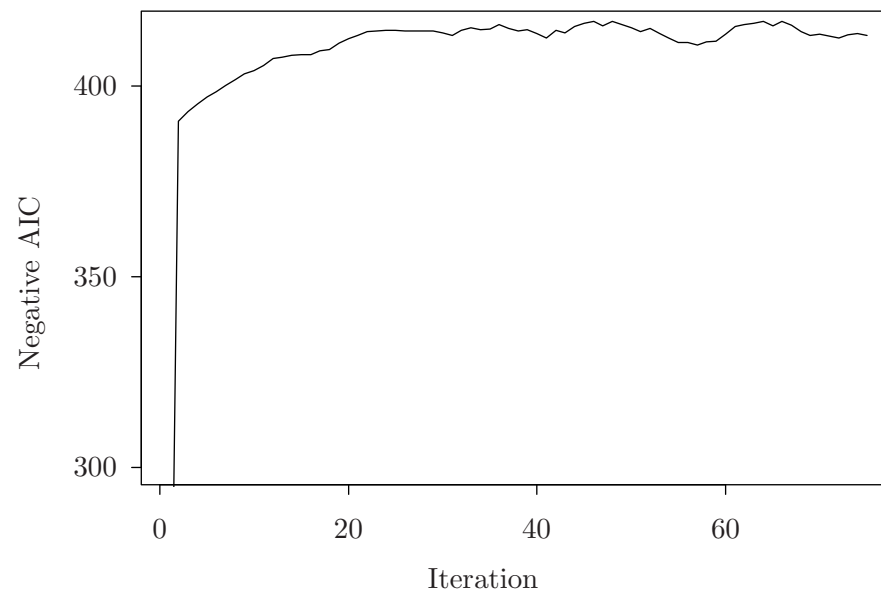


Figure 17: Results of tabu search for regression example.

Best solution was hit twice: iterations 44 and 66.

Simulated annealing

Motivated by analogy to the physics of slowly cooling solids.

Traditionally problems are posed as **minimizations**. We adopt this convention for this algorithm only.

Relies on a sequence of numbers, $\tau_0, \tau_1, \tau_2 \dots$, called **temperatures**. Temperature parameterizes how willing the algorithm should make non-improving steps.

Starts at time $t = 0$ with an initial point $\theta^{(0)}$ and a temperature τ_0 . The algorithm is run in stages indexed by $j = 0, 1, 2, \dots$, and each stage consists of several iterations. The length of the j th stage is m_j .

Simulated annealing algorithm:

1. Select a candidate solution θ^* within the neighborhood of $\theta^{(t)}$, say $\mathcal{N}(\theta^{(t)})$, according to a proposal density $g^{(t)}(\cdot \mid \theta^{(t)})$.
2. Randomly decide whether to adopt θ^* as the next candidate solution or to keep another copy of the current solution. Specifically, let $\theta^{(t+1)} = \theta^*$ with probability equal to $\min\left(1, \exp\left\{(f(\theta^{(t)}) - f(\theta^*))/\tau_j\right\}\right)$. Otherwise, let $\theta^{(t+1)} = \theta^{(t)}$.
3. Repeat steps 1 and 2 a total of m_j times.
4. Increment j . Update $\tau_j = \alpha(\tau_{j-1})$ and $m_j = \beta(m_{j-1})$. Go to step 1.

$\alpha()$ should slowly decrease temperatures to zero.

$\beta()$ should increase stage lengths as temperatures drop.

The new candidate solution is always adopted when it is superior to the current solution, and sometimes adopted even when it is inferior. This is *stochastic descent* designed to enable escape from uncompetitive local minima.

Neighborhoods for simulated annealing:

Small and easily computed. E.g., in genetic mapping, swap any two adjacencies.

Neighborhood structure must allow all elements of Θ to *communicate* (any two candidate solutions must be connectable via a sequence of neighborhoods).

Common proposal distribution: discrete uniform over $\mathcal{N}(\theta^{(t)})$.

Cooling schedule and convergence:

Cooling should be slow.

Simulated annealing produces a Markov chain because $\theta^{(t+1)}$ depends only on $\theta^{(t)}$.

In principle, we would like to run the chain at each fixed temperature long enough that the Markov chain is approximately in its stationary distribution before the temperature is reduced.

Theoretical analysis of cooling schedules leads to “recommendations” requiring impracticably long run lengths, sometimes longer than exhaustive search

Simulated annealing can be viewed as producing a sequence of homogeneous Markov chains (one at each temperature) or a single inhomogeneous Markov chain (with temperature decreasing between transitions).

First fix temperature at τ . Suppose that proposing θ_i from $\mathcal{N}(\theta_j)$ has the same probability as proposing θ_j from $\mathcal{N}(\theta_i)$ for any pair of solutions θ_i and θ_j in Θ . Then the sequence of $\theta^{(t)}$ generated by simulated annealing is a Markov chain with stationary distribution $\pi_\tau(\theta) \propto \exp\{-f(\theta)/\tau\}$. This is the *Metropolis algorithm*.

Suppose there are M global minima and the set of these solutions is $\mathcal{M}$. Denote the minimal value of f on Θ as $f_{\min}$. Then the stationary distribution of the chain for a fixed τ is given by

$$\pi_{\tau}(\boldsymbol{\theta}_i) = \frac{\exp\{- (f(\boldsymbol{\theta}_i) - f_{\min})/\tau\}}{M + \sum_{j \notin \mathcal{M}} \exp\{- (f(\boldsymbol{\theta}_j) - f_{\min})/\tau\}} \quad (46)$$

for each $\boldsymbol{\theta}_i \in \Theta$.

Now, as $\tau \rightarrow 0$ from above, the limit of $\exp\{- (f(\boldsymbol{\theta}_i) - f_{\min})/\tau\}$ is 0 if $i \notin \mathcal{M}$ and 1 if $i \in \mathcal{M}$. Thus

$$\lim_{\tau \downarrow 0} \pi_{\tau}(\boldsymbol{\theta}_i) = \begin{cases} 1/M & \text{if } i \in \mathcal{M} \\ 0 & \text{otherwise.} \end{cases} \quad (47)$$

Practical advice on cooling schedules:

Popular approach: set $m_j = 1$ for all j and reduce temperature very slowly according to $\alpha(\tau_{j-1}) = \frac{\tau_{j-1}}{1+a\tau_{j-1}}$ for a small value of a .

A second option is to set $\alpha(\tau_{j-1}) = a\tau_{j-1}$ for $a < 1$ (usually $a \geq 0.9$). Increase stage lengths as temperatures decrease, using, e.g., $\beta(m_{j-1}) = bm_{j-1}$ for $b > 1$.

See book and references for fancier options.

Selection of the initial temperature, τ_0 , is usually problem dependent. Try using a positive τ_0 value so that $\exp\{(f(\boldsymbol{\theta}_i) - f(\boldsymbol{\theta}_j))/\tau_0\}$ is close to one for any pair of solutions $\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$ in Θ . This provides any point in the parameter space with a reasonable chance of being visited in early iterations of the algorithm.

Rule of thumb: larger temperature drops require longer runs after the drop.

Final advice: Long runs at high temperatures are unneeded. Decrease temperature rapidly at first but slowly thereafter.

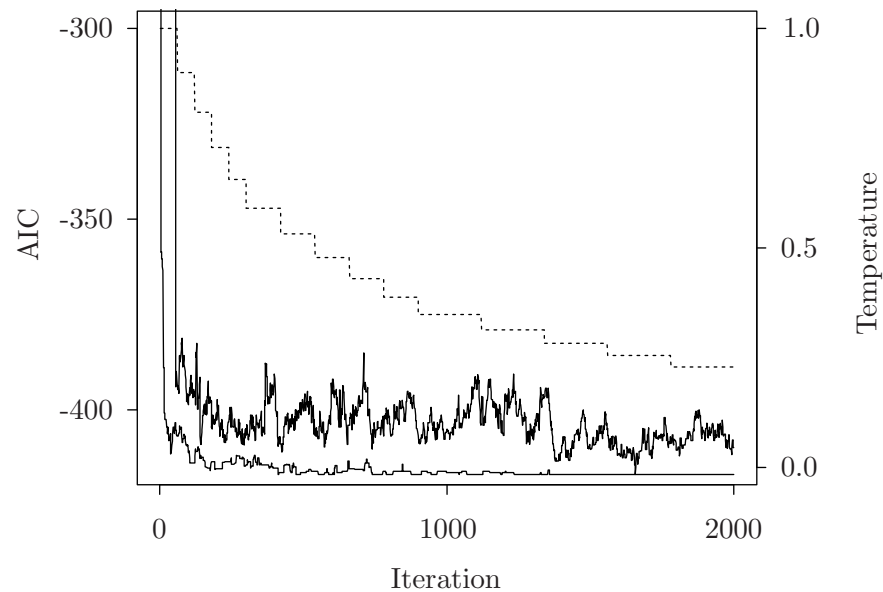


Figure 18: Results of two simulated annealing minimizations of regression model AIC for regression example. Temperature for the bottom curve is shown with the dotted line and the right axis. Neighborhoods given by adding or deleting one predictor. Discrete uniform proposal. 15-stage cooling schedule with stage lengths of 5×60 , 5×120 , and 5×220 . Cooling given by $\alpha(\tau_{j-1}) = 0.9\tau_{j-1}$ after each stage. The bottom curve corresponds to $\tau_0 = 1$, where we observe sticking because there is little tolerance for uphill moves. A second run with $\tau_0 = 10$ (top solid line) yielded considerable mixing, with many uphill proposals accepted as moves.

Genetic algorithms

Mimic the process of Darwinian natural selection.

Candidate solutions are envisioned as biological organisms represented by their genetic code.

The fitness of an organism is analogous to the quality of a candidate solution.

Breeding among highly fit organisms provides the best opportunity to pass along desirable attributes to future generations, while breeding among less fit organisms (and rare genetic mutations) ensures population diversity.

Over time, the organisms in the population should evolve to become increasingly fit, thereby providing a set of increasingly good candidate solutions to the optimization problem.

Basic definitions:

Every candidate solution corresponds to a *individual* or *organism*, and every organism is completely described by its chromosome.

A chromosome is a sequence of C symbols, each of which consists of a single choice from a pre-determined alphabet. E.g., '100110001'.

The C elements of the chromosome are the *genes*. The values that might be stored in a gene (i.e., the alphabet) are *alleles*. The position of a gene in the chromosome is its *locus*.

The information encoded in an individual's chromosome is its *genotype*, ϑ . The expression of the genotype in the organism itself is its *phenotype*, θ .

GA's consider many candidate solutions simultaneously. Let the t th *generation* consist of a collection of P organisms $\theta_1^{(t)}, \dots, \theta_P^{(t)}$ having corresponding genotypes (encodings) $\vartheta_1^{(t)}, \dots, \vartheta_P^{(t)}$.

The fitness of an organism (or its genotype) depends on the corresponding $f(\boldsymbol{\theta}_i^{(t)})$.

As generations progress, organisms inherit from their parents bits of genetic code that are associated with high fitness if fit parents are predominantly selected for breeding.

An *offspring* is a new organism inserted in the $(t + 1)$ th generation to replace a member of the t th generation. The offspring's chromosome is determined from those of two *parent* chromosomes belonging to the t th generation.

Example: regression model selection. Assume 9 predictors. Genotype $\boldsymbol{\vartheta}_i^{(t)} = \text{'100110001'}$ corresponds to the phenotype of a model containing only the fitted parameters for predictors 1, 4, 5, and 9. The fitness of this individual equals the negative AIC of this model.

Example: genetic mapping. Assume 9 genetic markers. Genotype $\boldsymbol{\vartheta}_i^{(t)} = \text{'752631948'}$ is called a *permutation chromosome*, and indicates a phenotype having the corresponding map ordering.

Selection mechanisms

Breeding drives most genetic change. The process by which parents are chosen to produce offspring is called the *selection mechanism*.

Simplest: Select one parent (or both) with probability proportional to fitness and select the other parent completely at random.

Better: Select with probability proportional to

$$\frac{2r_i}{P(P+1)} \quad (48)$$

where r_i is the rank of $f(\theta_i^{(t)})$ among generation t .

Popular: Tournament selection. Randomly partition the set of chromosomes in generation t into k disjoint subsets of equal size. Select the best individual in each group as a parent. Repeat to obtain sufficient parents. Finally, randomly pair parents for breeding.

Genetic operators

After two parents from the t th generation have been selected for breeding, their chromosomes are combined in some way that allows schemata from each parent to be inherited by their offspring, who become members of generation $t + 1$.

Crossover: Select a random position between two adjacent loci and split both parent chromosomes at this position. Glue the left chromosome segment from one parent to the right segment from the other parent to form an offspring chromosome.

Mutation: Randomly change one or more alleles in the chromosome. Usually applied after breeding (crossover).

Permutation chromosomes require more sophisticated crossover and mutation operators. See below.

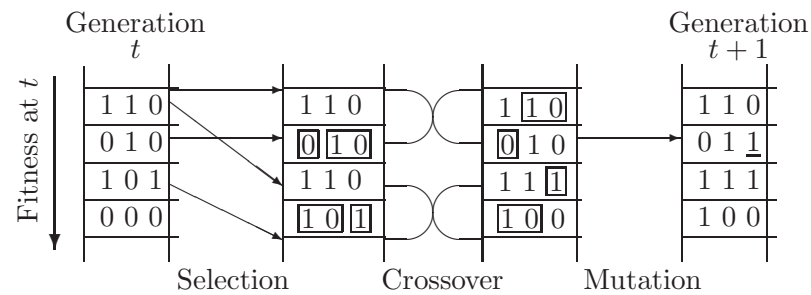


Figure 19: An example of generation production in a genetic algorithm for a population of size $P = 4$ with chromosomes of length $C = 3$. Crossovers are illustrated by boxing portions of some chromosomes. Mutation is indicated by an underlined gene in the final column.

Practical advice

Initial population: Purely random chromosomes.

Generation size: Larger provides greater search diversity, but also greater computing burden. For binary encoding of chromosomes, try $C \leq P \leq 2C$, where C is the chromosome length. For permutation chromosomes, try $2C \leq P \leq 20C$. Real applications often have $10 \leq P \leq 200$, but a review of empirical studies suggests that P can often be as small as 30.

Mutation rates: Typically very low, in the neighborhood of 1%. Studies have suggested a rate of $1/C$ or a rate roughly proportional to $1/(P\sqrt{C})$.

Steady state GA: Define the *generation gap*, G , to be the proportion of the generation to be replaced by generated offspring. $G = 1$ corresponds to a canonical genetic algorithm with distinct, non-overlapping generations. At the other extreme, $G = 1/P$ corresponds to incremental updating of the population one offspring at a time. Such a *steady state* GA genetic algorithm produces one offspring at a time to replace the least fit (or some random relatively unfit) individual.

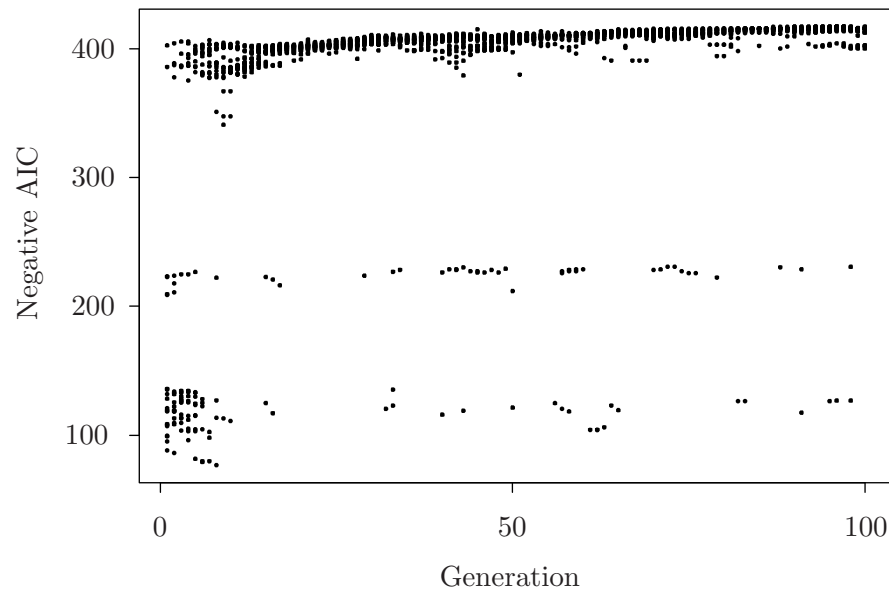


Figure 20: Results of a genetic algorithm for regression model selection, with 100 generations of size $P = 20$, using rank-based fitness, simple crossover, and 1% mutation.

Darwinian ‘survival of the fittest’ is clearly seen. The twenty random starting individuals quickly coalesce into three effective subspecies, with the best of these slowly overwhelming the rest. The best model was first found in generation 87.

Permutation chromosomes

Mutation is easy: swap two random alleles.

Crossover is hard:

Let $p = 9$ and consider crossover for the two parent chromosomes '752631948' and '912386754'.

Let the crossover point fall between the third and fourth loci, splitting the chromosomes as '752|631948' and '912|386754'.

Ordinary crossover would produce offspring '752386754' and '912631948'.

Both of these are invalid permutation chromosomes because both contain some duplicate alleles.

Order crossover

A random collection of loci is chosen, and the order in which the alleles in these loci appear in one parent is imposed on the same alleles in the other parent to produce one offspring.

Suppose parents are '752631948' and '912386754'. Suppose that the fourth, sixth, and seventh loci are randomly chosen.

In the first parent, the alleles in these loci are 6, 1, and 9. We must rearrange the 6, 1, and 9 alleles in the second parent to impose this order.

The remaining alleles in the second parent are '**238*754'. Inserting 6, 1, and 9 in this order yields '612389754' as the offspring.

The roles of the parents can be switched to produce a second offspring.

Edge recombination crossover

(Using traveling salesman metaphor...)

1. Construct a table that stores all the links that lead into and out of each city in either parent.
2. Choose the initial city from one parent, favoring the one with fewer links.
3. Note the onward links from this city.
4. Update the table to delete the chosen city everywhere in the table.
5. Select from among the choices in step 3, favoring the choice with fewest onward links in the updated table.
6. Update the table again and continue in this fashion.

Table 4: Edge tables showing the cities linked to or from each allele in either parent for each of the first three steps of edge recombination crossover. Beneath each column is the offspring chromosome resulting from each step.

Step 1		Step 2		Step 3	
City	Links	City	Links	City	Links
1	3, 9, 2	1	3, 2	1	3, 2
2	5, 6, 1, 3	2	5, 6, 1, 3	2	5, 6, 1, 3
3	6, 1, 2, 8	3	6, 1, 2, 8	3	6, 1, 2, 8
4	9, 8, 5	4	8, 5	4	Used
5	7, 2, 4	5	7, 2, 4	5	7, 2
6	2, 3, 8, 7	6	2, 3, 8, 7	6	2, 3, 8, 7
7	8, 5, 6	7	8, 5, 6	7	8, 5, 6
8	4, 7, 3, 6	8	4, 7, 3, 6	8	7, 3, 6
9	1, 4	9	Used	9	Used
‘9*****’		‘94*****’		‘945*****’	

Example for parents ‘752631948’ and ‘912386754’. The table shows the first few steps leading to ‘945*****’.

Continuing this process might yield the offspring ‘945786312’ by the following steps: select 7; select 8; select 6; randomly select 3 from the choices of 2 and 3; randomly select 1 from the choices of 1 and 2; select 2.

Part I: Optimization Methods

1. Optimizing smooth univariate functions

2. Optimizing smooth multivariate functions

3. Combinatorial optimization

4. EM Optimization strategies

EM, SEM, MCEM, ECM, EM gradient, accelerated EM strategies

5. Concluding remarks

EM Algorithm

An iterative optimization strategy motivated by a notion of missingness and by consideration of the conditional distribution of what is missing given what is observed.

Can be very simple to implement. Can reliably find an optimum through stable, uphill steps.

Difficult likelihoods often arise when data are missing. EM simplifies such problems. In fact, the ‘missing data’ may not truly be missing: they may be only a conceptual ploy to exploit the EM simplification!

Notation

X : Observed variables.

Z : Missing or latent variables.

Y : Complete data $Y = (X, Z)$.

In Bayesian settings, **X**, **Z**, and **Y** often refer to sets of parameters, rather than data.

Suppose we seek to maximize $L(\boldsymbol{\theta}|\mathbf{x})$ with respect to $\boldsymbol{\theta}$.

Define $Q(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})$ to be the expectation of the joint log likelihood for the complete data, conditional on the observed data $\mathbf{X} = \mathbf{x}$. Namely

$$Q(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) = \mathbb{E} \left\{ \log L(\boldsymbol{\theta}|\mathbf{Y}) \mid \mathbf{x}, \boldsymbol{\theta}^{(t)} \right\} \quad (49)$$

$$= \mathbb{E} \left\{ \log f_{\mathbf{Y}}(\mathbf{y}|\boldsymbol{\theta}) \mid \mathbf{x}, \boldsymbol{\theta}^{(t)} \right\} \quad (50)$$

$$= \int [\log f_{\mathbf{Y}}(\mathbf{y}|\boldsymbol{\theta})] f_{\mathbf{Z}|\mathbf{X}}(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta}^{(t)}) d\mathbf{z} \quad (51)$$

where (51) emphasizes that $\mathbf{Z}$ is the only random part of $\mathbf{Y}$ once we are given $\mathbf{X} = \mathbf{x}$.

The EM Algorithm

Start with $\theta^{(0)}$. Then

1. E step: Compute $Q(\theta|\theta^{(t)})$.
2. M step: Maximize $Q(\theta|\theta^{(t)})$ with respect to θ . Set $\theta^{(t+1)}$ equal to the maximizer of Q .
3. Return to the E step unless a stopping criterion has been met.

Trivial example:

$Y_1, Y_2 \sim \text{i.i.d. Exp}(\theta)$ with $y_1 = 5$ observed but y_2 missing.

The complete data log likelihood function is

$$\log\{L(\theta|\mathbf{y})\} = \log\{f_{\mathbf{Y}}(\mathbf{y}|\theta)\} = 2\log\{\theta\} - \theta y_1 - \theta y_2. \quad (52)$$

Thus

$$Q(\theta|\theta^{(t)}) = 2\log\{\theta\} - 5\theta - \theta/\theta^{(t)} \quad (53)$$

since $E\{Y_2|y_1, \theta^{(t)}\} = E\{Y_2|\theta^{(t)}\} = 1/\theta^{(t)}$ follows from independence.

The maximizer of $Q(\theta|\theta^{(t)})$ is the root of $2/\theta - 5 - 1/\theta^{(t)} = 0$. Thus $\theta^{(t+1)} = \frac{2\theta^{(t)}}{5\theta^{(t)}+1}$.
Converges quickly to $\hat{\theta} = 0.2$.

The E step and M step do not need to be re-derived at each iteration

This example is not realistic. Easy analytic solution. Taking the required expectation is trickier in real applications because one needs to know the conditional distribution of the complete data given the missing data.

Example: Peppered moths

Wing color determined by a single gene with three possible alleles which we denote C, I, and T. C is dominant to I, and T is recessive to I. Thus genotypes CC, CI, and CT result in the *carbonaria* phenotype having solid black coloring. The TT genotype results in the *typica* phenotype with light-colored wings. The II and IT genotypes produce an intermediate *insularia* phenotype with mottled wings.

There are six possible genotypes, but only three phenotypes are measurable in field work.

In UK and USA, *carbonaria* nearly replaced the paler phenotypes in areas affected by coal-fired industries. This change in allele frequencies in the population is cited as an instance when we may observe micro-evolution occurring on a human time scale. The theory (supported by experiments) is that “differential predation by birds on moths that are variously conspicuous against backgrounds of different reflectance” induces selectivity that favors *carbonaria* in times/regions where sooty, polluted conditions reduce the reflectance of the surface of tree bark on which the moths rest. When improved environmental standards reduced pollution, prevalence of the lighter-colored phenotypes increased and *carbonaria* prevalence plummeted.

How do we estimate allele frequencies from phenotype counts?

Under Hardy-Weinberg, if the allele frequencies in the population are p_C , p_I , and p_T , then the genotype frequencies should be p_C^2 , $2p_Cp_I$, $2p_Cp_T$, p_I^2 , $2p_Ip_T$, and p_T^2 , for genotypes CC, CI, CT, II, IT, and TT, respectively.

Capture n moths, of which there are n_C , n_I , and n_T of the *carbonaria*, *insularia*, and *typica* phenotypes, respectively. Thus $n = n_C + n_I + n_T$. If we knew the genotype of each moth rather than merely its phenotype, we could generate genotype counts n_{CC} , n_{CI} , n_{CT} , n_{II} , n_{IT} , and n_{TT} . Allele frequencies could then be easily tabulated: each moth with genotype CI contributes one C allele and one I allele, whereas a II moth contributes two I alleles.

Observed data: $\mathbf{x} = (n_{\text{C}}, n_{\text{I}}, n_{\text{T}})$

Complete data: $\mathbf{y} = (n_{\text{CC}}, n_{\text{CI}}, n_{\text{CT}}, n_{\text{II}}, n_{\text{IT}}, n_{\text{TT}})$

Complete data log likelihood is multinomial:

$$\begin{aligned} \log\{f_{\mathbf{Y}}(\mathbf{y}|\mathbf{p})\} &= n_{\text{CC}} \log\{p_{\text{C}}^2\} + n_{\text{CI}} \log\{2p_{\text{C}}p_{\text{I}}\} + n_{\text{CT}} \log\{2p_{\text{C}}p_{\text{T}}\} \\ &\quad + n_{\text{II}} \log\{p_{\text{I}}^2\} + n_{\text{IT}} \log\{2p_{\text{I}}p_{\text{T}}\} + n_{\text{TT}} \log\{p_{\text{T}}^2\} \\ &\quad + \log \binom{n}{n_{\text{CC}} \ n_{\text{CI}} \ n_{\text{CT}} \ n_{\text{II}} \ n_{\text{IT}} \ n_{\text{TT}}} . \end{aligned} \quad (54)$$

Many-to-one mapping from $\mathbf{Y}$ to $\mathbf{X}$: $(n_{\text{C}}, n_{\text{I}}, n_{\text{T}}) = (n_{\text{CC}} + n_{\text{CI}} + n_{\text{CT}}, n_{\text{II}} + n_{\text{IT}}, n_{\text{TT}})$.

E-Step:

Conditional on n_C and a parameter vector $\mathbf{p}^{(t)} = (p_C^{(t)}, p_I^{(t)})$, the latent counts for the three *carbonaria* genotypes have a three-cell multinomial distribution with count parameter n_C and cell probabilities proportional to $(p_C^{(t)})^2$, $2p_C^{(t)}p_I^{(t)}$, and $2p_C^{(t)}p_T^{(t)}$.

Same reasoning for other genotypes.

Thus

$$E\{N_{CC}|n_C, n_I, n_T, \mathbf{p}^{(t)}\} = n_{CC}^{(t)} = \frac{n_C (p_C^{(t)})^2}{(p_C^{(t)})^2 + 2p_C^{(t)}p_I^{(t)} + 2p_C^{(t)}p_T^{(t)}} \quad (55)$$

and so forth.

Thus

$$Q(\mathbf{p}|\mathbf{p}^{(t)}) = n_{CC}^{(t)} \log\{p_C^2\} + n_{CI}^{(t)} \log\{2p_C p_I\} + n_{CT}^{(t)} \log\{2p_C p_T\} \\ + n_{II}^{(t)} \log\{p_I^2\} + n_{IT}^{(t)} \log\{2p_I p_T\} + n_{TT}^{(t)} \log\{p_T^2\} + k(n_C, n_I, n_T, \mathbf{p}^{(t)}). \quad (56)$$

M-Step:

Differentiating with respect to p_c and p_i yields

$$\frac{dQ(\mathbf{p}|\mathbf{p}^{(t)})}{dp_c} = \frac{2n_{cc}^{(t)} + n_{ci}^{(t)} + n_{ct}^{(t)}}{p_c} - \frac{2n_{tt}^{(t)} + n_{ct}^{(t)} + n_{it}^{(t)}}{1 - p_c - p_i} \quad (57)$$

$$\frac{dQ(\mathbf{p}|\mathbf{p}^{(t)})}{dp_i} = \frac{2n_{ii}^{(t)} + n_{it}^{(t)} + n_{ci}^{(t)}}{p_i} - \frac{2n_{tt}^{(t)} + n_{ct}^{(t)} + n_{it}^{(t)}}{1 - p_c - p_i}. \quad (58)$$

Setting these derivatives equal to zero and solving for p_c and p_i completes the M step, yielding

$$p_c^{(t+1)} = \frac{2n_{cc}^{(t)} + n_{ci}^{(t)} + n_{ct}^{(t)}}{2n} \quad (59)$$

$$p_i^{(t+1)} = \frac{2n_{ii}^{(t)} + n_{it}^{(t)} + n_{ci}^{(t)}}{2n} \quad (60)$$

$$p_t^{(t+1)} = \frac{2n_{tt}^{(t)} + n_{ct}^{(t)} + n_{it}^{(t)}}{2n}, \quad (61)$$

Observed data: $n_C = 85$, $n_I = 196$, and $n_T = 341$

Table 5: EM results for peppered moth example. $R^{(t)}$ is the relative convergence criterion; $D_C^{(t)}$, and $D_I^{(t)}$ are ratios of consecutive errors.

t	$p_C^{(t)}$	$p_I^{(t)}$	$R^{(t)}$	$D_C^{(t)}$	$D_I^{(t)}$
0	0.333333	0.333333			
1	0.081994	0.237406	5.7×10^{-1}	0.0425	0.337
2	0.071249	0.197870	1.6×10^{-1}	0.0369	0.188
3	0.070852	0.190360	3.6×10^{-2}	0.0367	0.178
4	0.070837	0.189023	6.6×10^{-3}	0.0367	0.176
5	0.070837	0.188787	1.2×10^{-3}	0.0367	0.176
6	0.070837	0.188745	2.1×10^{-4}	0.0367	0.176
7	0.070837	0.188738	3.6×10^{-5}	0.0367	0.176
8	0.070837	0.188737	6.4×10^{-6}	0.0367	0.176

The nature of EM

Ascent: Each M-step increases the log likelihood.

Convergence: is linear (slow!). Rate is inversely related to the proportion of missing data.

Optimization transfer:

$$l(\boldsymbol{\theta}|\mathbf{x}) \geq Q(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) + l(\boldsymbol{\theta}^{(t)}|\mathbf{x}) - Q(\boldsymbol{\theta}^{(t)}|\boldsymbol{\theta}^{(t)}) = G(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}). \quad (62)$$

The last two terms in $G(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})$ are constant with respect to $\boldsymbol{\theta}$, so Q and G are maximized at the same $\boldsymbol{\theta}$. Further, G is tangent to l at $\boldsymbol{\theta}^{(t)}$, and lies everywhere below l . We say that G is a *minorizing function* for l . EM transfers optimization from l to the surrogate function G , which is more convenient to maximize.

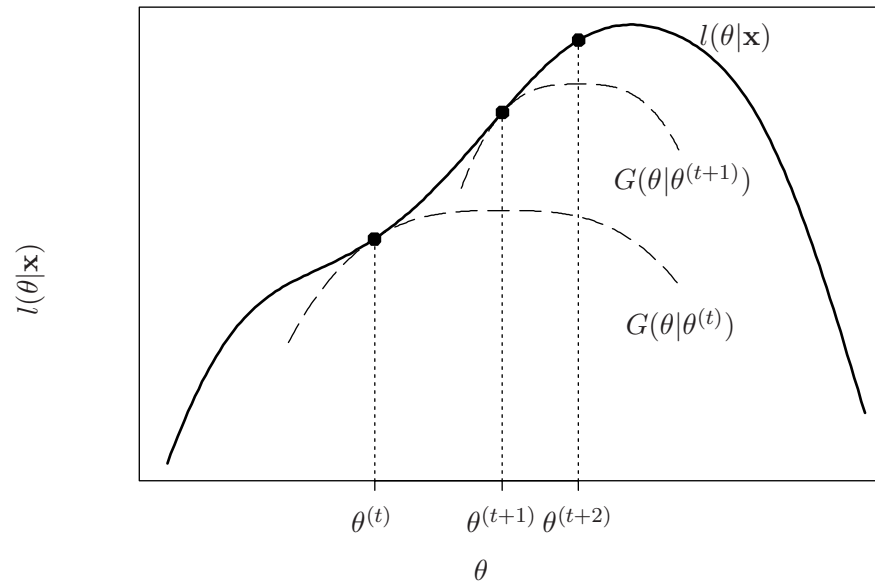


Figure 21: One-dimensional illustration of EM algorithm as a minorization or optimization transfer strategy.

Each E step forms a minorizing function G , and each M step maximizes it to provide an uphill step.

EM in exponential families

1. **E step:** Compute the expected values of the sufficient statistics for the complete data, given the observed data and using the current parameter guesses, $\boldsymbol{\theta}^{(t)}$. Let $\mathbf{s}^{(t)} = E\{\mathbf{s}(\mathbf{Y})|\mathbf{x}, \boldsymbol{\theta}^{(t)}\} = \int \mathbf{s}(\mathbf{y}) f_{\mathbf{Z}|\mathbf{X}}(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta}^{(t)}) d\mathbf{z}$.
2. **M step:** Set $\boldsymbol{\theta}^{(t+1)}$ to the value which makes the unconditional expectation of the sufficient statistics for the complete data equal to $\mathbf{s}^{(t)}$. In other words, $\boldsymbol{\theta}^{(t+1)}$ solves $E\{\mathbf{s}(\mathbf{Y})|\boldsymbol{\theta}\} = \mathbf{s}^{(t)}$.
3. Return to the E step unless a convergence criterion has been met.

The missing information principle

Can show

$$\log f_{\mathbf{X}}(\mathbf{x}|\boldsymbol{\theta}) = Q(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) - H(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) \quad (63)$$

where

$$H(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) = \text{E} \left\{ \log f_{\mathbf{Z}|\mathbf{X}}(\mathbf{Z}|\mathbf{x}, \boldsymbol{\theta}) \mid \mathbf{x}, \boldsymbol{\theta}^{(t)} \right\}. \quad (64)$$

Taking second order partial derivatives of (63) and negating both sides yields

$$-l''(\boldsymbol{\theta}|\mathbf{x}) = -\mathbf{Q}''(\boldsymbol{\theta}|\boldsymbol{\omega})|_{\boldsymbol{\omega}=\boldsymbol{\theta}} + \mathbf{H}''(\boldsymbol{\theta}|\boldsymbol{\omega})|_{\boldsymbol{\omega}=\boldsymbol{\theta}} \quad (65)$$

where the primes on $\mathbf{Q}''$ and $\mathbf{H}''$ denote derivatives with respect to the first argument, $\boldsymbol{\theta}$.

Rewrite (65) as

$$\hat{\mathbf{i}}_{\mathbf{X}}(\boldsymbol{\theta}) = \hat{\mathbf{i}}_{\mathbf{Y}}(\boldsymbol{\theta}) - \hat{\mathbf{i}}_{\mathbf{Z}|\mathbf{X}}(\boldsymbol{\theta}) \quad (66)$$

where $\hat{\mathbf{i}}_{\mathbf{X}}(\boldsymbol{\theta}) = -l''(\boldsymbol{\theta}|\mathbf{x})$ is the observed information, and $\hat{\mathbf{i}}_{\mathbf{Y}}(\boldsymbol{\theta})$ and $\hat{\mathbf{i}}_{\mathbf{Z}|\mathbf{X}}(\boldsymbol{\theta})$ will be called the complete information and the missing information, respectively.

The observed information equals the complete information minus the missing information.

Obtaining variance estimates

1. Bootstrap
2. SEM algorithm (next)

Less common methods include Louis's method (using missing information principle), empirical information, and numerical differentiation.

Supplemented EM (SEM) algorithm

Let Ψ denotes the EM mapping, having fixed point $\hat{\theta}$ and Jacobian matrix $\Psi'(\theta)$ with (i, j) th element equaling $\frac{d\Psi_i(\theta)}{d\theta_j}$. It can be shown that

$$\Psi'(\hat{\theta})^T = \hat{\mathbf{i}}_{Z|X}(\hat{\theta})\hat{\mathbf{i}}_Y(\hat{\theta})^{-1} \quad (67)$$

Further use of the missing information principle leads to

$$\widehat{\text{var}}\{\hat{\theta}\} = \hat{\mathbf{i}}_Y(\hat{\theta})^{-1} \left(\mathbf{I} + \Psi'(\hat{\theta})^T (\mathbf{I} - \Psi'(\hat{\theta})^T)^{-1} \right). \quad (68)$$

SEM is numerically stable and requires little extra work.

SEM algorithm:

1. Run the EM algorithm to convergence, finding $\hat{\theta}$.
2. Restart the algorithm from some $\theta^{(0)}$ near to $\hat{\theta}$. For $t = 0, 1, 2, \dots$
 - (a) Take a standard E step and M step to produce $\theta^{(t+1)}$ from $\theta^{(t)}$.
 - (b) For $j = 1, \dots, p$, define $\theta^{(t)}(j) = (\hat{\theta}_1, \dots, \hat{\theta}_{j-1}, \theta_j^{(t)}, \hat{\theta}_{j+1}, \dots, \hat{\theta}_p)$ and

$$r_{ij}^{(t)} = \frac{\Psi_i(\theta^{(t)}(j)) - \hat{\theta}_i}{\theta_j^{(t)} - \hat{\theta}_j} \quad (69)$$

for $i = 1, \dots, p$. (Recall that $\Psi(\hat{\theta}) = \hat{\theta}$.)

- (c) Stop when all $r_{ij}^{(t)}$ have converged

3. The (i, j) th element of $\Psi'(\hat{\theta})$ equals $\lim_{t \rightarrow \infty} r_{ij}^{(t)}$. Plug the final estimate of $\Psi'(\hat{\theta})$ into (68) to get the variance.

Facilitating the E-step

Monte Carlo EM (MCEM)

Replace the t th E step with

1. Draw missing datasets $\mathbf{Z}_1^{(t)}, \dots, \mathbf{Z}_{m^{(t)}}^{(t)}$ i.i.d. from $f_{\mathbf{Z}|\mathbf{X}}(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta}^{(t)})$. Each $\mathbf{Z}_j^{(t)}$ is a vector of all the missing values needed to complete the observed dataset, so $\mathbf{Y}_j = (\mathbf{x}, \mathbf{Z}_j)$ denotes a completed dataset where the missing values have been replaced by $\mathbf{Z}_j$.
2. Calculate $\hat{Q}^{(t+1)}(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) = \frac{1}{m^{(t)}} \sum_{j=1}^{m^{(t)}} \log f_{\mathbf{Y}}(\mathbf{Y}_j^{(t)}|\boldsymbol{\theta})$.

Then $\hat{Q}^{(t+1)}(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})$ is a Monte Carlo estimate of $Q(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})$.

The M step is modified to maximize $\hat{Q}^{(t+1)}(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})$.

Increase $m^{(t)}$ as iterations progress to reduce the Monte Carlo variability of $\hat{Q}$. MCEM will not converge in the same sense as ordinary EM, rather values of $\boldsymbol{\theta}^{(t)}$ will bounce around the true maximum, with a precision that depends on $m^{(t)}$.

Facilitating the M-step

ECM algorithm

Replaces the M step with a series of computationally simpler conditional maximization (CM) steps.

Call the collection of simpler CM steps after the t th E step a CM *cycle*. Thus, the t th iteration of ECM is comprised of the t th E step and the t th CM cycle.

Let S denote the total number of CM steps in each CM cycle.

For $s = 1, \dots, S$, the s th CM step in the t th cycle requires the maximization of $Q(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})$ subject to (or conditional on) a constraint, say

$$\mathbf{g}_s(\boldsymbol{\theta}) = \mathbf{g}_s(\boldsymbol{\theta}^{(t+(s-1)/S)}) \quad (70)$$

where $\boldsymbol{\theta}^{(t+(s-1)/S)}$ is the maximizer found in the $(s - 1)$ th CM step of the current cycle.

When the entire cycle of S steps of CM has been completed, we set $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t+S/S)}$ and proceed to the E step for the $(t + 1)$ th iteration.

The art of constructing an effective ECM algorithm lies in choosing the constraints cleverly.

Choice 1: Iterated Conditional Modes / Gauss-Seidel

Partition $\boldsymbol{\theta}$ into S subvectors, $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_S)$.

In the s th CM step, maximize Q with respect to $\boldsymbol{\theta}_s$ while holding all other components of $\boldsymbol{\theta}$ fixed.

This amounts to the constraint induced by the function

$$g_s(\boldsymbol{\theta}) = (\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_{s-1}, \boldsymbol{\theta}_{s+1}, \dots, \boldsymbol{\theta}_S).$$

Choice 2:

At the s th CM step, maximize Q with respect to all other components of θ while holding θ_s fixed.

Then $g_s(\theta) = \theta_s$.

Additional systems of constraints can be imagined, depending on the particular problem context.

A variant of ECM inserts an E step between each pair of CM steps, thereby updating Q at every stage of the CM cycle.

(Sketched) Example:

Multivariate regression (Meng and Rubin, 1993)

Let $U_1, \dots, U_n$ be independent where

$$U_i \sim N_d(\mu_i, \Sigma) \quad (71)$$

for $\mu_i = V_i\beta$, where the V_i are known $d \times p$ design matrices, and β and Σ are unknown parameters.

To estimate β and Σ when some elements of some of the U_i are missing, we can partition each CM cycle into two stages: one for β and one for Σ .

The algebra is messy, but the idea turns out to be simple.

- **E-step:** Find the expected values of the complete data sufficient statistics conditional on the observed data and current guesses $\beta^{(t)}, \Sigma^{(t)}$. (The sufficient stats are $\sum_{i=1}^n U_{ij}$ for $j = 1, \dots, d$ and $\sum_{i=1}^n U_{ij}U_{ik}$ for $j, k = 1, \dots, d$.)
- **CM 1:** $\beta^{(t+1/2)}$ is the weighted least squares estimate given $\Sigma = \Sigma^{(t)}$, computed using the sufficient stats from the E-step.
- **CM 2:** $\Sigma^{(t+2/2)}$ is the sample covariance matrix of the completed data, using $\beta = \beta^{(t+1/2)}$.
- **Return to the E-step**

EM gradient algorithm

Replace the M step with a single step of Newton's method, thereby approximating the maximum without actually solving for it exactly.

Instead of maximizing, choose:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \mathbf{Q}''(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})^{-1} \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}} \mathbf{Q}'(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}} \quad (72)$$

$$= \boldsymbol{\theta}^{(t)} - \mathbf{Q}''(\boldsymbol{\theta}|\boldsymbol{\theta}^{(t)})^{-1} \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}} \mathbf{l}'(\boldsymbol{\theta}^{(t)}|\mathbf{x}) \quad (73)$$

Ascent is ensured for canonical parameters in exponential families. Backtracking can ensure ascent in other cases; inflating steps can speed convergence.

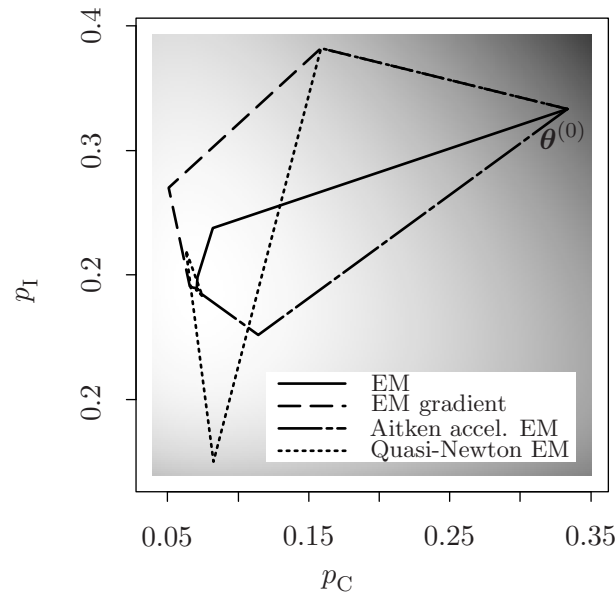


Figure 22: Steps taken by the EM gradient algorithm (long dashes). Ordinary EM steps are shown with the solid line. Steps from two methods from later sections (Aitken and quasi-Newton acceleration) are also shown, as indicated in the key. The observed data log likelihood is shown with the grey scale, with light shading corresponding to high likelihood. All algorithms were started from $p_C = p_I = 1/3$.

EM Acceleration Methods

Standard EM is reliable but slow. Acceleration methods use the EM setup to motivate particular forms of Newton-like steps.

Aitken acceleration: Equivalent to applying the Newton-Raphson method to find a zero of $\Psi(\theta) - \theta$ where Ψ is the mapping defined by the ordinary EM algorithm producing $\theta^{(t+1)} = \Psi(\theta^{(t)})$.

Quasi-Newton acceleration: Take

$$\mathbf{M}^{(t)} = \mathbf{Q}''(\theta | \theta^{(t)}) \Big|_{\theta = \theta^{(t)}} - \mathbf{B}^{(t)}$$

where $\mathbf{B}^{(t)}$ is crafted to make $\mathbf{M}^{(t)}$ approximate $\mathbf{l}''(\theta^{(t)} | \mathbf{x})$. A potentially superior strategy approximates $(\mathbf{l}'')^{-1}$ instead of $\mathbf{l}''$.

A final example: the Baum-Welch algorithm

A *hidden Markov model* can be used to describe the joint probability of a sequence of unobserved (‘hidden’) discrete state variables, $\mathbf{H} = (H_0, \dots, H_n)$, and a sequence of corresponding observed variables $\mathbf{O} = (O_0, \dots, O_n)$ for which O_i is dependent on H_i for each i .

We say that H_i emits O_i .

Let the (discrete) state spaces for elements of $\mathbf{H}$ and $\mathbf{O}$ be $\mathcal{H}$ and $\mathcal{E}$, respectively.

Such models are extremely useful in statistical genetics, signal processing, and environmental time series. For example, the hidden states can be “weather patterns” and the observed variables can be measurable weather events like rain, snow, smog, etc.

The Baum-Welch algorithm fits such models. It is an EM algorithm.

The parameters of a HMM are $\theta = (\pi, \mathbf{P}, \mathbf{E})$, where π is a vector of initial state probabilities, $\mathbf{P}$ is a matrix of transition probabilities, $\mathbf{E}$ is a matrix of emission probabilities.

For an observed sequence $\mathbf{o}$, define the *forward variables* to be

$$\alpha(i, h) = P[\mathbf{O}_{\leq i} = \mathbf{o}_{\leq i}, H_i = h] \quad (74)$$

and the *backward variables* to be

$$\beta(i, h) = P[\mathbf{O}_{> i} = \mathbf{o}_{> i} | H_i = h] \quad (75)$$

for $i = 1, \dots, n$ and each $h \in \mathcal{H}$. Dependence on θ is suppressed.

There are efficient recursive formulas to calculate the $\alpha(i, h)$ and $\beta(i, h)$.

The E-step boils down to:

Let $N(h)$ denote the number of times $H_0 = h$, let $N(h, h^*)$ denote the number of transitions from h to h^* , and let $N(h, o)$ denote the number of emissions of o when the underlying state is h . Then:

$$E\{N(h)\} = \frac{\alpha(0, h)\beta(0, h)}{P[\mathbf{O} = \mathbf{o}|\boldsymbol{\theta}]} \quad (76)$$

$$E\{N(h, h^*)\} = \sum_{i=0}^{n-1} \frac{\alpha(i, h)p(h, h^*)e(h^*, o_{i+1})\beta(i+1, h^*)}{P[\mathbf{O} = \mathbf{o}|\boldsymbol{\theta}]} \quad (77)$$

$$E\{N(h, o)\} = \sum_{i: O_i=o} \frac{\alpha(i, h)\beta(i, h)}{P[\mathbf{O} = \mathbf{o}|\boldsymbol{\theta}]} \quad (78)$$

The M-step boils down to:

$$\pi(h)^{(t+1)} = \frac{\mathbb{E}\{N(h)|\boldsymbol{\theta}^{(t)}\}}{\sum_{h^* \in \mathcal{H}} \mathbb{E}\{N(h^*)|\boldsymbol{\theta}^{(t)}\}} \quad (79)$$

$$p(h, h^*)^{(t+1)} = \frac{\mathbb{E}\{N(h, h^*)|\boldsymbol{\theta}^{(t)}\}}{\sum_{h^{**} \in \mathcal{H}} \mathbb{E}\{N(h, h^{**})|\boldsymbol{\theta}^{(t)}\}} \quad (80)$$

$$e(h, o)^{(t+1)} = \frac{\mathbb{E}\{N(h, o)|\boldsymbol{\theta}^{(t)}\}}{\sum_{o^* \in \mathcal{E}} \mathbb{E}\{N(h, o^*)|\boldsymbol{\theta}^{(t)}\}}. \quad (81)$$

(Oversimplified) HMM data:

A region is hypothesized to cycle between two weather patterns: “tending-wet” and “tending-dry”. The probability of measurable precipitation is w for a day during a tending-wet cycle and d for a day in the tending-dry cycle. Cycles switch from dry to wet or vice versa with probability s .

Observed Data: was there measurable precipitation on each of 200 consecutive days?

Latent Data: which pattern was extant on each day.

Goal: Estimate w , d , and s .

10

100

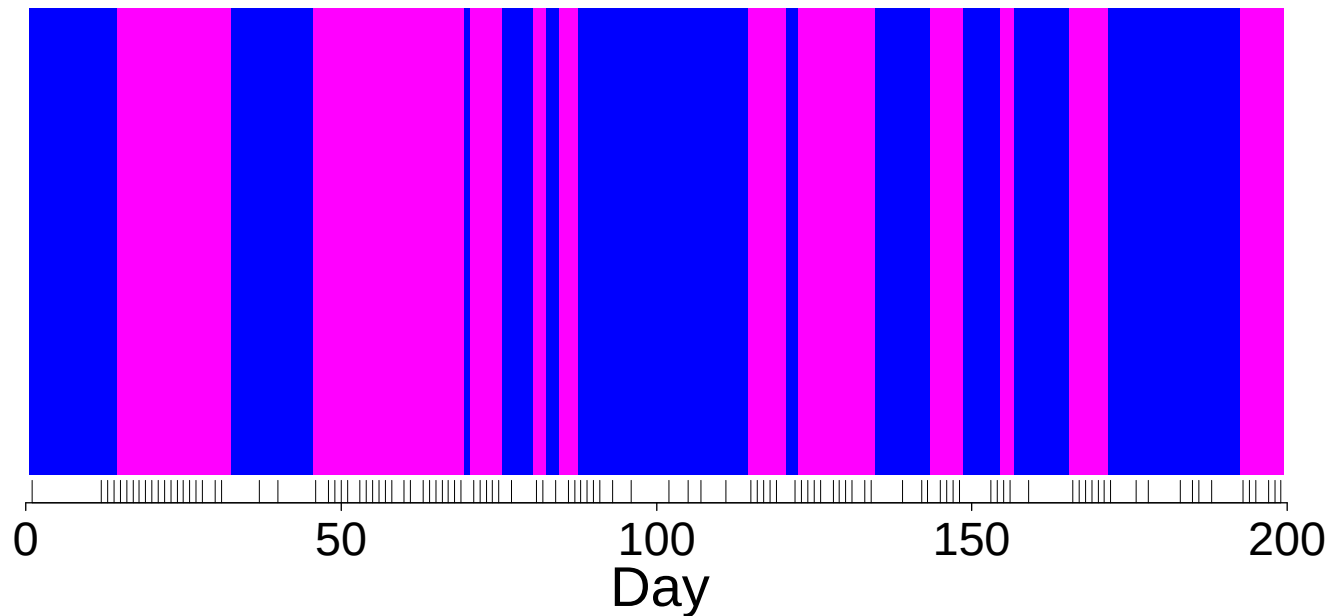


Figure 23: Simple data for a meteorological hidden Markov model.

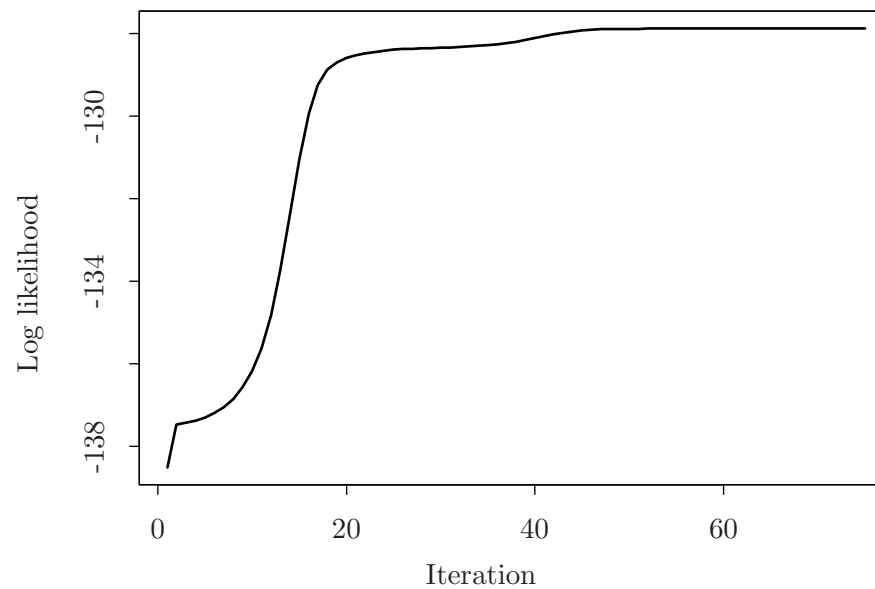


Figure 24: Log likelihood for parameters of hidden Markov model, against Baum-Welch iteration.

As an EM algorithm, the Baum-Welch approach reliably ascends as iterations progress.

Iteration, t	$d^{(t)}$	$w^{(t)}$	$s^{(t)}$
0	0.4500	0.5500	0.4000
1	0.4941	0.6042	0.3983
5	0.4632	0.6365	0.3872
10	0.3853	0.7127	0.3467
20	0.2217	0.8658	0.1750
30	0.2349	0.8528	0.1444
40	0.2475	0.8497	0.1314
50	0.2595	0.8468	0.1190
75	0.2608	0.8460	0.1174
100	0.2608	0.8460	0.1174

Table 6: Selected iterations during Baum-Welch search for MLEs for hidden Markov model for rain data.

The data were simulated with $d = 0.25$, $w = 0.85$, and $s = 0.10$.

Part I: Optimization Methods

1. Optimizing smooth univariate functions
2. Optimizing smooth multivariate functions
3. Combinatorial optimization
4. EM Optimization strategies
5. Concluding remarks

Optimization: Some concluding thoughts

- We have discussed a toolbox full of tools, both simple and advanced.
- Rarely do areas of application require discipline-specific optimization methods. Usually, many tools can solve the same problem.
- Many of these tools are available in some software packages; some aren't. Understanding how and why the tool works can help you tailor usage of existing packages to diagnose problems and coax out convergence.
- Be cautious and skeptical of convergence. Always try diverse starting values. Confirm movement along all dimensions. Compare results to simpler estimates (e.g. MOM). Be wary of globally noncompetitive local modes.
- Your time is valuable. Don't be a purist—be a pragmatist. Choose the tool that is easiest to implement. Switch tools readily if you are struggling with one. Deal with constraints via the easiest method possible: ignore, penalize, or transform. Favor options like Gauss-Seidel iteration and ECM for problems with many dimensions.

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration (Chapter 6)

Monte Carlo integration; variance reduction techniques: importance sampling, antithetic sampling, control variates; sampling importance resampling (SIR)

8. Markov Chain Monte Carlo Methods (Chapter 7)

Metropolis-Hastings algorithm, Gibbs sampling, implementation issues

9. Advanced MCMC Methods (Chapters 7 and 8)

Slice sampling and other auxiliary variable methods, Reversible jump MCMC, Perfect sampling, Langevin Metropolis-Hastings Algorithm, Hit-and-run algorithm, Multiple-try Metropolis-Hastings Algorithm

10. Integration: Concluding remarks

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration

8. Markov Chain Monte Carlo Methods

9. Advanced MCMC Methods

10. Integration: Concluding remarks

Introduction to Integration

The problem:

Consider a one-dimensional integral of the form

$$\int_a^b f(x) dx$$

- For only a few functions, f , can the value of the integral be derived analytically.
- For the rest, numerical approximations of the integral are often useful.

Examples of integration in statistics:

- Likelihood based inference: maximum likelihood inference problems when the likelihood itself is a function of one or more integrals
- Bayesian inference: In Bayesian analyses, posterior moments can be written in the form of an integral but typically cannot be evaluated analytically.

Two approaches to approximating integrals

1. Numerical Integration

Idea: Approximate $\int_a^b f(x) dx$ via the sum of many polygons under the curve, $f(x)$.

Partition the interval $[a, b]$ into n subintervals, $[x_i, x_{i+1}]$ for $i = 0, \dots, n - 1$ with $x_0 = a$ and $x_n = b$. Within each interval insert $m + 1$ nodes, so for $[x_i, x_{i+1}]$ let x_{ij}^* for $j = 0, \dots, m$. Then

$$\int_a^b f(x) dx = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) dx \approx \sum_{i=0}^{n-1} \sum_{j=0}^m A_{ij} f(x_{ij}^*)$$

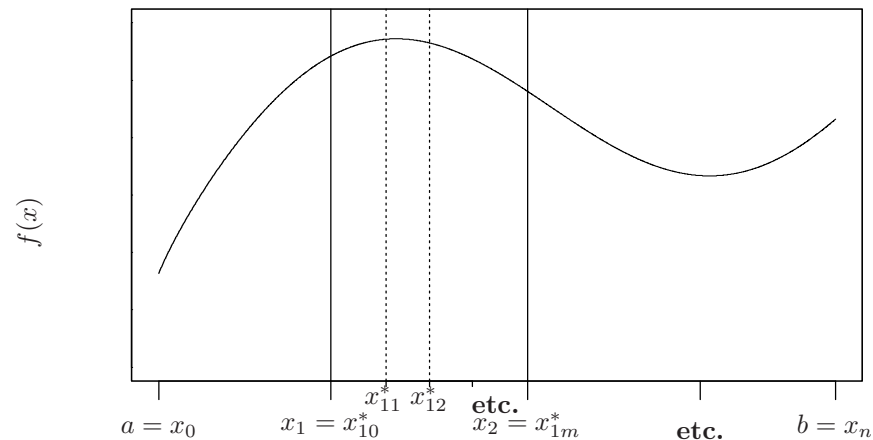
for some set of constants, A_{ij} .

2. Monte Carlo Integration

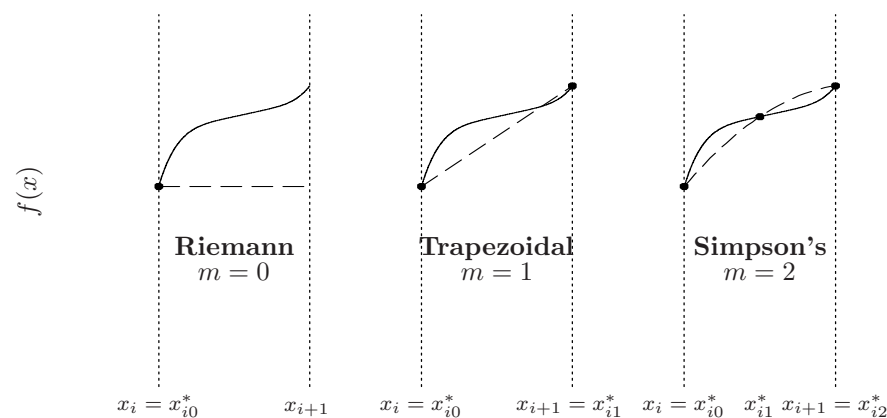
To approximate $\mu = \mathbf{E}\{\mathbf{X}\} = \int \mathbf{x}f(\mathbf{x}) d\mathbf{x}$, obtain an i.i.d. random sample $\mathbf{X}_1, \dots, \mathbf{X}_n$ from f and then approximate μ via the sample average:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{X}_i$$

Numerical integration



To integrate f between a and b , the interval is partitioned into n subintervals, $[x_i, x_{i+1}]$, each of which is further partitioned using $m + 1$ nodes, $x_{i0}^*, \dots, x_{im}^*$.



The approximation (dashed) to f (solid) provided on the subinterval $[x_i, x_{i+1}]$, for the Riemann, trapezoidal, and Simpson's rules. Note that the approximation matches f at each x_{ij}^* .

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration

Monte Carlo integration; variance reduction techniques: importance sampling, antithetic sampling, control variates; sampling importance resampling (SIR)

8. Markov Chain Monte Carlo Methods

9. Advanced MCMC Methods

10. Integration: Concluding remarks

Monte Carlo Integration

Monte Carlo Integration: statistical estimation of the value of an integral using evaluations of an integrand at a set of points drawn randomly from a distribution with support over the range of integration

Some uses:

- Bayesian analysis:
 - Posterior moments can be written in the form of an integral but typically cannot be evaluated analytically
 - Posterior probabilities can be written as the expectation of an indicator function with respect to the posterior.
- Likelihood analyses:
 - Marginalization of a joint density relies upon integration.
 - Maximum likelihood fit of a generalized linear mixed model involves integration

Introduction to the Monte Carlo Method

Consider $E\{h(\mathbf{X})\}$.

Many quantities of interest in inferential statistical analyses can be expressed as the expectation of a function of a random variable.

Let

- f denote the density of $\mathbf{X}$
- μ denote the expectation of $h(\mathbf{X})$ with respect to f

When an i.i.d. random sample $\mathbf{X}_1, \dots, \mathbf{X}_n$ is obtained from f , we can approximate μ by a sample average:

$$\hat{\mu}_{\text{MC}} = \frac{1}{n} \sum_{i=1}^n h(\mathbf{X}_i) \rightarrow \int h(\mathbf{x}) f(\mathbf{x}) d\mathbf{x} = \mu$$

as $n \rightarrow \infty$ by the strong law of large numbers.

Introduction to the Monte Carlo Method

Many other quantities can be estimated in this manner.

Let

- $v(\mathbf{x}) = (h(\mathbf{x}) - \mu)^2$
- Assume that $h(\mathbf{X})^2$ has finite expectation under f .
- Then the sampling variance of $\hat{\mu}_{\text{MC}}$ is $\sigma^2/n = \text{E}\{v(\mathbf{X})/n\}$, where the expectation is taken with respect to f .

Monte Carlo estimator of σ^2 :

$$\widehat{\text{var}}\{\hat{\mu}_{\text{MC}}\} = \frac{1}{n-1} \sum_{i=1}^n (h(\mathbf{X}_i) - \hat{\mu}_{\text{MC}})^2$$

When σ^2 exists, the central limit theorem implies that $\hat{\mu}_{\text{MC}}$ has an approximate normal distribution for large n , so approximate confidence bounds and statistical inference for μ follow.

Introduction to the Monte Carlo Method

Monte Carlo integration provides slow convergence:

- Monte Carlo integration has convergence of order $\mathcal{O}(n^{-1/2})$
- With n nodes, numerical integration methods have convergence of order $\mathcal{O}(n^{-2})$ or better

But Monte Carlo integration is nonetheless a very powerful tool:

- Numerical integration methods:
 - Difficult to extend to multidimensional problems
 - Perform best when the integrand is smooth
- Monte Carlo integration:
 - Does not attempt any systematic exploration of the p -dimensional support region of f , so is less hampered by high dimensionality
 - Does not require that the integrand be smooth

Simulation

Monte Carlo integration replaces the systematic grid of nodes used in numerical integration with a set of points chosen randomly from a probability distribution.

So, the first step in Monte Carlo integration is figuring out how to draw realizations from some distribution.

Three basic approaches for exact simulation of random variables:

1. Generate from standard parametric families
2. Inverse CDF

For any continuous distribution function F , if $U \sim \text{Unif}(0, 1)$, then $X = F^{-1}(U) = \inf\{x : F(x) \geq U\}$ has cumulative distribution function equal to F .

3. Rejection sampling

Two approaches to simulating approximate samples:

1. Sampling importance resampling
2. Markov chain Monte Carlo

Simulation: Rejection sampling

Goal: generate realizations from target density f

- Let g denote another density from which we know how to sample and we can easily calculate $g(x)$.
- Let $e(\cdot)$ denote an *envelope*, having the property $e(x) = g(x)/\alpha \geq f(x)$ for all x for which $f(x) > 0$ for a given constant $\alpha \leq 1$.

Rejection sampling:

1. Sample $Y \sim g$.
2. Sample $U \sim \text{Unif}(0, 1)$.
3. Reject Y if $U > f(Y)/e(Y)$. In this case, do not record the value of Y as an element in the target random sample. Instead, return to step 1.
4. Otherwise, keep the value of Y . Set $X = Y$ and consider X to be an element of the target random sample. Return to step 1 until you have accumulated a sample of the desired size.

Simulation: Rejection sampling

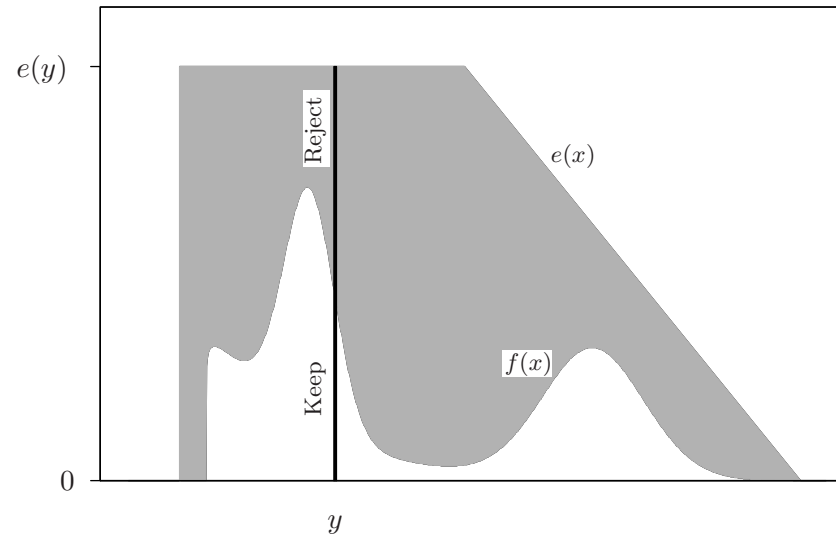


Illustration of rejection sampling for a target distribution, f , using a rejection sampling envelope e .

The draws that are kept using this algorithm constitute an i.i.d. sample from the target density f ; there is no approximation involved.

Variance reduction techniques

Can we do better than the simple Monte Carlo estimator of

$$\int h(\mathbf{x})f(\mathbf{x}) d\mathbf{x} \approx \frac{1}{n} \sum_{i=1}^n h(\mathbf{X}_i)$$

where the variables $\mathbf{X}_1, \dots, \mathbf{X}_n$ are randomly sampled from f ?

Yes!

Goal: integral estimates with lower variance than the simplest Monte Carlo approach

1. Importance sampling
2. Antithetic sampling
3. Control variates
4. Rao-Blackwellization

Importance sampling: motivation

An example to motivate importance sampling.

Goal: estimate the probability that a die roll will yield a 1

- Roll die n times
- Expect to see about $n/6$ ones.
- A point estimate of the true probability that a die roll will yield a 1 is the proportion of ones in the sample.
- The variance of this estimator is $\frac{5}{36n}$ if the die is fair.
- Recall the coefficient of variation

$$\text{cv}\{X\} = \frac{\text{var}\{X\}^{1/2}}{\text{E}\{X\}}$$

To achieve an estimate with a CV of, say, 5%, a person should expect to have to roll the die 2,000 times.

Importance sampling: motivation

To reduce the number of rolls required, consider biasing the die by replacing the faces bearing 2 and 3 with additional 1 faces.

- Increases the probability of rolling a 1 to 0.5, but we are no longer sampling from the target distribution provided by a fair die.
- Correct for this by weighting each roll of a 1 by $1/3$:
Let $Y_i = 1/3$ if the roll is a 1 and $Y_i = 0$ otherwise.
- The expectation of the sample mean of the Y_i is $1/6$, and the variance of the sample mean is $\frac{1}{36n}$.
- To achieve a coefficient of variation of 5% for this estimator, a person should expect to need only 400 rolls.

Importance sampling

Main idea: Improved accuracy is achieved by causing the event of interest to occur more frequently than it would under the naive Monte Carlo sampling framework, thereby enabling more precise estimation of it.

For very rare events, extremely large reductions in Monte Carlo variance are possible.

Some importance sampling terminology:

- The die rolling example is successful because an **importance sampling distribution** (corresponding to rolling the die with three ones) is used to over-sample a portion of the state space that receives lower probability under the target distribution (for the outcome of a fair die).
- An **importance weighting** corrects for this bias and can provide an improved estimator.

Importance sampling

Importance sampling is based upon the principle that the expectation of $h(\mathbf{X})$ with respect to its density f can be written in the alternative form

$$\mu = \int h(\mathbf{x})f(\mathbf{x}) d\mathbf{x} = \int h(\mathbf{x})\frac{f(\mathbf{x})}{g(\mathbf{x})}g(\mathbf{x}) d\mathbf{x}$$

where g is another density function called the *importance sampling function* or *envelope*.

This suggests that a Monte Carlo approach to estimating $E\{h(\mathbf{X})\}$ is to draw $\mathbf{X}_1, \dots, \mathbf{X}_n$ i.i.d. from g and use the estimator

$$\hat{\mu}_{\text{IS}}^* = \frac{1}{n} \sum_{i=1}^n h(\mathbf{X}_i)w^*(\mathbf{X}_i)$$

where $w^*(\mathbf{X}_i) = f(\mathbf{X}_i)/g(\mathbf{X}_i)$ are unstandardized weights, also called *importance ratios*.

For this strategy to be convenient, it must be easy to sample from g and to evaluate f , even when it is not easy to sample from f .

Importance sampling

Equivalently, we can write

$$\mu = \frac{\int h(\mathbf{x})f(\mathbf{x}) d\mathbf{x}}{\int f(\mathbf{x}) d\mathbf{x}} = \frac{\int h(\mathbf{x})\frac{f(\mathbf{x})}{g(\mathbf{x})}g(\mathbf{x}) d\mathbf{x}}{\int \frac{f(\mathbf{x})}{g(\mathbf{x})}g(\mathbf{x}) d\mathbf{x}}$$

This suggests drawing $\mathbf{X}_1, \dots, \mathbf{X}_n$ i.i.d. from g and using the estimator

$$\hat{\mu}_{\text{IS}} = \sum_{i=1}^n h(\mathbf{X}_i)w(\mathbf{X}_i)$$

where $w(\mathbf{X}_i) = w^*(\mathbf{X}_i) / \sum_{i=1}^n w^*(\mathbf{X}_i)$ are standardized weights.

This second approach is important because it can be used when f is known only up to a proportionality constant, as is frequently the case when f is a posterior density in a Bayesian analyses.

Importance sampling

Convergence: Both estimators can be shown to converge using the strong law of large numbers, as long as the support of the envelope includes all of the support of f .

Choosing standardized or unstandardized weights:

Standardized weights

- Should provide a better estimator when $w^*(\mathbf{X})$ and $h(\mathbf{X})w^*(\mathbf{X})$ are strongly correlated
- Do not require knowing the proportionality constant for f
- Can be inferior to unstandardized weights when estimating small probabilities

Importance sampling

Choosing g :

- In order for the estimators to avoid excessive variability, it is important that $f(\mathbf{x})/g(\mathbf{x})$ is bounded and that g has heavier tails than f .
- If this requirement is not met, then some standardized importance weights will be huge.
- A rare draw from g with much higher density under f than under g will receive huge weight and inflate the variance of the estimator.
- One strategy: Choose the function g so that $f(\mathbf{x})/g(\mathbf{x})$ is large only when $h(\mathbf{x})$ is very small.

Importance sampling in practice

An attractive feature of any importance sampling method is the possibility of **reusing the simulations**.

Some strategies:

1. The same sampled points and weights can be used to compute a variety of Monte Carlo integral estimates of different quantities.
2. The weights can be changed to reflect
 - An alternative importance sampling envelope.
Use to assess or improve performance of the estimator itself.
 - An alternative target distribution.
Use to estimate the expectation of $h(\mathbf{X})$ with respect to a different density.

Importance sampling in Bayesian analysis

Idea: update estimates based on a revised posterior distribution

Can use importance sampling for

- Bayesian sensitivity analysis
- Sequential updating of previous results via Bayes' Theorem in light of new information

Updates can be carried out by multiplying each existing weight, $w(\mathbf{X}_i)$, by an adjustment factor.

Example:

If f is a posterior distribution for $\mathbf{X}$ using prior p_1 , then weights equal to

$$w(\mathbf{X}_i)p_2(\mathbf{X}_i)/p_1(\mathbf{X}_i)$$

for $i = 1, \dots, n$ can be used with the existing sample to provide inference from the posterior distribution using prior p_2 .

Importance sampling example: Network failure

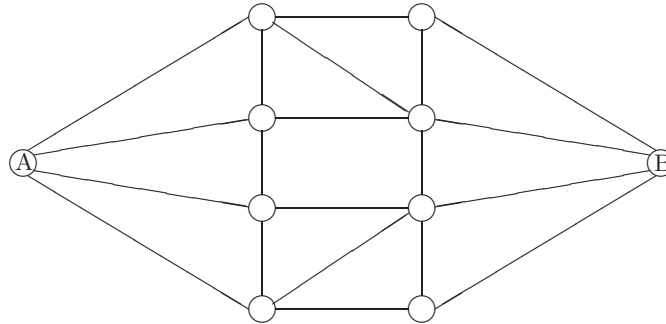


Figure 25: Network connecting A and B

- Many systems can be represented by connected graphs
- Graph is comprised of nodes (circles) and edges (line segments)
- A signal sent from A to B must follow a path along any available edges.
- In order for the signal to successfully reach B , a connected path from A to B must exist.

Importance sampling example: Network failure

Imperfect network reliability means that the signal may fail to be transmitted correctly between any pair of connected nodes—in other words, some edges may be broken.

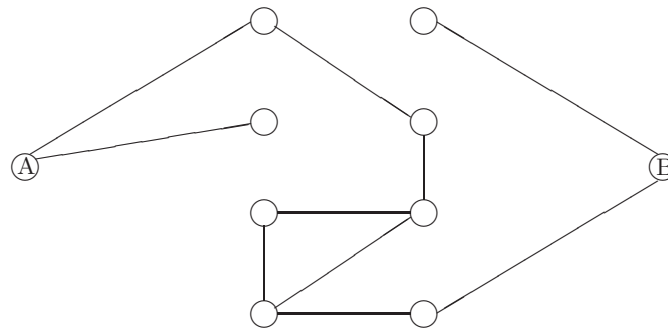


Figure 26: A degraded version of the network in Figure 25.

- This network has 10 broken edges
- If the lowest horizontal edge in the figure above were broken, the network would fail

Importance sampling example: Network failure

Network graphs can be used to model many systems:

- Transmission of signals
 - Analog voice transmission
 - Electro-magnetic digital signals
 - Optical transmission of digital data
- Conceptual model
 - Edges represent different machines or people whose participation may be needed to achieve some outcome

Typical quantity of interest: probability of network failure given specific probabilities for the failure of each edge

Importance sampling example: Network failure

Simple example:

- Network in Figure 25 with 20 potential edges
- Each edge is assumed to fail independently with the same probability,
 $p = 0.05$

Let

- $\mathbf{X} = (X_1, \dots, X_{20})$ summarize a random outcome for each edge
- $b(\mathbf{X}) = \#$ of broken edges in $\mathbf{X}$
- Network failure indicator:

$$h(\mathbf{X}) = \begin{cases} 1 & \text{if } A \text{ and } B \text{ are not connected} \\ 0 & \text{if } A \text{ and } B \text{ are connected} \end{cases}$$

The probability of network failure is

$$\mu = E\{h(\mathbf{X})\}.$$

Computing μ for a network of any realistic size can be a very difficult combinatorial problem.

Importance sampling example: Network failure

Naive Monte Carlo estimate of μ

- Draw $\mathbf{X}_1, \dots, \mathbf{X}_n$ independently and uniformly at random from the set of all possible network configurations whose edges fail independently with probability p .
- Compute

$$\hat{\mu}_{\text{MC}} = \frac{1}{n} \sum_{i=1}^n h(\mathbf{X}_i).$$

This estimator has variance $\mu(1 - \mu)/n$.

For $n = 100,000$ and $p = 0.05$, simulation yields

- Monte Carlo mean estimate $\hat{\mu}_{\text{MC}} = 2.00 \times 10^{-5}$
- Monte Carlo standard error estimate 1.41×10^{-5}

Importance sampling example: Network failure

Problem: $h(\mathbf{X})$ is very rarely 1 unless p is unrealistically large.

Result: huge number of networks simulations are required in order to estimate μ with sufficient precision.

Importance sampling strategy: focus on simulation of $\mathbf{X}$ for which $h(\mathbf{X}) = 1$, compensating for this bias through the assignment of importance weights.

The calculations that follow adopt this strategy, using the non-standardized importance weights

Importance sampling example: Network failure

Simulate $\mathbf{X}_1^*, \dots, \mathbf{X}_n^*$ by generating network configurations formed by breaking edges in Figure 25 assuming independent edge failure with probability $p^* > p$.

The importance weight for $\mathbf{X}_i^*$ can be written as

$$w^*(\mathbf{X}_i^*) = \left(\frac{1-p}{1-p^*} \right)^{20} \left(\frac{p(1-p^*)}{p^*(1-p)} \right)^{b(\mathbf{X}_i^*)}$$

The importance sampling estimator of μ is

$$\hat{\mu}_{\text{IS}}^* = \frac{1}{n} \sum_{i=1}^n h(\mathbf{X}_i^*) w^*(\mathbf{X}_i^*).$$

Importance sampling example: Network failure

Naive simulation strategy using $p = 0.05$:

2 of 100,000 simulated networks failed

Importance sampling strategy with $p^* = 0.25$:

497 of 100,000 simulated networks failed

Importance sampling estimates with $n = 100,000$ and $p^* = 0.25$:

- Monte Carlo mean estimate $\hat{\mu}_{\text{IS}}^* = 1.01 \times 10^{-5}$
- Monte Carlo standard error estimate 1.56×10^{-6}

14-fold variance reduction by importance sampling!

Antithetic sampling

Idea: Find two identically distributed unbiased Monte Carlo estimators, say $\hat{\mu}_1$ and $\hat{\mu}_2$, that are negatively correlated.

Average these estimators

$$\hat{\mu}_{\text{AS}} = (\hat{\mu}_1 + \hat{\mu}_2)/2$$

The antithetic sampling estimator has variance equal to

$$\text{var}\{\hat{\mu}_{\text{AS}}\} = \frac{1}{4}(\text{var}\{\hat{\mu}_1\} + \text{var}\{\hat{\mu}_2\}) + \frac{1}{2}\text{cov}\{\hat{\mu}_1, \hat{\mu}_2\} = \frac{(1 + \rho)\sigma^2}{2n}$$

where ρ is the correlation between the two estimators and σ^2/n is the variance of either estimator using a sample of size n

Allocating all $2n$ samples to one estimator would yield a variance of $\sigma^2/2n > (1 + \rho)\sigma^2/2n$ since $\rho < 0$.

Antithetic sampling

Given an initial estimator, $\hat{\mu}_1$, how should you construct a second, identically distributed estimator $\hat{\mu}_2$ that is negatively correlated with $\hat{\mu}_1$?

- Let $\mathbf{X}$ denote a set of i.i.d. random variables, $\{\mathbf{X}_1, \dots, \mathbf{X}_n\}$
- Suppose

$$\hat{\mu}_1(\mathbf{X}) = \sum_{i=1}^n h_1(\mathbf{X}_i)/n,$$

where h_1 is a real-valued function with $h_1(\mathbf{X}_i) = h_1(X_{i1}, \dots, X_{im})$

- Assume $E\{h_1(\mathbf{X}_i)\} = \mu$.
- Similarly, let $\hat{\mu}_2(\mathbf{X}) = \sum_{i=1}^n h_2(\mathbf{X}_i)/n$

Can show that if h_1 and h_2 are functions of m random variables, $U_1, \dots, U_m$, and if each function is monotone in each argument, then

$$\text{cov}\{h_1(U_1, \dots, U_m), h_2(1 - U_1, \dots, 1 - U_m)\} \leq 0.$$

Antithetic sampling strategy

The Monte Carlo integral estimate $\hat{\mu}_1(\mathbf{X})$ can be written as

$$\hat{\mu}_1(\mathbf{X}) = \frac{1}{n} \sum_{i=1}^n h_1(F_1^{-1}(U_{i1}), \dots, F_m^{-1}(U_{im}))$$

where

- F_j is the cumulative distribution function of X_{ij} , $j = 1, \dots, m$
- the U_{ij} are independent $\text{Unif}(0, 1)$ random variables

Since F_j is a cumulative distribution function, its inverse is nondecreasing. Therefore, $h_1(F_1^{-1}(U_{i1}), \dots, F_m^{-1}(U_{im}))$ is monotone in each U_{ij} for $j = 1, \dots, m$ whenever h_1 is monotone in its arguments.

Antithetic sampling strategy

Now, if $U_{ij} \sim \text{Unif}(0, 1)$, then $1 - U_{ij} \sim \text{Unif}(0, 1)$. Hence,

$$h_2(1 - U_{i1}, \dots, 1 - U_{im}) = h_1(F_1^{-1}(1 - U_{i1}), \dots, F_m^{-1}(1 - U_{im}))$$

is monotone in each argument and has the same distribution as $h_1(F_1^{-1}(U_{i1}), \dots, F_m^{-1}(U_{im}))$. Therefore,

$$\hat{\mu}_2(\mathbf{X}) = \frac{1}{n} \sum_{i=1}^n h_1(F_1^{-1}(1 - U_{i1}), \dots, F_m^{-1}(1 - U_{im}))$$

is a second estimator of μ having the same distribution as $\hat{\mu}_1(\mathbf{X})$.

- **For these estimators, $\text{cov}\{\hat{\mu}_1(\mathbf{X}), \hat{\mu}_2(\mathbf{X})\} \leq 0$.**
- **Can show the estimator $\hat{\mu}_{\text{AS}} = (\hat{\mu}_1 + \hat{\mu}_2)/2$ will have smaller variance than $\hat{\mu}_1$ would have with a sample of size $2n$.**
- **Only need to generate a single set of n random numbers, with the other n derived from the antithetic principle.**

Antithetic sampling example: Network failure, cont.

- Let the i th simulated network, $\mathbf{X}_i$, be determined by standard uniform random variables $U_{i1}, \dots, U_{im}$ where $m = 20$.
- $\mathbf{X}_i$ is obtained by breaking the j th edge when $U_{ij} < p$ for $j = 1, \dots, m$
- Network failure indicator: $h(\mathbf{X}_i) = h(U_{i1}, \dots, U_{im})$ where

$$h(\mathbf{X}_i) = \begin{cases} 1 & \text{if } A \text{ and } B \text{ are not connected} \\ 0 & \text{if } A \text{ and } B \text{ are connected} \end{cases}$$

Note: h is nondecreasing in each U_{ij}

- Obtain antithetic network draws, say $\mathbf{X}_i^*$, by breaking the j th edge when $U_{ij} > 1 - p$, for the same set of U_{ij} used to generate $\mathbf{X}_i$.
- The negative correlation induced by this strategy will ensure that

$$\frac{1}{2n} \left(\sum_{i=1}^n h(\mathbf{X}_i) + h(\mathbf{X}_i^*) \right)$$

is a superior estimator to $\frac{1}{2n} \sum_{i=1}^{2n} h(\mathbf{X}_i)$.

Control variates

The control variate strategy improves estimation of an unknown integral by relating the estimate to some correlated estimator of an integral whose value is known.

Suppose

- We wish to estimate the unknown quantity $\mu = E\{h(\mathbf{X})\}$
- We know of a related quantity, $\theta = E\{c(\mathbf{Y})\}$, whose value can be determined analytically.

Let $(\mathbf{X}_1, \mathbf{Y}_1), \dots, (\mathbf{X}_n, \mathbf{Y}_n)$ denote pairs of random variables observed independently as simulation outcomes.

Control variates

The simple Monte Carlo estimators are

$$\hat{\mu}_{\text{MC}} = \frac{1}{n} \sum_{i=1}^n h(\mathbf{X}_i) \quad \text{and} \quad \hat{\theta}_{\text{MC}} = \frac{1}{n} \sum_{i=1}^n c(\mathbf{Y}_i)$$

$\hat{\theta}_{\text{MC}}$ is unnecessary since θ can be found analytically, but note that $\hat{\mu}_{\text{MC}}$ will be correlated with $\hat{\theta}_{\text{MC}}$ when $\text{cor}\{h(\mathbf{X}_i), c(\mathbf{Y}_i)\} \neq 0$.

For example: If the correlation is positive, an unusually high outcome for $\hat{\theta}_{\text{MC}}$ should tend to be associated with an unusually high outcome for $\hat{\mu}_{\text{MC}}$.

If comparison of $\hat{\theta}_{\text{MC}}$ to θ suggests such an outcome, then

- Adjust $\hat{\mu}_{\text{MC}}$ downward accordingly
- Make opposite adjustment when the correlation is negative

Control variate estimator

$$\hat{\mu}_{\text{CV}} = \hat{\mu}_{\text{MC}} + \lambda(\hat{\theta}_{\text{MC}} - \theta)$$

where λ is a parameter to be chosen by the user.

Control variates

Can show that

$$\text{var}\{\hat{\mu}_{\text{CV}}\} = \text{var}\{\hat{\mu}_{\text{MC}}\} + \lambda^2 \text{var}\{\hat{\theta}_{\text{MC}}\} + 2\lambda \text{cov}\{\hat{\mu}_{\text{MC}}, \hat{\theta}_{\text{MC}}\}$$

Minimizing this quantity with respect to λ shows that the minimal variance,

$$\min_{\lambda} (\text{var}\{\hat{\mu}_{\text{CV}}\}) = \text{var}\{\hat{\mu}_{\text{MC}}\} - \frac{(\text{cov}\{\hat{\mu}_{\text{MC}}, \hat{\theta}_{\text{MC}}\})^2}{\text{var}\{\hat{\theta}_{\text{MC}}\}},$$

is obtained when

$$\lambda = \frac{-\text{cov}\{\hat{\mu}_{\text{MC}}, \hat{\theta}_{\text{MC}}\}}{\text{var}\{\hat{\theta}_{\text{MC}}\}}. \quad (82)$$

- Optimal λ depends on unknown moments of $h(\mathbf{X}_i)$ and $c(\mathbf{Y}_i)$, but these can be estimated using the sample $(\mathbf{X}_1, \mathbf{Y}_1), \dots, (\mathbf{X}_n, \mathbf{Y}_n)$.
- Proportional reduction in variance obtained by using $\hat{\mu}_{\text{CV}}$ instead of $\hat{\mu}_{\text{MC}}$ is equal to the square of the correlation between $\hat{\mu}_{\text{MC}}$ and $\hat{\theta}_{\text{MC}}$.

Control variates in practice

Consider the regression model

$$E\{h(\mathbf{X}_i)|\mathbf{Y}_i = \mathbf{y}_i\} = \beta_0 + \beta_1 c(\mathbf{y}_i)$$

with the usual regression assumptions and estimators. Then

$$\begin{aligned}\hat{\mu}_{\text{CV}} &= \hat{\mu}_{\text{MC}} + \hat{\lambda}(\hat{\theta}_{\text{MC}} - \theta) \\ &= \hat{\beta}_0 + \hat{\beta}_1 \theta \quad \text{where } \hat{\lambda} = -\hat{\beta}_1\end{aligned}$$

- Use linear regression software to estimate $\hat{\mu}_{\text{CV}}$ and
- Standard error of $\hat{\mu}_{\text{CV}}$ is the standard error for a regression fitted value

Other details:

- $\hat{\mu}_{\text{MC}}$ and $\hat{\theta}_{\text{MC}}$ often depend on same random variables, so $\mathbf{X}_i = \mathbf{Y}_i$
- Can use more than one control variate, so for m control variates,

$$\hat{\mu}_{\text{CV}} = \hat{\mu}_{\text{MC}} + \sum_{j=1}^m \lambda_j (\hat{\theta}_{\text{MC},j} - \theta_j)$$

Control variate example 1: Importance sampling

Importance sampling is built upon the idea of sampling from an envelope that induces a correlation between $h(\mathbf{X})w^*(\mathbf{X})$ and $w^*(\mathbf{X})$.

- We know $E\{w^*(\mathbf{X})\} = 1$
- Use the control variate $\bar{w}^* = \sum_{i=1}^n w^*(\mathbf{X}_i)/n$

If $\bar{w}^* > 1$, then

- Average value of $h(\mathbf{X})w^*(\mathbf{X})$ is also probably unusually high
- $\hat{\mu}_{\text{IS}}^*$ probably differs from its expectation, μ

Importance sampling control variate estimator:

$$\hat{\mu}_{\text{ISCV}} = \hat{\mu}_{\text{IS}}^* + \lambda(\bar{w}^* - 1)$$

where $\hat{\mu}_{\text{IS}}^*$ = importance sampling estimator with unstandardized weights.

- Estimate λ and the standard error of $\hat{\mu}_{\text{ISCV}}$ from a regression of $h(\mathbf{X})w^*(\mathbf{X})$ on $w^*(\mathbf{X})$
- $\hat{\mu}_{\text{ISCV}}$ often has lower mean squared error than $\hat{\mu}_{\text{IS}}^*$

Control variate example: Option Pricing

A *call option* is a financial instrument that gives its holder the right—but not the obligation—to buy a specified amount of a financial asset, on or by a specified maturity date, for a specified price.

For a European call option, the option can be exercised only at the maturity date.

The *strike price* is the price at which the transaction is completed if the option is exercised.

Control variate example: Option Pricing

Let

- $S^{(t)}$ denote the price of the underlying financial asset (say, a stock) at time t
- K denote the strike price
- T denote the maturity date

When time T arrives, the holder of the call option will not wish to exercise his option if $K > S^{(T)}$, because he can obtain the stock more cheaply on the open market.

However, the option will be valuable if $K < S^{(T)}$, because he can buy the stock at the low price K and immediately sell it at the higher market price $S^{(T)}$.

It is of interest to determine how much the buyer of this call option should pay at time $t = 0$ for this option with strike price K at maturity date T .

Control variate example: Option Pricing

The Nobel Prize-winning model introduced by Black, Scholes, and Merton in 1973 provides a popular approach to determining the fair price of an option using a stochastic differential equation.

The fair price of an option is the amount to be paid at time $t = 0$ that would exactly counterbalance the expected payoff at maturity.

We'll consider the simplest case: a European call option on a stock that pays no dividends. The fair price of such an option can be determined analytically under the Black–Scholes model, but estimation of the fair price via Monte Carlo is an instructive starting point.

Control variate example: Option Pricing

According to the Black–Scholes model, the value of the stock at day T can be simulated as

$$S^{(T)} = S^{(0)} \exp \left\{ \left(r - \frac{\sigma^2}{2} \right) \frac{T}{365} + \sigma Z \sqrt{\frac{T}{365}} \right\} \quad (83)$$

where

- r is the risk-free rate of return (typically the return rate of the US Treasury bill that matures on day $T - 1$)
- σ is the stock's volatility (an annualized estimate of the standard deviation of $\log(S^{(t+1)}/S^{(t)})$ under a lognormal price model)
- Z is a standard normal deviate.

Control variate example: Option Pricing

If we knew that the price of the stock at day T would equal $S^{(T)}$, then the fair price of the call option would be

$$C = \exp\{-rT/365\} \max\{0, S^{(T)} - K\}, \quad (84)$$

discounting the payoff to present value. Since $S^{(T)}$ is unknown to the buyer of the call, the fair price to pay at $t = 0$ is the expected value of the discounted payoff, namely $E\{C\}$. Thus, a Monte Carlo estimate of the fair price to pay at $t = 0$ is

$$\bar{C} = \frac{1}{n} \sum_{i=1}^n C_i, \quad (85)$$

where the C_i are simulated from (83) and (84) for $i = 1, \dots, n$ using an i.i.d. sample of standard normal deviates, $Z_1, \dots, Z_n$.

Since the true fair price, $E\{C\}$, can be computed analytically in this instance, there is no need to apply the Monte Carlo approach.

Control variate example: Option Pricing

A special type of European call option, called an *Asian option* has a payoff based on the average price of the underlying stock throughout the holding period.

To simulate the payoff of an Asian call option, apply (83) sequentially T times, each time advancing the stock price one day and recording the simulated closing price for that day, so

$$S^{(t+1)} = S^{(t)} \exp \left\{ \frac{r - \sigma^2/2}{365} + \frac{\sigma Z^{(t)}}{\sqrt{365}} \right\} \quad (86)$$

for a sequence of standard normal deviates, $\{Z^{(t)}\}$, where $t = 0, \dots, T-1$. The discounted payoff is then

$$A = \exp\{-rT/365\} \max\{0, \bar{S} - K\}, \quad (87)$$

where $\bar{S} = \sum_{t=1}^T S^{(t)}/T$ and the $S^{(t)}$ for $t = 1, \dots, T$ are the random variables representing future stock prices at the averaging times.

The fair price to pay at $t = 0$ is $E\{A\}$, but in this case there is no known analytic solution for it.

Control variate example: Option Pricing

Denote the standard Monte Carlo estimator for the fair price of an Asian call option as

$$\hat{\mu}_{\text{MC}} = \bar{A} = \frac{1}{n} \sum_{i=1}^n A_i, \quad (88)$$

where the A_i are simulated independently as described above.

If $\bar{S}$ is replaced in (87) by the geometric average of the price of the underlying stock throughout the holding period, an analytic solution for $E\{A\}$ is available, so $\theta = E\{A_{\text{geom}}\}$

Alternatively, one could estimate the fair price of an Asian call option with geometric averaging using the same sort of Monte Carlo strategy described above. Denote this Monte Carlo estimator as $\hat{\theta}_{\text{MC}}$.

The estimator $\hat{\theta}_{\text{MC}}$ makes an excellent control variate for estimation of μ . Let $\hat{\mu}_{\text{CV}} = \hat{\mu}_{\text{MC}} + \lambda(\hat{\theta}_{\text{MC}} - \theta)$. Since we expect the fair price of the two Asian options (arithmetic and geometric mean pricing) to be very highly correlated, a reasonable initial guess is to use $\lambda = -1$.

Control variate example: Option Pricing

Suppose that

- underlying stock has current price $S^{(0)} = 100$
- strike price $K = 102$
- volatility $\sigma = 0.3$
- there are $N = 50$ days to maturity, so simulation of the maturity price requires 50 iterations of (86)
- risk-free rate of return is $r = 0.05$

Then

- the geometric mean price option has a fair price of $\theta = 1.82$
- simulations show that the true fair value of the arithmetic mean price option is roughly $\mu = 1.876$
- using $n = 100,000$ simulations, we can estimate μ using either $\hat{\mu}_{\text{MC}}$ or $\hat{\mu}_{\text{CV}}$. Both estimators tend to give answers in the vicinity of μ .

But what is of interest is the standard error of estimates of μ .

Control variate example: Option Pricing

We replicated the entire Monte Carlo estimation process 100 times, obtaining 100 values for $\hat{\mu}_{\text{MC}}$ and for $\hat{\mu}_{\text{CV}}$.

The sample standard deviation of the values obtained for

- $\hat{\mu}_{\text{MC}}$ was 0.0107
- $\hat{\mu}_{\text{CV}}$ was 0.000295

Thus, the control variate approach provided an estimator with 36 times smaller standard error.

An additional step involving estimating λ yields a 63-fold improvement.

Rao-Blackwellization

Consider estimation of $\mu = E\{h(\mathbf{X})\}$ using a random sample $\mathbf{X}_1, \dots, \mathbf{X}_n$ drawn from f .

Let $\mathbf{X}_i = (\mathbf{X}_{i1}, \mathbf{X}_{i2})$ for some partitioning of the coordinates.

Recall:

- $E\{h(\mathbf{X}_i)\} = E\{E\{h(\mathbf{X}_i)|\mathbf{X}_{i2}\}\}$, where the outer expectation is taken with respect to the distribution of $\mathbf{X}_{i2}$
- Rao-Blackwell theorem states that one can reduce the variance of an unbiased estimator by conditioning the estimator on the sufficient statistics

Rao-Blackwellization

Idea: apply Rao-Blackwell to improve Monte Carlo estimates

- Let $\mathbf{X}_i = (\mathbf{X}_{i1}, \mathbf{X}_{i2})$
- Assume that the conditional expectation $E\{h(\mathbf{X}_i)|\mathbf{x}_{i2}\}$ can be solved analytically

The **Rao-Blackwellized estimator** can be defined as

$$\hat{\mu}_{\text{RB}} = \frac{1}{n} \sum_{i=1}^n E\{h(\mathbf{X}_i)|\mathbf{X}_{i2}\},$$

and has the same mean as the ordinary Monte Carlo estimator, $\hat{\mu}_{\text{MC}}$.

Note that

$$\text{var}\{\hat{\mu}_{\text{MC}}\} = \frac{1}{n} \text{var}\{E\{h(\mathbf{X}_i)|\mathbf{X}_{i2}\}\} + \frac{1}{n} E\{\text{var}\{h(\mathbf{X}_i)|\mathbf{X}_{i2}\}\} \geq \text{var}\{\hat{\mu}_{\text{RB}}\}$$

so $\hat{\mu}_{\text{RB}}$ is superior to $\hat{\mu}_{\text{MC}}$ in terms of mean squared error.

Rao-Blackwellization example: Rejection sampling

Idea: improve Monte Carlo estimate of μ by using all draws in rejection sampling instead of just the accepted draws

Recall ordinary rejection sampling:

- Candidates $Y_1, \dots, Y_M$ are generated sequentially, and some are rejected.
- The uniform random variables $U_1, \dots, U_M$ provide the rejection decisions, with Y_i being rejected if $U_i > w^*(Y_i)$, where $w^*(Y_i) = f(Y_i)/e(Y_i)$.
- Rejection sampling stops at a random time, M , with the acceptance of the n th draw, yielding $X_1, \dots, X_n$.

Rao-Blackwellization example: Rejection sampling

The ordinary Monte Carlo estimator of $\mu = E\{h(X)\}$ can then be re-expressed as

$$\hat{\mu}_{\text{MC}} = \frac{1}{n} \sum_{i=1}^M h(Y_i) 1_{\{U_i \leq w^*(Y_i)\}}, \quad (89)$$

Improve $\hat{\mu}_{\text{MC}}$ using all the candidate Y_i draws (suitably weighted), rather than merely the accepted draws.

Rao-Blackwellization of (89) yields the estimator

$$\hat{\mu}_{\text{RB}} = \frac{1}{n} \sum_{i=1}^M h(Y_i) t_i(\mathbf{Y})$$

where the $t_i(\mathbf{Y})$ are random quantities that depend on $\mathbf{Y} = (Y_1, \dots, Y_M)$ and M according to

$$\begin{aligned} t_i(\mathbf{Y}) &= E \left\{ 1_{\{U_i \leq w^*(Y_i)\}} \mid M, Y_1, \dots, Y_M \right\} \\ &= P[U_i < w^*(Y_i) \mid M, Y_1, \dots, Y_M]. \end{aligned} \quad (90)$$

Rao-Blackwellization example: Rejection sampling

- $t_M(\mathbf{Y}) = 1$ since the final candidate was accepted.
- For other candidates, the probability in (90) can be found by averaging over permutations of subsets of the realized sample:

$$t_i(\mathbf{Y}) = \frac{w^*(Y_i) \sum_{A \in \mathcal{A}_i} \prod_{j \in A} w^*(Y_j) \prod_{j \notin A} (1 - w^*(Y_j))}{\sum_{B \in \mathcal{B}} \prod_{j \in B} w^*(Y_j) \prod_{j \notin B} (1 - w^*(Y_j))}$$

where $\mathcal{A}_i$ is the set of all subsets of $\{1, \dots, i-1, i+1, \dots, M-1\}$ containing $n-2$ elements, and $\mathcal{B}$ is the set of all subsets of $\{1, \dots, M-1\}$ containing $n-1$ elements.

- Casella and Robert (1996) offer a recursion formula for computing the $t_i(\mathbf{Y})$, but this is difficult to implement unless n is fairly small.

The sampling importance resampling algorithm

The *sampling importance resampling* (SIR) algorithm simulates realizations approximately from some target distribution.

Let

- $\mathbf{X} = (X_1, \dots, X_p)$ denote a random vector with density $f(\mathbf{x})$
- $g(\mathbf{x})$ denote the density corresponding to a multivariate envelope for f

For the target density f , the weights used to correct sampling probabilities are called the *standardized importance weights*, and are defined as

$$w(\mathbf{x}_i) = \frac{f(\mathbf{x}_i)/g(\mathbf{x}_i)}{\sum_{i=1}^m f(\mathbf{x}_i)/g(\mathbf{x}_i)} \quad (91)$$

for a collection of values $\mathbf{x}_1, \dots, \mathbf{x}_m$ drawn i.i.d. from an envelope g .

Although not necessary for general importance sampling, it is useful to standardize the weights as in (91) so they sum to 1. When $f = cq$ for some unknown proportionality constant c , the unknown c cancels in the numerator and denominator of (91).

SIR algorithm

We may view importance sampling as approximating f by the discrete distribution having mass $w(\mathbf{x}_i)$ on each observed point $\mathbf{x}_i$ for $i = 1, \dots, m$. Rubin (1987, 1988) proposed sampling from this distribution to provide an approximate sample from f .

The SIR algorithm therefore proceeds as follows:

1. Sample candidates $\mathbf{Y}_1, \dots, \mathbf{Y}_m$ i.i.d. from g .
2. Calculate the standardized importance weights, $w(\mathbf{Y}_1), \dots, w(\mathbf{Y}_m)$.
3. Resample $\mathbf{X}_1, \dots, \mathbf{X}_n$ from $\mathbf{Y}_1, \dots, \mathbf{Y}_m$ with replacement with probabilities $w(\mathbf{Y}_1), \dots, w(\mathbf{Y}_m)$.

A random variable $\mathbf{X}$ drawn with the SIR algorithm has distribution that converges to f as $m \rightarrow \infty$.

Since SIR generates $\mathbf{X}_1, \dots, \mathbf{X}_n$ approximately i.i.d. from f , one may proceed with Monte Carlo integration such as estimating the expectation of $h(\mathbf{X})$ by

$$\hat{\mu}_{\text{SIR}} = \sum_{i=1}^n h(\mathbf{X}_i)/n$$

SIR algorithm

SIR is sensitive to the choice of g , which should have heavier tails than f .

Although both SIR and rejection sampling rely on the ratio of target to envelope, they differ in an important way.

- **Rejection sampling** is perfect, in the sense that the distribution of a generated draw is exactly f , but it requires a random number of draws to obtain a sample of size n .
- The **SIR algorithm** uses a pre-determined number of draws to generate an n -sample but permits a random degree of approximation to f in the distribution of the sampled points.

SIR algorithm: Implementation details

Sample size: When conducting SIR, it is important to consider the relative sizes of the initial sample and the resample. These sample sizes are m and n , respectively.

- In principle, we require $n/m \rightarrow 0$ for distributional convergence of the sample.
- In practice, the maximum tolerable ratio n/m depends on the quality of the envelope. We have sometimes found $n/m \leq 1/10$ tolerable so long as the resulting resample does not contain too many replicates of any initial draw.

SIR algorithm: Implementation details

Envelope, g : The SIR algorithm can be sensitive to the choice of g .

Strategies:

- The support of g must include the entire support of f if a reweighted sample from g is to approximate a sample from f .
- g should have heavier tails than f , or more generally g should be chosen to ensure that $f(\mathbf{x})/g(\mathbf{x})$ never grows too large.
- When the distribution of weights is highly skewed, use to a different envelope or a different sampling strategy.

SIR example: Slash distribution

The random variable Y has a *slash distribution* if $Y = X/U$ where $X \sim N(0, 1)$ and $U \sim \text{Unif}(0, 1)$ independently.

Consider using the slash distribution as a SIR envelope to generate standard normal variates, and conversely using the normal distribution as a SIR envelope to generate slash variates. Since it is easy to simulate from both densities using standard methods, SIR is not needed in either case, but examining the results is instructive.

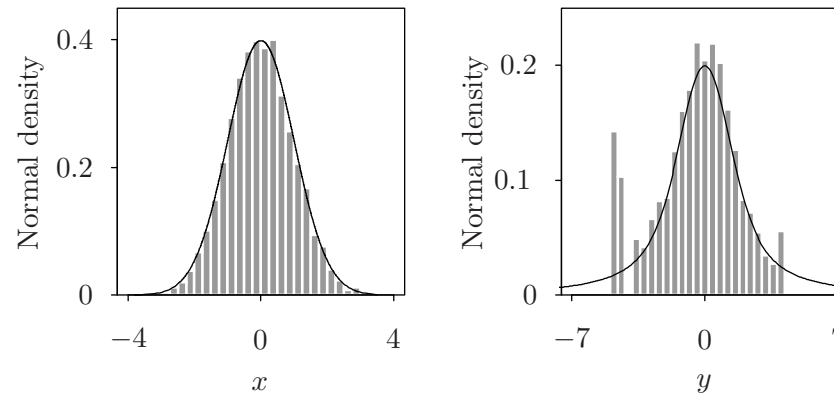
The slash density function is

$$f(y) = \begin{cases} \frac{1 - \exp\{-y^2/2\}}{y^2\sqrt{2\pi}}, & y \neq 0, \\ \frac{1}{2\sqrt{2\pi}}, & y = 0. \end{cases}$$

This density has very heavy tails. Therefore, it is a fine importance sampling function for generating draws from a standard normal distribution using SIR.

The normal density is not a suitable importance sampling function for SIR use when generating draws from the slash distribution, because the envelope's tails are far lighter than the target's.

SIR example: Slash distribution



- Let $m = 100,000$ and $n = 5000$.
- The left panel shows a histogram of approximate draws from a standard normal density obtained via SIR with a slash distribution envelope.
- The right panel shows a histogram of approximate draws from a slash density obtained via SIR using a normal envelope.
- The solid lines show the target densities.

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration

8. Markov Chain Monte Carlo Methods

Metropolis-Hastings algorithm, Gibbs sampling, implementation issues

9. Advanced MCMC Methods

10. Integration: Concluding remarks

Markov chain Monte Carlo

- Markov chain Monte Carlo (MCMC) methods introduced below can be used to generate a draw from a distribution that approximates some target distribution $f(\cdot)$
- The basic idea is that we know that $X \sim f(x)$, but we can't get $f(x)$ in closed form or we can't simulate easily from $f(x)$.
- The MCMC methods introduced below can be used to generate a draw from a distribution that approximates f , but they are more properly viewed as methods for generating a sample from which expectations of functions of X can reliably be estimated.

Brief review of Markov chains: Definitions

1. A discrete-time, discrete-space Markov chain is $X^{(0)}, X^{(1)}, \dots$ where $X^{(t)}$ obeys the *Markov property* that

$$P \left[X^{(t)} \mid x^{(0)}, \dots, x^{(t-1)} \right] = P \left[X^{(t)} \mid x^{(t-1)} \right]$$

2. A Markov chain is governed by a *transition probability matrix*, P , with the ij th element equaling $P \left[X^{(t+1)} = j \mid X^{(t)} = i \right]$.

3. Let π denote a vector of probabilities with i th element π_i denoting the marginal probability that $X^{(t)} = i$. Then the marginal distribution of $X^{(t+1)}$ must be $\pi^T P$.

If $\pi^T P = \pi^T$ then π is a *stationary distribution* for P .

4. A Markov chain is *irreducible* if any state j can be reached from any state i in a finite number of steps for all i and j .
5. A Markov chain is *periodic* if it can visit certain portions of the state space only at regularly spaced intervals.

Brief review of Markov chains: Stationary distribution

If $X^{(1)}, X^{(2)}, \dots$ are realizations from an irreducible and aperiodic Markov chain with stationary distribution π , then

1. $X^{(n)}$ converges in distribution to the distribution given by π
2. For any function h

$$\frac{1}{n} \sum_{t=1}^n h(X^{(t)}) \rightarrow E_{\pi}\{h(X)\}$$

almost surely as $n \rightarrow \infty$, provided $E_{\pi}\{|h(X)|\}$ exists.

This is one form of the *ergodic theorem*, which is a generalization of the strong law of large numbers.

Markov chain Monte Carlo

- Let $\{\mathbf{X}^{(t)}\}$ denote a Markov chain for $t = 0, 1, 2, \dots$, where $\mathbf{X}^{(t)} = (X_1^{(t)}, \dots, X_p^{(t)})$ and the state space is either continuous or discrete.
- The MCMC sampling strategy is to construct an irreducible, aperiodic Markov chain for which the stationary distribution equals the target distribution, f .
- For sufficiently large t , a realization, $\mathbf{X}^{(t)}$, from this chain will have approximate marginal distribution f .
- There are various strategies for constructing a MCMC algorithm including the Metropolis-Hastings algorithm, the Gibbs sampler, etc.
- A popular application of MCMC methods is to facilitate Bayesian inference where f is a Bayesian posterior distribution for parameters $\mathbf{X}$

Metropolis-Hastings algorithm

The method begins at $t = 0$ with the selection of $\mathbf{X}^{(0)} = \mathbf{x}^{(0)}$ drawn at random from some starting distribution, g , with the requirement that $f(\mathbf{x}^{(0)}) > 0$.

Given $\mathbf{X}^{(t)} = \mathbf{x}^{(t)}$, the algorithm generates $\mathbf{X}^{(t+1)}$ as follows.

1. Sample a candidate value, $\mathbf{X}^*$, from a *proposal distribution* $g(\cdot | \mathbf{x}^{(t)})$.
2. Compute the *Metropolis-Hastings ratio*, $R(\mathbf{x}^{(t)}, \mathbf{X}^*)$, where

$$R(\mathbf{u}, \mathbf{v}) = \frac{f(\mathbf{v}) g(\mathbf{u} | \mathbf{v})}{f(\mathbf{u}) g(\mathbf{v} | \mathbf{u})}. \quad (92)$$

3. Sample a value for $\mathbf{X}^{(t+1)}$ according to the following:

$$\mathbf{X}^{(t+1)} = \begin{cases} \mathbf{X}^* & \text{with probability } \min\{R(\mathbf{x}^{(t)}, \mathbf{X}^*), 1\} \\ \mathbf{x}^{(t)} & \text{otherwise.} \end{cases} \quad (93)$$

4. Increment t and return to step 1.

Metropolis-Hastings algorithm

A few details:

- If the proposal distribution is chosen so that the chain is irreducible and aperiodic, then the unique limiting stationary distribution of the chain generated by the Metropolis-Hastings algorithm is the target distribution.
- Sequence $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots$ will likely include multiple copies of some points in the state space.

It is important to include these copies in the chain and in any estimates based on the output, since the frequencies of sampled points are used to correct for the fact that the proposal density differs from the target density.

Metropolis-Hastings algorithm

The distribution of realizations from the Metropolis-Hastings chain approximates the stationary distribution of the chain as t progresses, therefore

$$\mathbf{E} \{h(\mathbf{X})\} \approx \frac{1}{n} \sum_{i=1}^n h(\mathbf{x}^{(i)})$$

Some useful quantities that can be estimated include

- means: $\mathbf{E} \{h(\mathbf{X})\}$
- variances: $\mathbf{E} \{ (h(\mathbf{X}) - \mathbf{E} \{h(\mathbf{X})\})^2 \}$
- tail probabilities: $\mathbf{E} \{ 1_{\{h(\mathbf{X}) \leq q\}} \}$ for constant q , where $1_{\{A\}} = 1$ if A is true and 0 otherwise
- Virtually any other statistic that may be of interest!

Can also estimate f itself using density estimation methods

Choosing a proposal distribution

A well chosen proposal distribution produces

- candidate values that cover the support of the stationary distribution in a reasonable number of iterations
- candidate values that are not accepted or rejected too frequently

Both of these factors are related to the spread of the proposal distribution:

- Too diffuse: candidate values will be rejected frequently and the chain will require many iterations to adequately explore the space of the target distribution
- Too focused: the chain will remain in one small region of the target distribution for many iterations while other regions of the target distribution will not be adequately explored

Independence chains

Suppose that the proposal distribution for the Metropolis-Hastings algorithm is chosen such that for some fixed density g

$$g\left(\mathbf{x}^* | \mathbf{x}^{(t)}\right) = g(\mathbf{x}^*)$$

This yields an independence chain, where each candidate value is drawn independently of the past.

In this case, the Metropolis-Hastings ratio is

$$R(\mathbf{x}^{(t)}, \mathbf{X}^*) = \frac{f(\mathbf{X}^*) g(\mathbf{x}^{(t)})}{f(\mathbf{x}^{(t)}) g(\mathbf{X}^*)}$$

The resulting Markov chain is irreducible and aperiodic if $g(\mathbf{x}) > 0$ whenever $f(\mathbf{x}) > 0$.

Bayesian inference and MCMC

In the Bayesian paradigm, parameters are assumed to follow some probability distribution. Suppose that data $\mathbf{Y}$ has a distribution parameterized by θ .

Bayes Theorem:

$$\begin{aligned} p(\theta|\mathbf{y}) &= \frac{p(\theta)p(\mathbf{y}|\theta)}{\int p(\theta)p(\mathbf{y}|\theta) d\theta} \\ &= \frac{p(\theta)p(\mathbf{y}|\theta)}{p(\mathbf{y})} \\ &\propto p(\theta)p(\mathbf{y}|\theta) \end{aligned}$$

where

- $p(\theta|\mathbf{y})$ is the *posterior density* of θ
- $p(\mathbf{y}|\theta)$ is the *likelihood* or *sampling distribution*
- $p(\theta)$ is the *prior density*

In Bayesian applications we use MCMC to simulate from the posterior distribution $p(\theta|\mathbf{y})$, so the target distribution is typically the posterior distribution.

Bayesian inference and MCMC

Simple strategy: use the prior as a proposal distribution in an independence chain.

In our Metropolis-Hastings notation,

$$f(\boldsymbol{\theta}) = p(\boldsymbol{\theta}|\mathbf{y})$$

and

$$g(\boldsymbol{\theta}^*) = p(\boldsymbol{\theta}^*).$$

Then the Metropolis-Hastings ratio is given by:

$$R(\boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}^*) = \frac{p(\mathbf{y}|\boldsymbol{\theta}^*)}{p(\mathbf{y}|\boldsymbol{\theta}^{(t)})}$$

Comments:

- If proposal distribution= prior distribution, the Metropolis-Hastings ratio equals the likelihood ratio.
- By definition, the support of the prior covers the support of the target posterior, so the stationary distribution of this chain is the desired posterior.
- More sophisticated algorithms often have better performance.

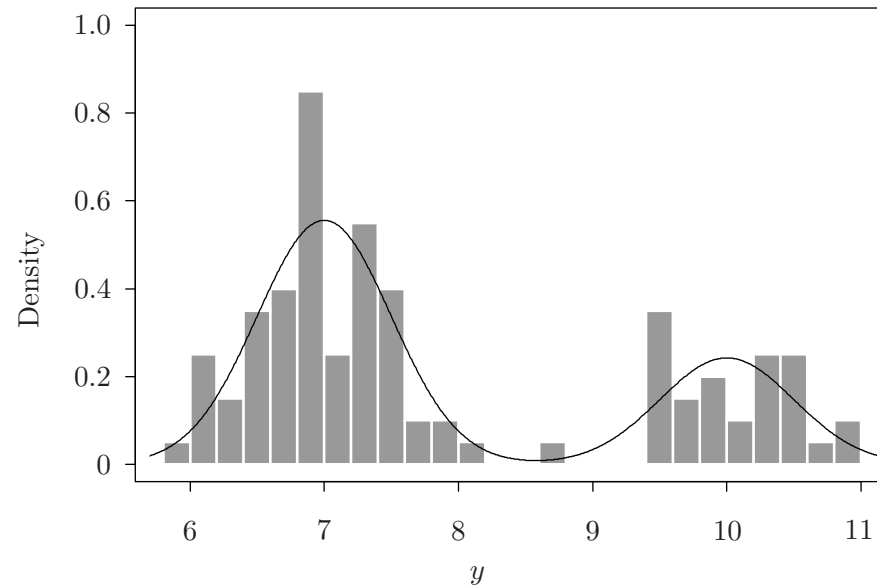
MCMC Example 1: Estimating a mixture parameter

Suppose we have observed data $y_1, y_2, \dots, y_{100}$ sampled independently and identically distributed from the mixture distribution

$$\delta N(7, 0.5^2) + (1 - \delta)N(10, 0.5^2) \quad (94)$$

- Mixture densities are common in real-life applications where, for example, the data may come from more than one population
- We will use MCMC techniques to construct a chain whose stationary distribution equals the posterior density of δ assuming a $\text{Unif}(0,1)$ prior distribution for δ

MCMC Example 1: Estimating a mixture parameter



Histogram of 100 observations simulated from the mixture distribution (94). The data were generated with $\delta = 0.7$, so we should find that the posterior density is concentrated in this area.

MCMC Example 1: Estimating a mixture parameter

Try two different MCMC proposal densities:

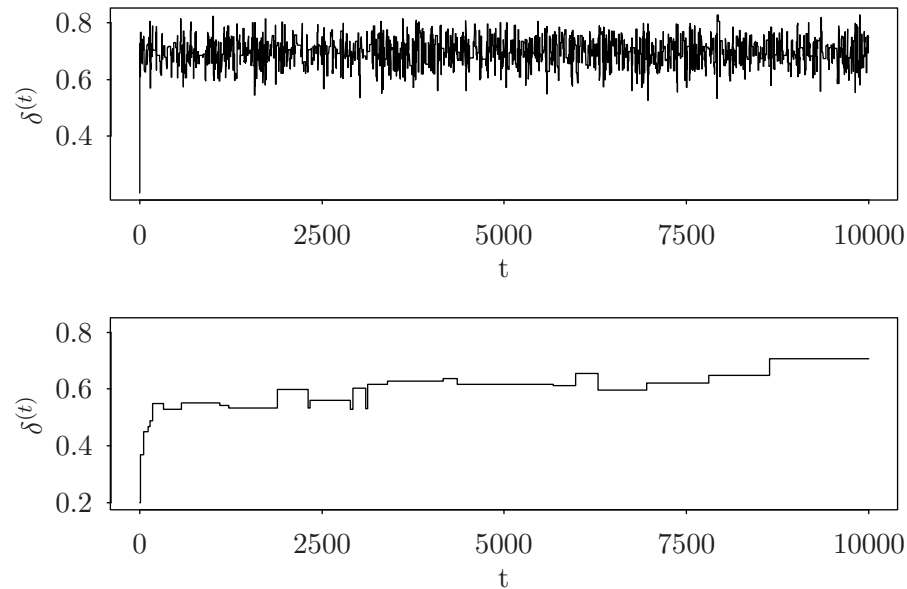
1. Beta(1,1): equivalent to a Unif(0,1) distribution
2. Beta(2,10):
 - skewed right with mean of 0.167
 - values of δ near 0.7 are unlikely to be generated from the proposal distribution

Generated 10,000 iterations of Metropolis-Hastings algorithm for each proposal.

Some output:

- Sample paths:
 - a plot of the chain realizations, $\delta^{(t)}$, against the iteration number, t
 - useful for investigating the behavior of the Markov chain
- Histogram of realizations

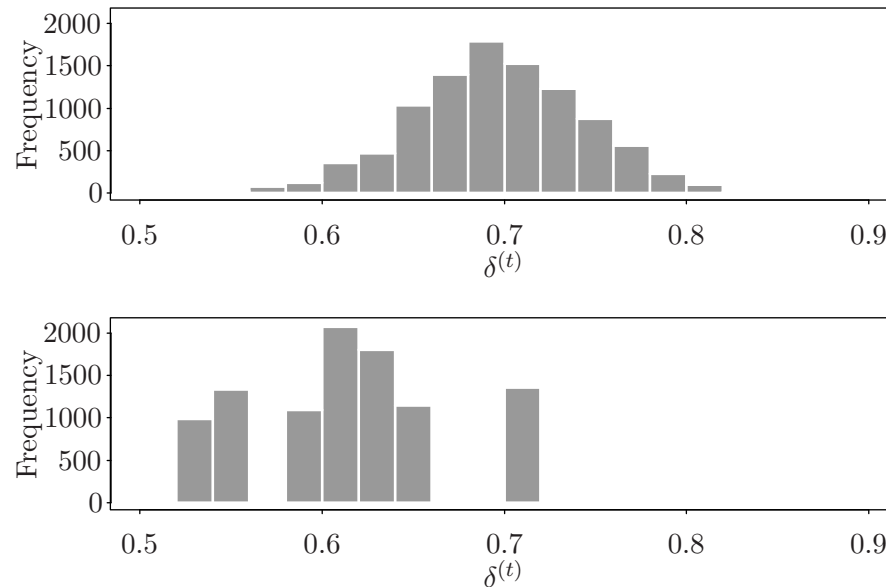
MCMC Example 1: Estimating a mixture parameter



Sample paths for δ from independence chains with proposal densities Beta(1,1) (top) and Beta(2,10) (bottom)

Burn-in period: Since initial iterates retain strong dependence on the starting value, eliminate them in estimation

MCMC Example 1: Estimating a mixture parameter



Histograms of $\delta^{(t)}$ for iterations 201-10,000 of independence chains with proposal densities Beta(1,1) (top) and Beta(2,10) (bottom)

Random walk chains

Random walk chain: Let $\mathbf{X}^*$ be generated by drawing

$$\boldsymbol{\epsilon} \sim h(\boldsymbol{\epsilon})$$

for some density h and then setting

$$\mathbf{X}^* = \mathbf{x}^{(t)} + \boldsymbol{\epsilon}.$$

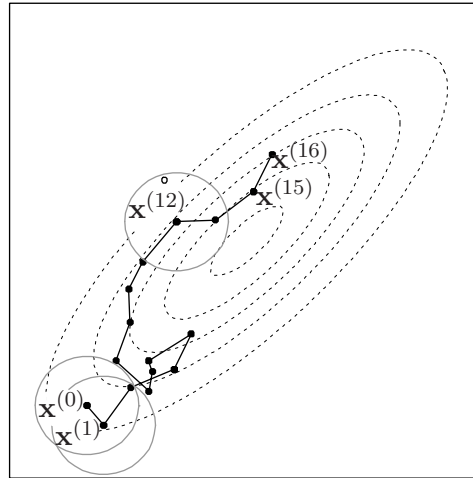
Note that the proposal is then given by

$$g\left(\mathbf{x}^* \mid \mathbf{x}^{(t)}\right) = h\left(\mathbf{x}^* - \mathbf{x}^{(t)}\right)$$

Common choices for h :

- uniform distribution over a ball centered at the origin
- scaled standard normal distribution
- scaled Student's t distribution

Random walk chains



Hypothetical random walk chain for sampling a two dimensional target distribution (dotted contours) using proposed increments sampled uniformly from a disk centered at the current value.

Gibbs Sampling

- Specifically adapted for multidimensional target distributions.
- Goal: still to construct a Markov chain whose stationary distribution—or some marginalization thereof—equals the target distribution, f
- Sequentially samples from univariate conditional distributions, which are often available in closed form

Basic Gibbs sampler

Suppose it is easy to sample from the univariate conditional distributions:

$$X_i | X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_p \sim f(x_i | x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_p)$$

A basic Gibbs sampler:

1. Select starting values $\mathbf{x}^{(0)}$ and set $t = 0$.
2. Generate, in turn,

$$\begin{aligned} X_1^{(t+1)} | \cdot &\sim f\left(x_1 | x_2^{(t)}, \dots, x_p^{(t)}\right) \\ X_2^{(t+1)} | \cdot &\sim f\left(x_2 | x_1^{(t+1)}, x_3^{(t)}, \dots, x_p^{(t)}\right) \\ &\vdots \\ X_{p-1}^{(t+1)} | \cdot &\sim f\left(x_{p-1} | x_1^{(t+1)}, x_2^{(t+1)}, \dots, x_{p-2}^{(t+1)}, x_p^{(t)}\right) \\ X_p^{(t+1)} | \cdot &\sim f\left(x_p | x_1^{(t+1)}, x_2^{(t+1)}, \dots, x_{p-1}^{(t+1)}\right) \end{aligned}$$

where $|\cdot$ denotes conditioning on the most recent updates to all other elements of $\mathbf{X}$.

3. Increment t and go to step 2.

Notes and enhancements to the Gibbs sampler

- The completion of step 2 for all components of $\mathbf{X}$ is called a *cycle*
- Can reorder the conditional distribution updates with each cycle
- Can block or group elements of $\mathbf{X}$ and update them jointly.

For example when $p = 4$, perhaps it is useful to consider

$$X_2^{(t+1)}, X_3^{(t+1)} \mid \cdot \sim f\left(x_2, x_3 \mid x_1^{(t+1)}, x_4^{(t)}\right)$$

- Can show Gibbs sampling is a Metropolis-Hastings algorithm with time inhomogeneous proposal distributions and with Metropolis-Hastings ratio R always equal to 1.

Hybrid Gibbs sampling

A *hybrid MCMC* algorithm might proceed with the following sequence of updates with $p = 6$:

1. Update $X_1^{(t+1)} \mid \left(x_2^{(t)}, x_3^{(t)}, x_4^{(t)}, x_5^{(t)}, x_6^{(t)} \right)$ with a Gibbs step.
2. Update $\left(X_2^{(t+1)}, X_3^{(t+1)} \right) \mid \left(x_1^{(t+1)}, x_4^{(t)}, x_5^{(t)}, x_6^{(t)} \right)$ with a Metropolis step.
3. Update $X_4^{(t+1)} \mid \left(x_1^{(t+1)}, x_2^{(t+1)}, x_3^{(t+1)}, x_5^{(t)}, x_6^{(t)} \right)$ with a step from a random walk chain.
4. Update $\left(X_5^{(t+1)}, X_6^{(t+1)} \right) \mid \left(x_1^{(t+1)}, x_2^{(t+1)}, x_3^{(t+1)}, x_4^{(t+1)} \right)$ with a Gibbs step.

The Metropolis-Hastings steps within a Gibbs algorithm are typically useful when the univariate conditional density for one or more elements of $\mathbf{X}$ is not available in closed form.

MCMC algorithm implementation

All MCMC algorithms described above have the correct limiting stationary distribution.

Mixing:

- How quickly does the chain forget its starting value?
- How quickly does the chain fully explore the support of the target distribution?
- How far apart in a sequence do observations need to be before they can be considered to be approximately independent?

Stationarity: has the chain has run sufficiently long so that

- it is reasonable to believe that the output adequately represents the target distribution?
- the output can be used reliably for estimation?

Ensuring good mixing and convergence

Strategies and diagnostics

1. Number of chains
2. Simple graphs to assess mixing and convergence
3. Reparameterization
4. Burn-in and run length

1. Number of chains

A difficult problem to diagnose: chain has become stuck in one or more modes of the target distribution.

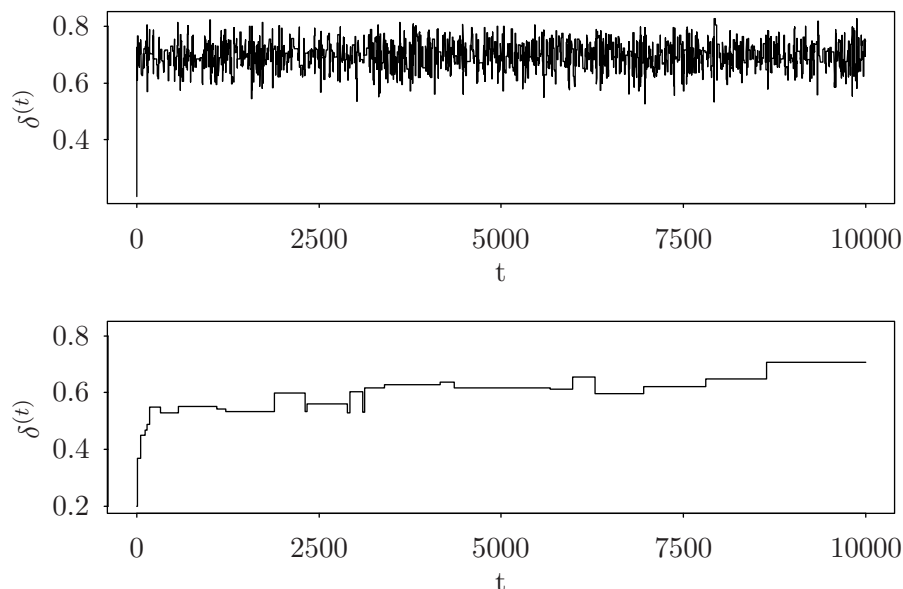
Partial solution: run multiple chains from diverse starting values and then compare the within- and between-chain behavior.

Diagnostics: sample path plots and Gelman-Rubin statistic

2. Simple graphs to assess mixing and convergence

Sample paths: plot of the iteration number versus the realizations of $X^{(t)}$ for $t = 0, 1, \dots$

- also called trace or history plots
- Want to see a very wiggly line that (perhaps) moves away from the starting value.



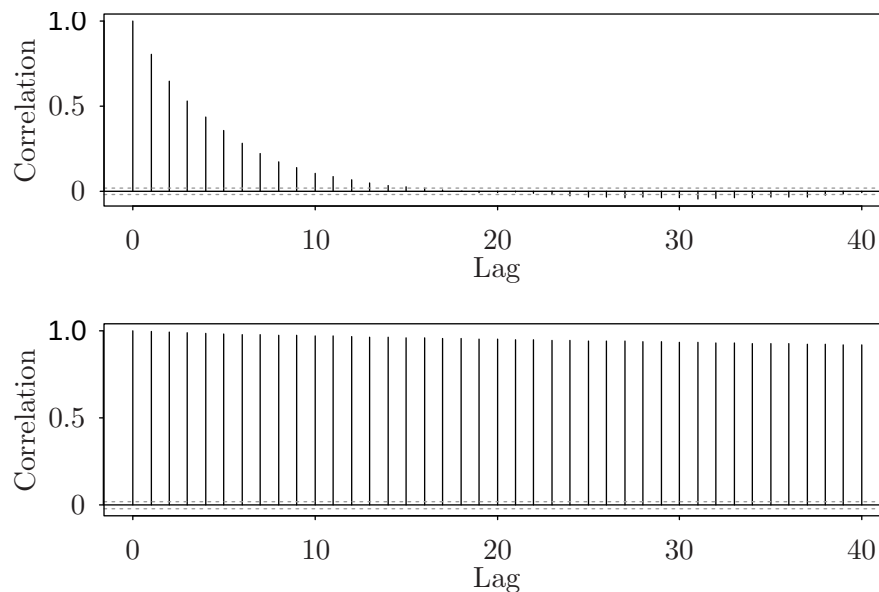
Sample paths for δ from independence chains with proposal densities Beta(1,1) (top) and Beta(2,10) (bottom)

2. Simple graphs to assess mixing and convergence

autocorrelation function (ACF) plot:

- A plot of the lag versus the correlation between iterates at that lag.
- Slow decay in the acf suggests poor mixing.
- Multi-parameter problems:
 - examine cross-correlations between parameters that might be related
 - High cross-correlations may indicate poor mixing of the chain

2. Simple graphs to assess mixing and convergence



Autocorrelation plots for example on estimating a mixture parameter with with proposal densities Beta(1,1) (top) and Beta(2,10) (bottom).

3. Reparameterization

- The mixing rates of both the Gibbs sampler and Metropolis-Hastings algorithms can be improved via reparameterization of the model.
- High correlation between the elements of $\mathbf{X}$ can lead to slow convergence. Reparameterization of the model can reduce the correlation and thus speed convergence.

Reparameterization approaches are typically adapted for specific models:

- If there are continuous covariates in a linear model, center and scale the covariates to reduce correlations between the parameters in the model.
- Hierarchical centering: parameters in a hierarchical Bayesian model are moved down the hierarchy. Particularly useful in models with random effects.

4. Burn-in and run length

Burn-in

- Initial MCMC iterates will not have exactly the correct marginal distribution, and the dependency on the initial point (or distribution) from which the chain was started may remain strong.
- To reduce the severity of this problem, the first several thousand values from the chain are typically discarded.

Gelman and Rubin statistic, R

- Compares the output of several chains
- If within and between chain variances are similar in magnitude, suggests that the chains are stationary
- Roughly: want $\sqrt{R} < 1.2$
- If not: increase run-length, reparameterize the model, etc.

4. Burn-in and run length: Gelman and Rubin statistic

Run the MCMC algorithm J separate times to produce equal-length chains ($J \geq 2$) with starting values dispersed over the support of the target density.

Let

- L denote the length of each chain after discarding D burn-in iterates.
- The variable of interest is X , and its value at the t th iteration of the j th chain is $x_j^{(t)}$.
- For the j th chain, the D values $x_j^{(0)}, \dots, x_j^{(D-1)}$ are discarded and the L values $x_j^{(D)}, \dots, x_j^{(D+L-1)}$ are retained.

4. Burn-in and run length: Gelman and Rubin statistic

Let

$$\bar{x}_j = \frac{1}{L} \sum_{t=D}^{D+L-1} x_j^{(t)} \text{ and } \bar{x}_{\cdot} = \frac{1}{J} \sum_{j=1}^J \bar{x}_j$$

and define the between-chain variance as

$$B = \frac{L}{J-1} \sum_{j=1}^J (\bar{x}_j - \bar{x}_{\cdot})^2$$

Now if the within-chain variance for the j th chain is

$$s_j^2 = \frac{1}{L-1} \sum_{t=D}^{D+L-1} \left(x_j^{(t)} - \bar{x}_j \right)^2, \text{ then let}$$

$$W = \frac{1}{J} \sum_{j=1}^J s_j^2$$

represent the mean of the J within-chain estimated variances.

Finally, let

$$R = \frac{\frac{L-1}{L}W + \frac{1}{L}B}{W}.$$

4. Burn-in and run length: Gelman and Rubin statistic

- If all the chains are stationary, then both the numerator and the denominator should estimate the marginal variance of X .
- If there are notable differences between the chains, then the numerator will exceed the denominator.
- As $L \rightarrow \infty$, $\sqrt{R} \rightarrow 1$.
- In practice, some authors suggest that $\sqrt{R} < 1.2$ is acceptable.

Remedies:

- If the chosen burn-in period did not yield an acceptable result, then D should be increased, L should be increased, or preferably both.
- If the $x_j^{(t)}$ iterates are transformed so that their distribution is approximately normal, the performance of this diagnostic is improved.
- Reparameterize the model and rerun the chain.

MCMC software

WinBUGS:

- WinBUGS is free software to carry out Gibbs sampling (www.mrc-bsu.cam.ac.uk/bugs/)
- Programming your own MCMC algorithm is the best way to learn what is going on in MCMC, but WinBUGS makes life a lot easier.
- Now available: code to run WinBUGS from within R

Other software available in R and other languages, but WinBUGS is the current popular choice

Output of MCMC procedures

Marginalization: If $\{\mathbf{X}^{(t)}\}$ represents a p -dimensional Markov chain, then the limiting distribution of $\{X_i^{(t)}\}$ is the i th marginal of f . If you are focused only on a property of this marginal, discard the rest of the simulation and analyze the realizations of $X_i^{(t)}$.

Mean: The most commonly used estimator is based on an empirical average. Discard the burn-in; then calculate the desired statistic by taking

$$\frac{1}{L} \sum_{t=D}^{D+L-1} h(\mathbf{X}^{(t)})$$

Standard deviation: sample standard deviation of realizations (after burn-in)

Probability estimates: The probability of any event can be estimated by the frequency of that event in the chain.

Output of MCMC procedures

Monte Carlo standard error: There are several ways to compute this.

Batch Method:

1. Separate the realizations into batches with, say, 50 consecutive iterations in each batch.
2. Compute the mean of each batch.
3. The estimated standard error is the standard deviation of these means divided by the square root of the number of batches.

Density/histogram:

- Histogram of the raw iterates.
- A kernel density estimate of the marginal posterior distributions.

Quantiles: Compute empirical quantiles

Other summary statistics: Be creative!

Practical implementation advice

Some rules of thumb:

- **Mixing:** For a Metropolis algorithm with normal target and proposal distributions, it has been suggested that a user should aim for acceptance rates of approximately
 - 45% for one- or two-dimensional problems
 - 23% for higher-dimensional problems
- **Burn-in lengths:** 0-50,000
- **Chain lengths:** 5,000-5,000,000
 - More samples means more accurate estimates of the posterior distribution.
 - One rule of thumb: the simulation should be run until the Monte Carlo error for each parameter of interest is less than 5% of the sample standard deviation.
 - Chain lengths will continue to increase as computing power grows.

MCMC Example 2:

Fur seal pup capture-recapture analysis

- After centuries of severe population reductions due to commercial and subsistence hunting, the abundance of fur seals in New Zealand has been increasing in recent years.
- Our goal is to estimate the number of pups in a fur seal colony using a capture-recapture approach.
- Estimation: can estimate population size and survival rates, etc.
- Consider, for example, high recapture rates suggest that the true population size does not greatly exceed the total number of unique individuals ever captured.

MCMC Example 2: Fur seal pups

Let

- N be the unknown population size to be estimated
- I be the number of census attempts
- $\mathbf{c} = (c_1, \dots, c_I)$ be the total number of captures (including recaptures) at each attempt
- r be the total number of distinct animals captured during the study

We assume that the population is closed during the period of the sampling which means that deaths, births and migrations are inconsequential during this period.

MCMC Example 2: Fur seal pups

- We consider a model with separate, unknown capture probabilities for each census effort, $\alpha = (\alpha_1, \dots, \alpha_I)$.
- This model assumes that all animals are equally catchable on any one capture occasion, but capture probabilities may change over time.

The likelihood for this model is

$$L(N, \boldsymbol{\alpha} | \mathbf{c}, r) \propto \frac{N!}{(N-r)!} \prod_{i=1}^I \alpha_i^{c_i} (1 - \alpha_i)^{N-c_i} \quad (95)$$

This model is sometimes called the $M(t)$ model

MCMC Example 2: Fur seal pups

- In a study conducted on the Otago Peninsula on the South Island of New Zealand, fur seal pups were marked and released during $I = 7$ census attempts during one season.
- It is reasonable to assume the population of pups was closed during the study period.
- $r = \sum_{i=1}^7 m_i = 84$ unique fur seals were observed during the sampling period.

Number of pups captured and recaptured during 7 census efforts in one season

		Census attempt, i						
		1	2	3	4	5	6	7
Number captured	c_i	30	22	29	26	31	32	35
Number newly caught	m_i	30	8	17	7	9	8	5

MCMC Example 2: Fur seal pups, priors

For estimation, one might adopt a hierarchical Bayesian framework where N and α are assumed to be a priori independent with the following priors:

- N : noninformative Jeffreys prior $f(N) \propto 1/N$
- capture probabilities: $f(\alpha_i | \theta_1, \theta_2) = \text{Beta}(\theta_1, \theta_2)$ for $i = 1, \dots, 7$, assumed to be a priori exchangeable

Previous analyses with the $M(t)$ model have indicated sensitivity to the prior distribution for the capture probabilities. To mitigate this sensitivity, we introduce a hyperprior for (θ_1, θ_2) :

$$f(\theta_1, \theta_2) \propto \exp \{ -(\theta_1 + \theta_2)/1000 \},$$

with (θ_1, θ_2) assumed to be a priori independent of the remaining parameters.

MCMC Example 2: Fur seal pups, Gibbs

Goal:

$$p(N, \alpha_1, \dots, \alpha_7, \theta_1, \theta_2 | \mathbf{c}, r) \propto p(\mathbf{c}, \mathbf{m} | N, \alpha_1, \dots, \alpha_7, \theta_1, \theta_2) \\ p(N) \left[\prod_{i=1}^7 p(\alpha_i | \theta_1, \theta_2) \right] p(\theta_1, \theta_2)$$

A Gibbs sampler can then be constructed by simulating from the conditional posterior distributions

$$N - 84 | \cdot \sim \mathbf{NegBin} \left(84, 1 - \prod_{i=1}^7 (1 - \alpha_i) \right) \\ \alpha_i | \cdot \sim \mathbf{Beta}(c_i + \theta_1, N - c_i + \theta_2) \quad \text{for } i = 1, \dots, 7 \\ \theta_1, \theta_2 | \cdot \sim k \left[\frac{\Gamma(\theta_1 + \theta_2)}{\Gamma(\theta_1)\Gamma(\theta_2)} \right]^7 \prod_{i=1}^7 \alpha_i^{\theta_1} (1 - \alpha_i)^{\theta_2} \exp\{-(\theta_1 + \theta_2)/1000\}$$

where $|\cdot$ denotes conditioning on the remaining parameters and the data, and k is an unknown constant.

MCMC Example 2: Fur seal pups, Gibbs

Problem: it is difficult to produce a chain for (θ_1, θ_2) with adequate mixing and convergence behavior.

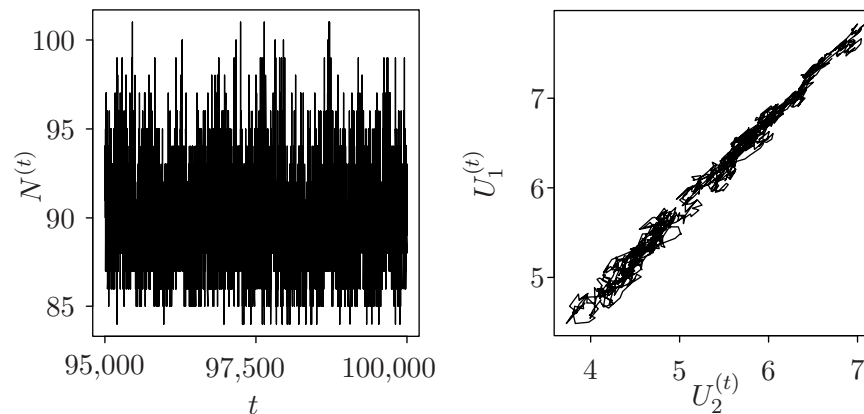
To improve performance, transform (θ_1, θ_2) to

$$\mathbf{U} = (U_1, U_2) = (\log \theta_1, \log \theta_2).$$

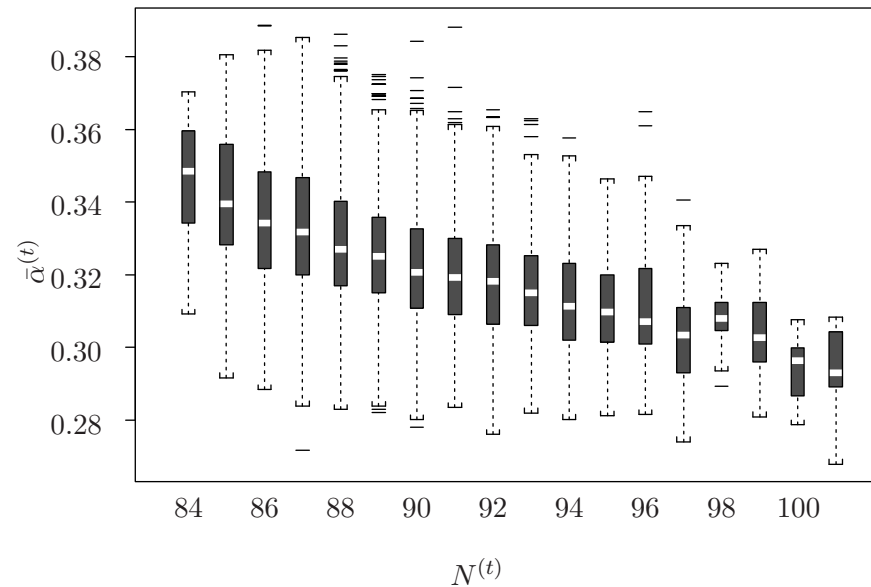
This requires transformation of the conditional densities.

MCMC Example 2: Fur seal pups, Gibbs

- The results below are based on a chain of 100,000 iterations with the first 1,000 iterations discarded for burn-in.
- Based on five runs of 100,000 iterations each, the Gelman and Rubin statistic for N is equal to 1.00047 which suggests the $N^{(t)}$ chain is roughly stationary.
- Sample paths for N (left panel) and U (right panel) for final 5,000 iterations in the seal pup example.

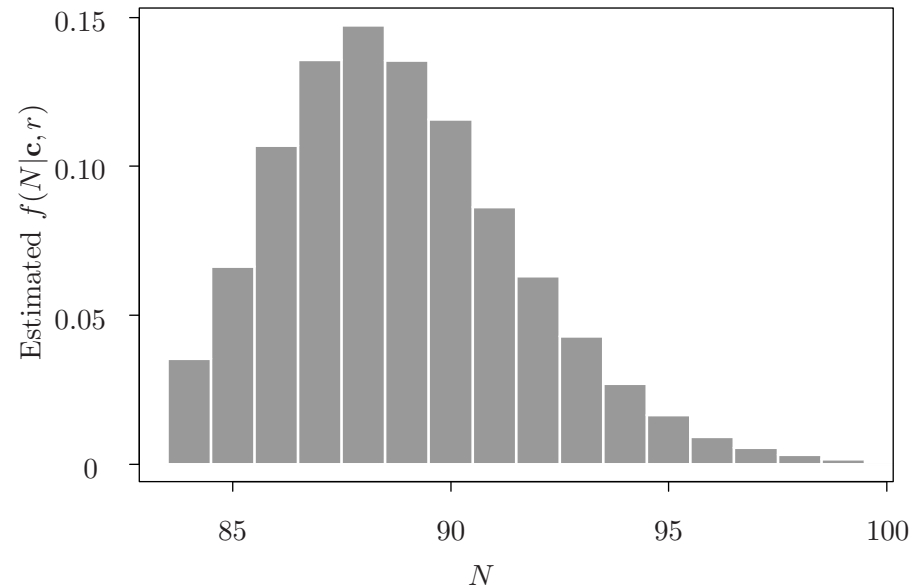


MCMC Example 2: Fur seal pups, Gibbs



Split boxplot of $\bar{\alpha}^{(t)}$ against $N^{(t)}$ for the seal pup example.

MCMC Example 2: Fur seal pups, Gibbs



Estimated marginal posterior probabilities for N for the seal pup example.

The posterior mean of N is 90 with a 95% highest posterior density interval of (84, 95).

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration

8. Markov Chain Monte Carlo Methods

9. Advanced MCMC Methods

Slice sampling and other auxiliary variable methods, Reversible jump MCMC, Perfect sampling, Langevin Metropolis-Hastings Algorithm, Hit-and-run algorithm, Multiple-try Metropolis-Hastings Algorithm

10. Integration: Concluding remarks

Slice sampling and other auxiliary variable methods

- In some cases standard MCMC methods can take too long to mix properly to be of practical use
- One remedy: augment the state space of the variable of interest
- Auxiliary variable methods can lead to chains which mix faster and require less tuning than standard MCMC methods

Consider a target distribution f which can be evaluated but not easily sampled. Construct an auxiliary variable algorithm as follows:

1. Augment the state space of $\mathbf{X}$ by the state space of a vector of auxiliary variables, $\mathbf{U}$.
2. Construct a Markov chain over the joint state space of $(\mathbf{X}, \mathbf{U})$ having stationary distribution $(\mathbf{X}, \mathbf{U}) \sim f(\mathbf{x}, \mathbf{u})$ which marginalizes to the target $f(\mathbf{x})$.
3. When simulation has been completed, discard $\mathbf{U}$ realizations and base inferences only on the marginal distribution of $\mathbf{X}$.

Slice sampling

Consider MCMC for a univariate variable $X \sim f(x)$, and suppose that it is impossible to sample directly from f .

- Introducing any univariate auxiliary variable U would allow us to consider a target density for $(X, U) \sim f(x, u)$.
- Writing $f(x, u) = f(x)f(u|x)$ suggests an auxiliary variable Gibbs sampling strategy that alternates between updates for X and U .
- The trick is to choose a U variable that speeds MCMC mixing for X .

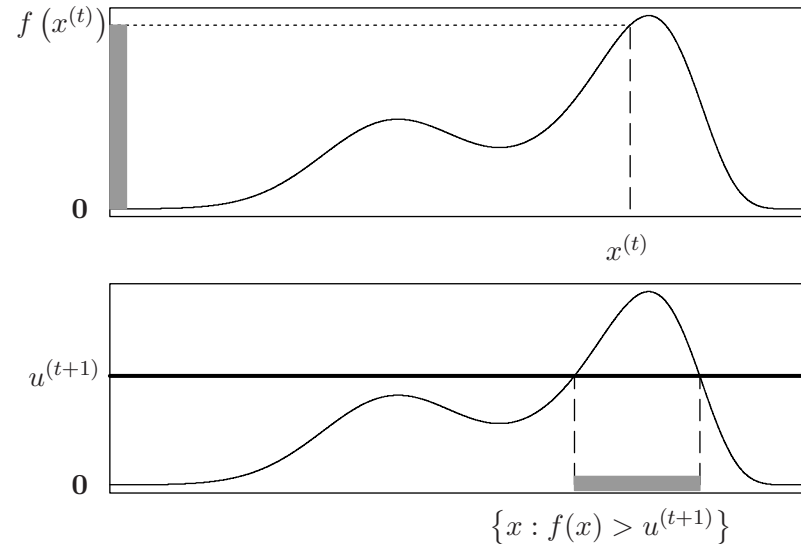
At iteration $t + 1$ of the slice sampler we alternately generate $X^{(t+1)}$ and $U^{(t+1)}$ according to

$$U^{(t+1)}|x^{(t)} \sim \text{Unif}\left(0, f\left(x^{(t)}\right)\right), \quad (96)$$

$$X^{(t+1)}|u^{(t+1)} \sim \text{Unif}\left\{x : f(x) \geq u^{(t+1)}\right\}. \quad (97)$$

The sampling in (97) can be challenging if f is not invertible. One approach to implementing equation (97) is to use rejection sampling.

Illustration of slice sampling

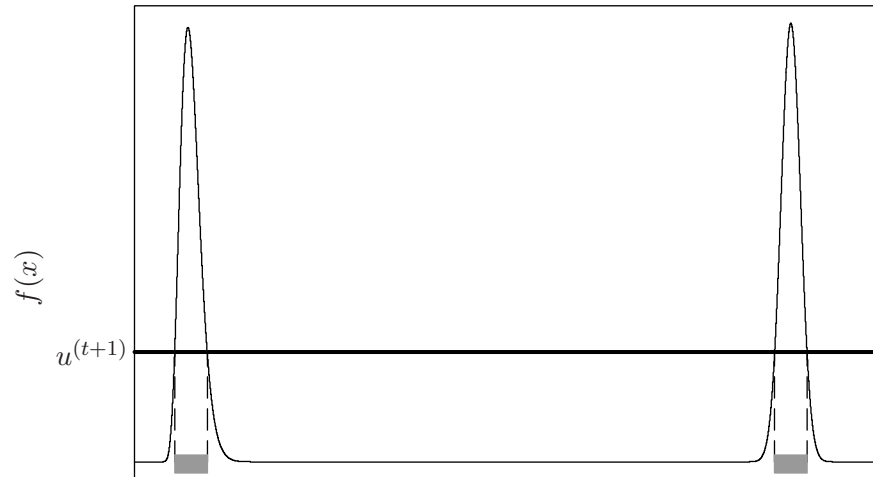


At iteration $t + 1$, the algorithm starts at $x^{(t)}$.

Upper panel: Draw $U^{(t+1)} \sim \text{Unif}(0, f(x^{(t)}))$. In the top panel this corresponds to sampling along the vertical shaded strip.

Lower panel: Draw $X^{(t+1)} | (U^{(t+1)} = u^{(t+1)})$ uniformly from the set of x values for which $f(x) \geq u^{(t+1)}$. In the lower panel this corresponds to sampling along the horizontal shaded strip.

Example: univariate multimodal target distribution



Generate samples from this multimodal target distribution using a

- **Standard Metropolis–Hastings algorithm:** algorithm may find one mode of the distribution, but it may take many iterations to find the other mode.
- **Slice sampler:** The horizontal shaded areas indicate the set defined in (97) from which $X^{(t+1)}|u^{(t+1)}$ is uniformly drawn. Hence the slice sampler will have about a 50% chance of switching modes each iteration. Therefore the slice sampler will mix much better with many fewer iterations required.

Reversible jump MCMC (RJMCMC)

- Standard MCMC algorithms require that the dimensionality of $\mathbf{X}^{(t)}$ and the interpretation of the elements of $\mathbf{X}^{(t)}$ do not change with t
- In many applications, it may be of interest to develop a chain that allows for changes in the dimension of the parameter space from one iteration to the next.
- Examples where RJMCMC methods are useful:
 - model selection
 - selection of the number of components in a mixture distribution
 - knot selection in nonparametric regression

RJMCMC: Notation

Consider constructing a Markov chain to explore a space of candidate models, each of which might be used to fit observed data y . Let

- $\mathcal{M}_1, \dots, \mathcal{M}_K$ = a collection of models under consideration.
- θ_m = parameters in the m th model.
- p_m = number of parameters in the m th model.
- In the Bayesian paradigm, we may envision random variables $\mathbf{X} = (M, \theta_M)$ which together index the model and parameterize inference for that model.

We may assign prior distributions to these parameters, then seek to simulate from their posterior distribution using a MCMC method for which the t th random draw is $\mathbf{X}^{(t)} = (M^{(t)}, \theta_{M^{(t)}}^{(t)})$.

- $\theta_{M^{(t)}}^{(t)}$ = parameters drawn for the model indexed by $M^{(t)}$
- $p_{M^{(t)}}$ = dimension $\theta_{M^{(t)}}^{(t)}$

RJMCMC: Overview

The goal of RJMCMC is to generate samples with joint posterior density

$$f(m, \boldsymbol{\theta}_m | \mathbf{y})$$

This posterior arises from Bayes' theorem via

$$f(m, \boldsymbol{\theta}_m | \mathbf{y}) \propto f(\mathbf{y} | m, \boldsymbol{\theta}_m) f(\boldsymbol{\theta}_m | m) f(m),$$

where

- $f(\mathbf{y} | m, \boldsymbol{\theta}_m)$ denotes the density of the observed data under the m th model and its parameters
- $f(\boldsymbol{\theta}_m | m)$ denotes the prior density for the parameters in the m th model
- A prior weight of $f(m)$ is assigned to the m th model with $\sum_{m=1}^K f(m) = 1$.

RJMCMC: Overview

The posterior factorization

$$f(m, \boldsymbol{\theta}_m | \mathbf{y}) = f(m | \mathbf{y}) f(\boldsymbol{\theta}_m | m, \mathbf{y})$$

suggests two important types of inference:

1. $f(m | \mathbf{y})$ is the posterior probability for the m th model, normalized over all models under consideration
2. $f(\boldsymbol{\theta}_m | m, \mathbf{y})$ is the posterior density of the parameters in the m th model.

RJMCMC enables the construction of an appropriate Markov chain for $\mathbf{X} = (M, \boldsymbol{\theta}_M)$ that jumps between models with parameter spaces of different dimensions.

The key to the RJMCMC algorithm is the introduction of auxiliary random variables at times t and $t + 1$ with dimensions chosen so that the augmented variables (namely $\mathbf{X}$ and the auxiliary variables) at times t and $t + 1$ have equal dimensions.

RJMCMC: Dimension matching

Consider a problem with two models, 1 and 2, such that:

- Model $\mathcal{M}_1$ has p_1 parameters
- Model $\mathcal{M}_2$ has p_2 parameters
- $p_2 > p_1$

Moving between models:

- Proposing a move from $\mathcal{M}_1$ to $\mathcal{M}_2$:

Generate θ_2 from an invertible deterministic function of both θ_1 and an independent random component U_1 . We can write

$$\theta_2 = \mathbf{q}_{1,2}(\theta_1, U_1)$$

- Proposing a move from $\mathcal{M}_2$ to $\mathcal{M}_1$:

Carry out via the inverse transformation, $(\theta_1, U_1) = \mathbf{q}_{1,2}^{-1}(\theta_2) = \mathbf{q}_{2,1}(\theta_2)$. Note that $\mathbf{q}_{2,1}$ is an entirely deterministic way to propose θ_1 from a given θ_2 .

RJMCMC: The algorithm

Assume that the chain is currently visiting model $m^{(t)}$, so the chain is in the state $\mathbf{x}^{(t)} = \left(m^{(t)}, \boldsymbol{\theta}_{m^{(t)}}^{(t)}\right)$. The next iteration of the RJMCMC algorithm can be summarized as follows:

1. Sample a candidate model $M^*|m^{(t)}$ from a proposal density with conditional density $g(\cdot|m^{(t)})$. The candidate model requires parameters $\boldsymbol{\theta}_{M^*}$ of dimension p_{M^*} .
2. Given $M^* = m^*$, generate an augmenting variable $\mathbf{U}|(m^{(t)}, \boldsymbol{\theta}_{m^{(t)}}^{(t)}, m^*)$ from a proposal distribution with density $h(\cdot|m^{(t)}, \boldsymbol{\theta}_{m^{(t)}}^{(t)}, m^*)$. Let

$$(\boldsymbol{\theta}_{m^*}^*, \mathbf{U}^*) = \mathbf{q}_{t,*}\left(\boldsymbol{\theta}_{m^{(t)}}^{(t)}, \mathbf{U}\right),$$

where $\mathbf{q}_{t,*}$ is an invertible mapping from $\left(\boldsymbol{\theta}_{m^{(t)}}^{(t)}, \mathbf{U}\right)$ to $(\boldsymbol{\theta}_{M^*}^*, \mathbf{U}^*)$ and the auxiliary variables have dimensions satisfying $p_{m^{(t)}} + p_{\mathbf{U}} = p_{m^*} + p_{\mathbf{U}^*}$

RJMCMC: The algorithm

3. For a proposed model, $M^* = m^*$, and the corresponding proposed parameter values $\boldsymbol{\theta}_{m^*}^*$, compute the Metropolis–Hastings ratio given by

$$\frac{f(m^*, \boldsymbol{\theta}_{m^*}^* | \mathbf{y}) g(m^{(t)} | m^*) h(\mathbf{u}^* | m^*, \boldsymbol{\theta}_{m^*}^*, m^{(t)})}{f(m^{(t)}, \boldsymbol{\theta}_{m^{(t)}}^{(t)} | \mathbf{Y}) g(m^* | m^{(t)}) h(\mathbf{u} | m^{(t)}, \boldsymbol{\theta}_{m^{(t)}}^{(t)}, m^*)} |\mathbf{J}(t)|, \quad (98)$$

where

$$\mathbf{J}(t) = \frac{d\mathbf{q}_{t,*}(\boldsymbol{\theta}, \mathbf{u})}{d(\boldsymbol{\theta}, \mathbf{u})} \bigg|_{(\boldsymbol{\theta}, \mathbf{u}) = (\boldsymbol{\theta}_{m^{(t)}}^{(t)}, \mathbf{U})}. \quad (99)$$

Accept the move to the model M^* with probability equal to the minimum of 1 and the expression in (98). If the proposal is accepted, set $\mathbf{X}^{(t+1)} = (M^*, \boldsymbol{\theta}_{M^*}^*)$. Otherwise, reject the candidate draw and set $\mathbf{X}^{(t+1)} = \mathbf{x}^{(t)}$.

The last term in (98) is the absolute value of the determinant of the Jacobian matrix arising from the change of variables from $(\boldsymbol{\theta}_{m^{(t)}}^{(t)}, \mathbf{U})$ to $(\boldsymbol{\theta}_{m^*}^*, \mathbf{U}^*)$. If $p_{M^{(t)}} = p_{M^*}$, then (98) simplifies to the standard Metropolis–Hastings ratio (92).

4. Discard $\mathbf{U}$ and $\mathbf{U}^*$. Return to step 1.

RJMCMC: Simple example with two models

Consider a problem with $K = 2$ possible models:

- Model $\mathcal{M}_1$ has a one-dimensional parameter space $\theta_1 = \alpha$
- Model $\mathcal{M}_2$ has a two-dimensional parameter space $\theta_2 = (\beta, \gamma)$. Thus $p_1 = 1$ and $p_2 = 2$.

Let $m_1 = 1$ and $m_2 = 2$ (model indexes).

If the chain is currently in state $(1, \theta_1)$ and $\mathcal{M}_2$ is proposed:

A random variable $U \sim h(u|1, \theta_1, 2)$ is generated from some proposal density h . Let $\beta = \alpha - U$ and $\gamma = \alpha + U$, so $\mathbf{q}_{1,2}(\alpha, u) = (\alpha - u, \alpha + u)$ and $\left| \frac{d\mathbf{q}_{1,2}(\alpha, u)}{d(\alpha, u)} \right| = 2$.

For a proposed move from $\mathcal{M}_1$ to $\mathcal{M}_2$, the Metropolis–Hasting ratio (98) is equal to

$$\frac{f(2, \beta, \gamma | \mathbf{Y}) g(1 | 2)}{f(1, \alpha | \mathbf{Y}) g(2 | 1) h(u | 1, \theta_1, 2)} \times 2. \quad (100)$$

RJMCMC: Simple example with two models

If the chain is currently in state $(2, \theta_2)$ and $\mathcal{M}_1$ is proposed:

The inverse mapping is given by $(\alpha, u) = \mathbf{q}_{2,1}(\beta, \gamma) = \left(\frac{\beta+\gamma}{2}, \frac{\beta-\gamma}{2}\right)$. Therefore $\left|\frac{d\mathbf{q}_{2,1}(\beta, \gamma)}{d(\beta, \gamma)}\right| = \frac{1}{2}$, and U^* is not required to match dimensions. This transition is entirely deterministic, so we replace $h(u^*|2, \theta_2, 1)$ in (98) with 1.

For a proposed move from $\mathcal{M}_2$ to $\mathcal{M}_1$, the Metropolis–Hastings ratio equals the reciprocal of (100)

Challenges to implementing RJMCMC

- The number of dimensions can be enormous, so it can be critical to select an appropriate proposal distribution h and to construct efficient moves between the different dimensions of the model space.
- Diagnosis of convergence is difficult

Other advanced MCMC algorithms

Perfect Sampling

- Generates a chain that has exactly reached the stationary distribution
- Method has promise, but challenges in implementation

Langevin Metropolis-Hastings algorithm

- A random walk with drift. The drift favors proposals that move towards modes of the target distribution
- A random walk with drift can be generated using the proposal

$$\mathbf{X}^* = \mathbf{x}^{(t)} + \mathbf{d}^{(t)} + \sigma \boldsymbol{\epsilon}^{(t)}$$

where

$$\mathbf{d}^{(t)} = \left(\frac{\sigma^2}{2} \right) \frac{\partial \log f(\mathbf{x})}{\partial \mathbf{x}} \bigg|_{\mathbf{x}=\mathbf{x}^{(t)}}$$

and $\boldsymbol{\epsilon}^{(t)}$ is a p -dimensional standard normal random variable.

- The scalar σ is a tuning parameter whose fixed value is chosen by the user to control the magnitude of proposed steps.

Other advanced MCMC algorithms

Hit-and-run algorithm

- Proposed move away from $\mathbf{x}^{(t)}$ is generated in two stages:
 1. Choose a direction to move
 2. Choose a distance to move in the chosen direction
- Resulting Markov chain isn't time invariant
- Can be useful when the state space of $\mathbf{X}$ is sharply constrained

Multiple-try Metropolis-Hastings algorithm

Use when chain is slow to converge or trapped in a local mode of f

- Generate a large number of candidates and
- Choose among the candidates in a manner that ensures that the chain retains the correct limiting stationary distribution

Part II: Monte Carlo Integration Methods

6. Introduction to integration

7. Monte Carlo Integration

8. Markov Chain Monte Carlo Methods

9. Advanced MCMC Methods

10. Integration: Concluding remarks

Integration: Concluding remarks

- As with optimization, it is often the case that more than one tool can be used to solve the same problem.
- Don't use MCMC like a black box and automatically trust your first answer from one MCMC run. MCMC often requires problem-specific customization to achieve good mixing and convergence.
- Transformation or re-expression of the problem can vastly improve results, both for quadrature and Monte Carlo approaches.
- Innovations in the theory and practice of many of the methods discussed today continue at a rapid pace. Today we gave you the building blocks to understand a number of key methods that will be used as springboards for the next developments.